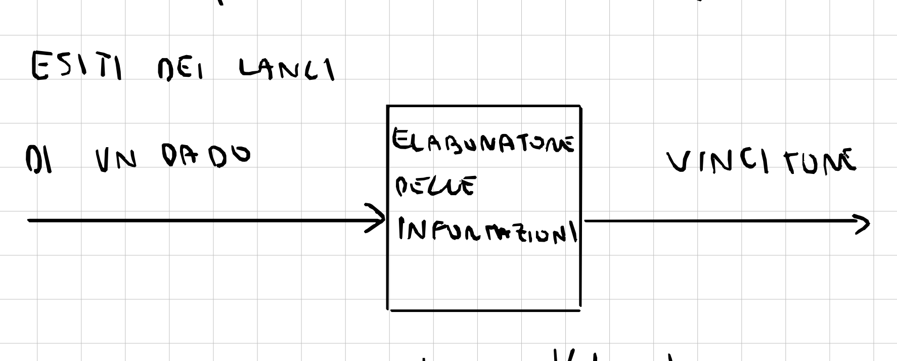
Introduzione

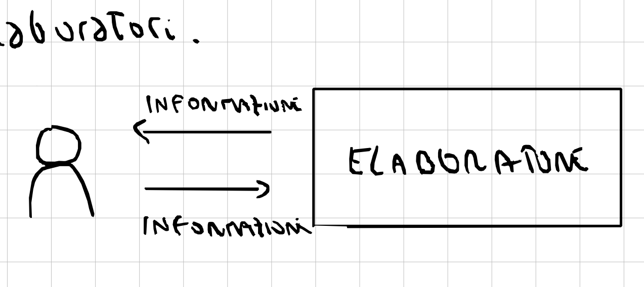
ESAME
- Prova di teoria (13-15 punti) da superare con almeno la metà
- Prova di progetto (18-20 punti) subito dopo la teoria, esercizi pratici

Questa materia tratta l'elaborazione delle informazioni.
Un'informazione è un'entità astratta in grado di arricchire la conoscenza.
Due possibilità rappresentano la "base minima" di informazione (es. 0 e 1, testa o croce, ...). L'informazione è più "preziosa" nel caso di più alternative.
L'informazione definisce una scelta da un insieme di possibili alternative. (esempi di informazione: 0 o 1 in un sistema binario, l'orario segnato da un orologio tra le 24·60 possibili alternative, ...).
L'informazione può essere elaborata per ricavare altre informazioni

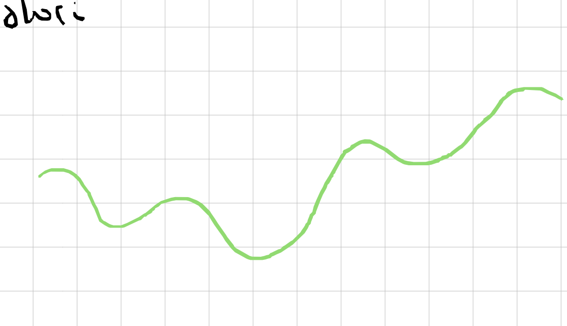
ES.



Studieremo come si realizza l'hardware su cui ragionano gli elaboratori.

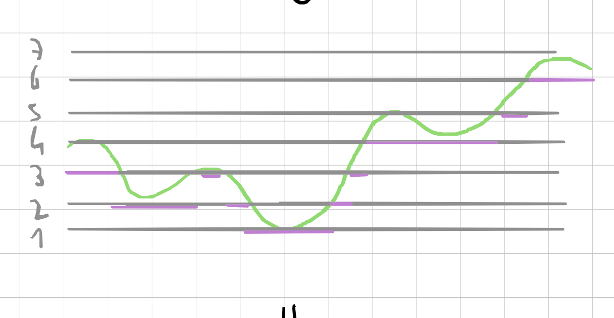


Il modo in cui rappresentiamo l'informazione è detto segnale, una grandezza fisica basata su fenomeni fisici misurabili, nel caso di elaboratori elettronici si comunica tramite segnali elettrici (basati ad esempio sulle onde elettromagnetiche).
Un segnale descrive il comportamento di una grandezza fisica, il cui valore varia nel tempo.
- Segnale analogico: ogni variazione di questa quantità "codifica" = rappresenta un segnale ∿∿
- Segnale digitale: solo alcune variazioni della grandezza fisica rappresentano informazioni diverse ⎍⎍ (es. solo quando si passa da un intervallo x a un intervallo y)
- Segnale binario: segnale digitale caratterizzato da due possibili valori



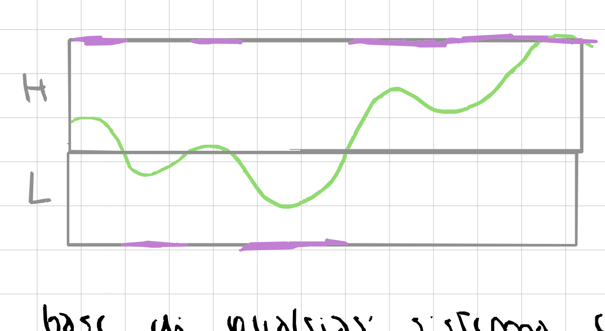
quantità continua
(segnale analogico)

⇓



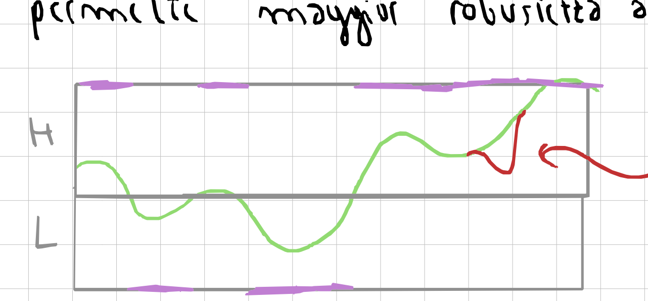
- segnale digitale si basa sempre su un segnale analogico pre-esistente!

⇓

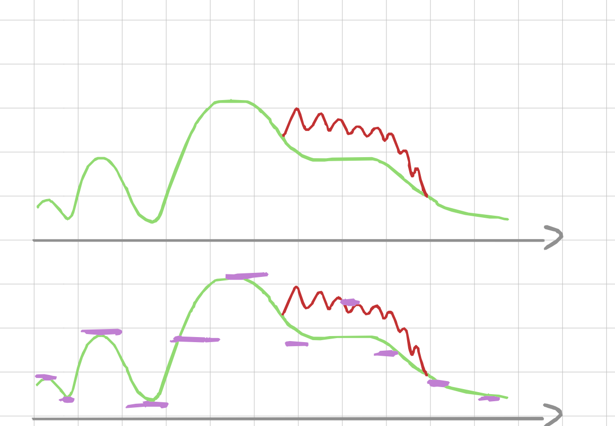


- segnale binario
Con un numero sufficiente di "intervalli binari" posso costruire qualsiasi segnale. È alla base di qualsiasi sistema elettronico. Un segnale analogico può dunque essere rappresentato da tanti segnali binari.

RUMORE
Qualsiasi tipo di fenomeno che altera il segnale che vogliamo trasmettere.
L'uso di segnali digitali in sostituzione a un segnale analogico permette maggior robustezza al rumore.



rumore che causa variazione di segnale
⇒ Il segnale digitale registra comunque high (H) (1)

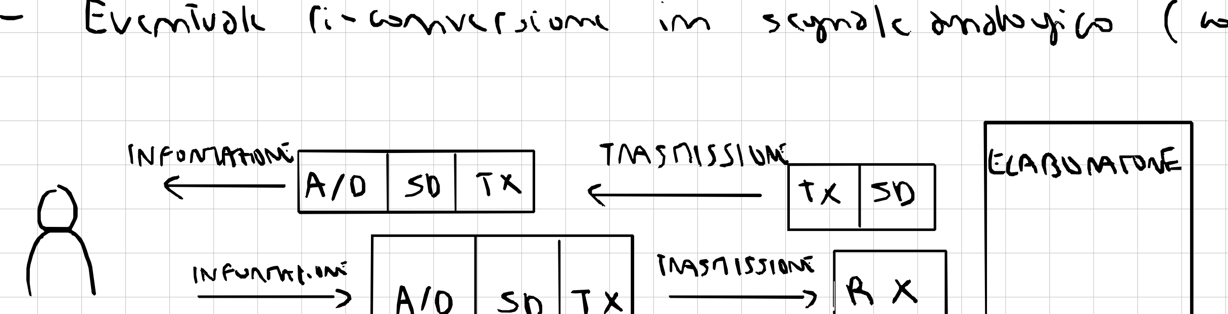


⇒ se il rumore è però troppo forte, il segnale digitale potrebbe comunque essere alterato.

Ne consegue che il segnale binario è il più robusto in assoluto (ovvio che per disturbi estremi il segnale finale risulta comunque disturbato)

ES: Se il colore è un segnale digitale, un pixel va a digitalizzare tale segnale come tre valori quantificati su una scala da 0 a 255

ELABORAZIONE DELLE INFORMAZIONI DI SISTEMI DIGITALI
- Digitalizzazione dell'informazione (conversione A/D)
- Trasmissione dell'informazione (es. dalla fotocamera al processore)
- Elaborazione da parte di uno o più sistemi digitali (SD)
- Eventuale ri-conversione in segnale analogico (conversione D/A)



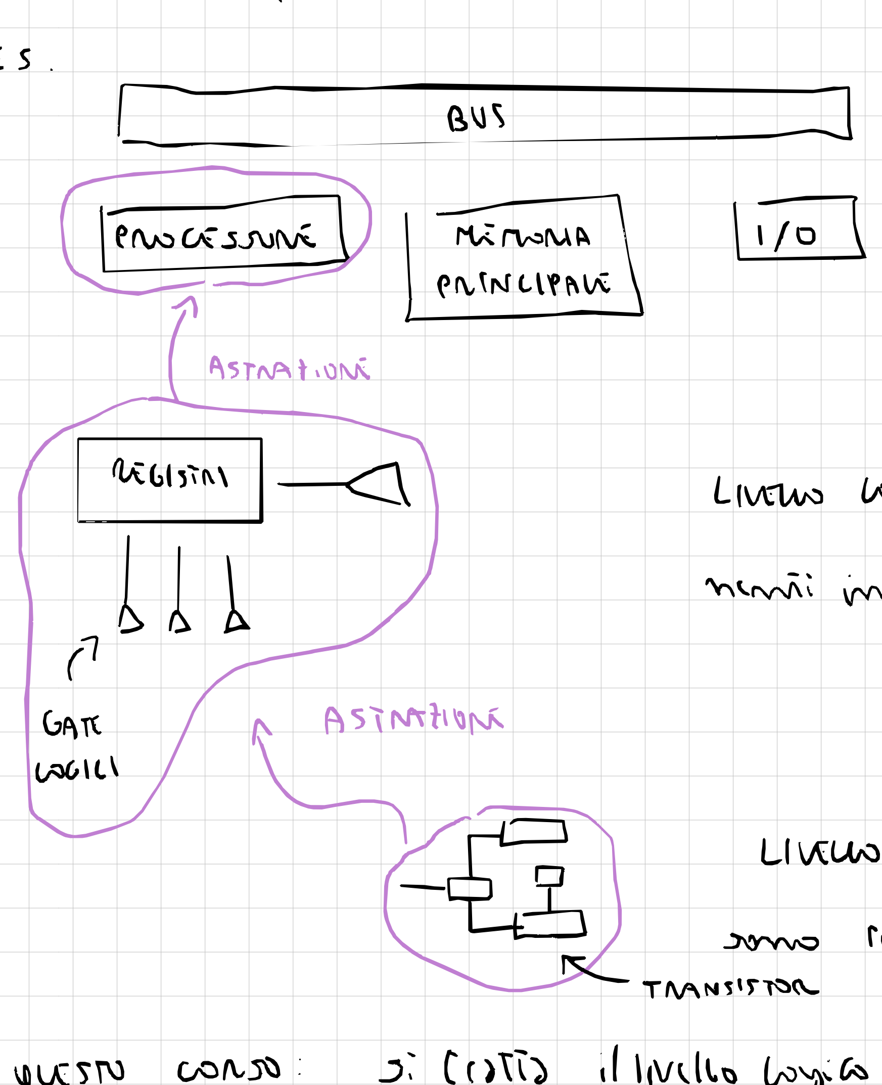
T = TRANSMIT
R = RECEIVE

Ovviamente si possono avere più informazioni e più trasmissioni.
Chi riceve l'informazione deve sapere esattamente come interpretare le informazioni ricevute.
ELABORARE INFORMAZIONI HA UN COSTO DI
- tempo (propagazione dei segnali, ecc.)
- energia (elettrica, meccanica, ...)
- spazio (superficie di silicio, spazio occupato in una stanza, ...)

Un sistema di elaborazione si definisce ottimo se impiega la minor quantità possibile di risorse
· non esistono sistemi ottimi in assoluto
· un sistema reale può essere ottimo rispetto a un singolo parametro, a discapito degli altri. (es. un segnale velocissimo, a discapito del consumo energetico)
· trade-off = compromesso tra i diversi parametri raggiunto da un certo sistema. (es. un segnale abbastanza veloce e un buon consumo energetico)

CPU, GPU, ecc. sono sistemi digitali. In questo corso ne studiamo i principi fondamentali.
I sistemi di elaborazione fisica (microprocessori, ecc.) sono in grado di processare segnali digitali.

RETI LOGICHE
Una rete logica è un modello astratto di una macchina digitale fisico. Una rete logica costituisce struttura e comportamento della macchina. Una rete logica NON descrive la realizzazione fisica della macchina. È appunto un progetto astratto (un modello astratto) di una macchina fisica.
ES. impareremo a progettare una rete logica che descrive un sommatore digitale (elaboratore che fa somme), che poi nella pratica può essere anche realizzato tramite componenti elettronici.
Una rete logica può a sua volta essere scomposta in componenti più semplici che la compongono o astratta in un'entità più generica che la rappresenta.
ES.



LIVELLO ARCHITETTONICO

LIVELLO LOGICO descrive come i sotto-componenti interagiscono tra loro

LIVELLO FISICO: come i componenti sono realizzati a livello fisico

QUESTO CORSO: si tratta il livello logico
CORSO DI CALCOLATORI ELETTRONICI: si tratta il livello fisico

DESCRIZIONE DEL COMPORTAMENTO (COME FA)
↓ SINTESI ↑ ANALISI
DESCRIZIONE DELLA STRUTTURA (COSA FA)

Lezione 17/04/25

DESCRIZIONE DEL COMPORTAMENTO (COSA FA)
↓ SINTESI ↑ ANALISI
DESCRIZIONE DELLA STRUTTURA (COM'È FATTA)

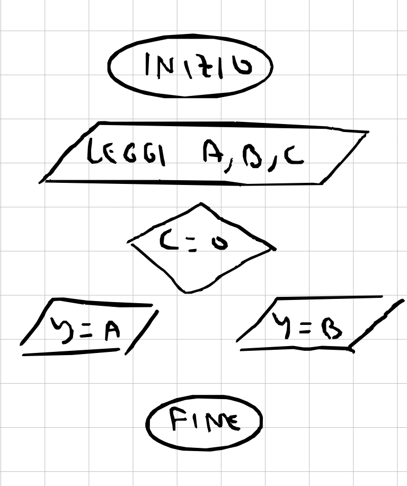
ANALISI: Comprendere il comportamento di una rete già esistente

SINTESI: Traduciamo il comportamento richiesto in una rete logica

COMPORTAMENTO DI UNA RETE LOGICA: definisce la relazione fra ingressi e uscite desiderate. Si può descrivere il comportamento tramite vari strumenti:
- testo
- diagrammi
- codice
- ...

Es. Multiplexer. Una rete M ha ingressi A,B,C e uscita y. Se C=0, l'uscita vale A, altrimenti B.

Possiamo rappresentare in vari modi questa rete



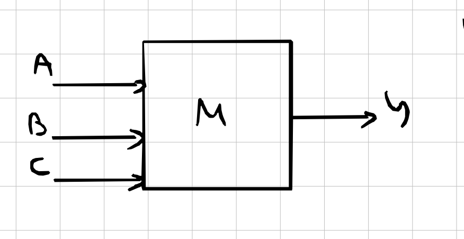
C	y
0	A
1	B

↳ TABELLA SEMPLIFICATA. LA TABELLA GIUSTA SAREBBE: (2^3 VARIABILI (A,B,C POSSONO ESSERE 0 o 1 => 2^3 possibilità)

A	B	C	y
0	0	1	0
0	1	0	0
0	0	0	0
1	0	0	1
0	1	1	1
1	0	1	0
1	1	0	1
1	1	1	1

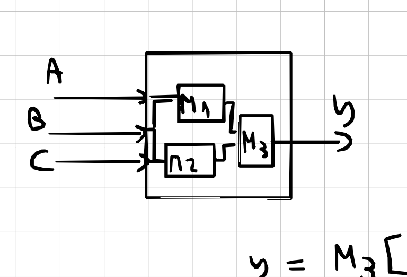
Capire se deve essere A,B o il loro valore

Modello black-box (close box)
=> non ne conosco le componenti



$y=M(A,B,C)$

Modello white-box (open-box)
=> ne esplicito le componenti



$y = M_3[M_1(A,C), M_2(B,C)]$

In realtà, a loro volta, M_1, M_2 e M_3 sono black-box ↗

COMPOSIZIONE: insieme di reti logiche che formano reti logiche

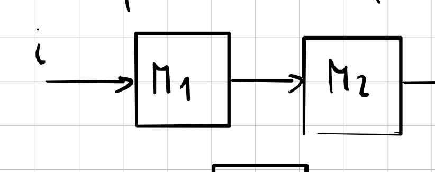
DECOMPOSIZIONE: una rete logica può essere decomposta in sotto-reti logiche. La decomposizione può avvenire fino al raggiungimento delle reti logiche atomiche, detti gate logici.

In realtà, M_1, M_2 e M_3 di un multiplexer sono gate logici.

I gate logici rappresentano operazioni elementari. Potremmo ad esempio combinare più multiplexer (composizione) per creare reti logiche più complesse.

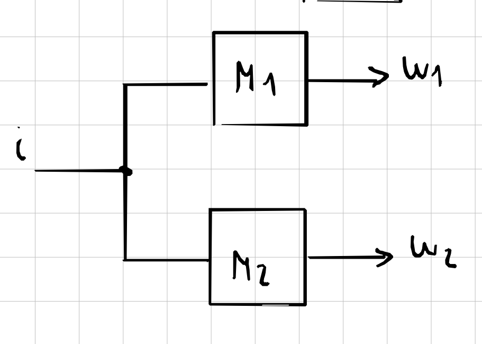
Esistono 3 tipi di composizione (M_1 e M_2 due reti)

- In serie



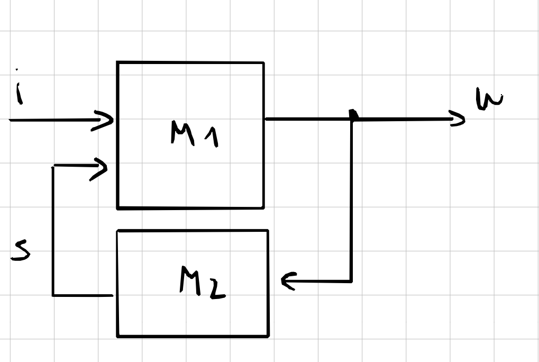
$u = M_2(M_1(i))$

- In parallelo



$$\begin{cases} w_1 = M_1(i) \\ w_2 = M_2(i) \end{cases}$$

- Ricorsione



$u = M_1(u, s)$

$s = M_2(u)$

$\Rightarrow u = M_1(i, M_2(u))$

Con la ricorsione stiamo implementando anche un meccanismo di memoria. M_1 processa sia l'ingresso sia ciò che è successo nelle iterazioni precedenti. s = stato. M_2 prende l'uscita elaborata da M_1 e la trasforma in un'informazione utile a M_1. Ogni volta che riceviamo un ingresso produciamo un'uscita e aggiorniamo lo stato (la memoria) della rete.

Si ricorda che nonostante le reti siano astrazioni, esse vengono implementate fisicamente tramite macchine. Nella realtà, qualsiasi elaborazione produce ritardo (τ) tra ingresso e uscita. Il modo in cui componiamo la rete impatta il ritardo.
- In serie $\tau = \tau_1 + \tau_2$
- In parallelo $\tau = MAX(\tau_1, \tau_2)$
- Ricorsione $\tau = ?$ perché M_1 e M_2 si influenzano a vicenda

Reti combinatorie: realizzate unicamente tramite composizione in serie o in parallelo => definiscono relazione univoca tra ingresso e uscita
=> non hanno stato

Reti sequenziali: realizzate mediante retroazioni (oltre a serie parallelo). i_1 e i_2 potrebbero creare u_1 e u_2 diversi per via della presa in considerazione dello stato. A seconda della complessità, una rete può anche tenere a memoria risultati di n iterazioni precedenti => relazioni non univoche tra ingressi e uscite

Rappresentare le informazioni

Qual'è la lingua che i sistemi digitali devono parlare per comunicare tra di loro? Ovvero come rappresentiamo l'informazione?

Un codice (linguaggio) che usa solo due simboli è detto binario. In generale, una stringa è un'informazione descritta da una sequenza univoca di simboli. Esempi di codici binari: 0 e 1, A e B, vero e falso, ...

Esempi di stringhe: 001, ABBAB, ...

Nel caso di codici binari, la singola cifra è definita bit. I sistemi digitali parlano in codice binario.

nibble = 4 bit

byte = 8 bit

Possiamo attribuire più significati a queste sequenze di bit.

Ad esempio, il codice ASCII è
- un codice a 7 bit => 128 combinazioni ($\overset{0 \text{ o } 1}{2^7}$)
  ↳ codice = linguaggio come detto prima

è un linguaggio basato sul codice (linguaggio) binario

capire bene la differenza tra combinazioni, configurazioni, ...

Un codice è detto ridondante se usa un numero m di combinazioni superiore alle possibili n configurazioni

Es. 00: nord 01: sud 11: ovest 10: est non ridondante

0000: nord 0001: sud 0011: ovest 0010: est ridondante

↑ 2^4 possibili combinazioni ma ne usiamo solo 4

Un codice ridondante non è per forza un male

Distanza fra configurazioni: 0010 e 1000 due cifre diverse => distanza 2

Nel caso di 4 bit con solo 4 combinazioni la distanza minima è 2.

La distanza è data solo dal numero di bit di differenza, NON dalla posizione di tali bit nella configurazione.

I codici descritti fin'ora non hanno valore numerico: 0 e 1 sono solo simboli, del tutto equivalenti ad A e B.

Esistono codici, detti sistemi numerici, i cui simboli hanno significato numerico (es. alfabeto romano)

Alfabeto romano => sistema numerico non posizionale (V è sempre 5 a prescindere dalla posizione)

Sistema numerico arabo => sistema numerico posizionale (5 rappresenta 5 unità, 5 decine, 5 centinaia, ... a seconda della posizione)

In un sistema posizionale è molto più semplice eseguire calcoli.

Un sistema posizionale è definito dal numero di simboli che lo compone
- sist. decimale: caratterizzato da 10 simboli, da 0 a 9

$2025 = 2 \cdot 10^3 + 0 \cdot 10^2 + 2 \cdot 10^1 + 5 \cdot 10^0 = 2025$

- in base b arbitraria

$2025_b = 2 \cdot b^3 + 2 \cdot b^2 + 2 \cdot b^1 + 2 \cdot b^0$

Dunque è necessario esplicitare la base per conoscere il significato numerico $2025_8 \neq 2025_{12}$ (2025 è una stringa)

Per progettare una rete logica è necessario conoscere l'algebra binaria

La base b di un sistema numerico definisce i numeri rappresentabili a parità di lunghezza della stringa. Es. base 10 => stringhe di lunghezza 4 rappresentiamo 10^4 numeri (da 0000 a 9999).

Se b=2 si parla di sistema numerico binario. Dunque con il codice binario possiamo decidere di non attribuire significato numerico ai simboli (es. ASCII) oppure farlo (sistema numerico binario - usato alla base di ogni sistema digitale).

n => numero massimo è $2^n - 1$

Lezione 23/04/25

Conversione binario decimale

$$\begin{matrix}1 & 1 & 0 & 1\\ 2^3+ & 2^2 & +0 & +2^0\end{matrix} \Rightarrow 13_{10}$$

↖ "in base 10"

Conversione decimale binario

13 : 2 = 6	r: 1	
6 : 2 = 3	r: 0	↑
3 : 2 = 1	r: 1	
1 : 2 = $\boxed{0}$	r: 1	

$\Rightarrow 1101_2$ ↖ in base 2

↑ ci si ferma a 0

Questo algoritmo vale per qualsiasi base rispetto alla base 10

SISTEMA ESADECIMALE

I simboli che lo compongono sono 0, ..., 9, A, B, C, D, E, F

(con A, B, C, D, E, F corrispondenti a valori numerici 11, 12, 13, 14, 15)

ES: h = base esadecimale

12_h

$1 \cdot 16^1 + 2 \cdot 16^0 = 18$

$\overset{\equiv A}{AD}$

$(10) \cdot 16^1 + 0 \cdot 16^0 = 160$

È utile perché $16 = 2^4$ ⇒ si possono rappresentare stringhe di bit in modo molto più compatto. Questo ci consente di convertire una stringa binaria in una stringa decimale in modo molto semplice

ES.

0010100101111010010110010101010

suddivido in nibbles

0 0101 0010 1111 0100 1011 0010 1010 = 052F4B2A

DECIMALE	BINARIO	ESADECIMALE
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	1001	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

I numeri esadecimali si usano ad esempio per rappresentare indirizzi sui computer, in quanto di facile conversione in binario e miglior leggibilità

OPERAZIONI ARITMETICHE TRA BINARI

Addizioni e sottrazioni si svolgono esattamente come nei decimali

```
2^5 2^4 2^3 2^2 2^1 2^0
     1   1   1   1
     1   1   0   1   0   ← 26 +
     0   0   1   1   1   ← 7 =
 1   0   0   0   0   1   ← 33
```

Per prodotti e divisioni invece, come nel caso decimale, si possono individuare algoritmi simili

```
      1 1 0 1 x      1 3
        1 0 0 =        4 =
      ---------
      0 0 0 0        5 2
    0 0 0 0 -
  1 1 0 1 - -
  -----------
  1 1 0 1 0 0
```

Divisione (da capire)

In ogni caso si basano sempre su somme e sottrazioni

Inoltre, nelle operazioni in cui compaiono potenze di 2, i calcoli si semplificano

1001 x 100 = 100100
11 x 1000 = 11 000

} aggiungo n zeri

11001 / 100 = "110,01" = 110 con resto 01, come nei decimali quando divido per 10

Con n bit, siamo in overflow se il numero di bit del risultato dell'operazione è > n

```
  1 1 0 1 0
  0 0 1 1 1
[1]0 0 0 0 1
```

↑ overflow ← sommatore a 5 bit

ci indica che il valore restituito dal sommatore non è corretto (gli manca un 1).

NUMERI BINARI CON SEGNO

Esistono due codifiche per rappresentare numeri negativi

- Segno - valore assoluto: il primo bit rappresenta il segno - (1) o +, il resto del numero è il valore assoluto

ES. $-37 = \underbrace{1}_{-}\underbrace{100101}_{37}$

- Complemento a 2: il primo bit codifica una quantità negativa, i rimanenti codificano quantità positive

$\underbrace{1}_{-64}\underbrace{011011}_{+27} = -37$

Utilizzeremo principalmente il complemento a 2

Proprietà: dato un numero binario N di n bit, il suo complemento a 2 è la quantità da sommare per ottenere 2^n

$^2N = 2^n - N$

si può anche ottenere l'opposto invertendo 0 e 1 e sommando +1 al valore ottenuto

$1011 \Rightarrow 0100 + 1 = 0101$

$\downarrow$ -0 $\downarrow$

$-8+3=-5$ ←——→ 5

opposti

Il metodo segno - valore assoluto ha il problema che 1000 e 0000 rappresentano entrambi 0 (a prescindere dal numero di 0).

Il complemento a 2 ha invece molti vantaggi, tra cui il fatto che somme e sottrazioni possono essere svolte mediante la sola addizione.

Siano A e B entrambi numeri in complemento a 2, si può calcolare A-B, con A > B, come A+B'+1 con B' = B invertito di uni e zeri

ES. -37+18

```
1011011 +   ← A
0010010 =   ← B
-------
1101101
```

→ B' ↗ 0010010 + 1 = 0010011 = -19

Anche con numeri in complemento a 2 si può avere overflow. Tuttavia, il risultato ottenuto dal calcolatore è incerto in termini di utilità di overflow. In alcuni casi l'overflow non ci dice nulla, in altri sì. Bisogna guardare gli ultimi due riporti, non solo l'ultimo

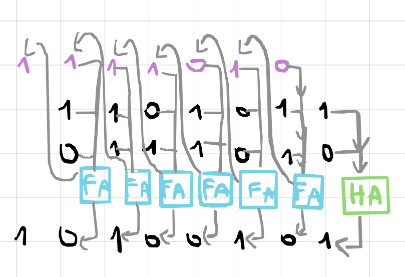
```
 1 1 1 1        1 0 0 0        0 0 0 0        0 1 1 1     riporti
   1 1 1 1 +      1 1 0 0 +      0 0 0 1 +      0 1 1 1 +
   0 1 1 1 =      1 0 0 0        0 1 1 0 =      0 1 1 1 =
 1 0 1 1 0      1 1 1 0 0      0 0 1 1 1      0 1 1 1 0
```

risultati restituiti dal calcolatore a 4 bit

- Ultimi 2 riporti diversi ⇒ overflow
- Ultimi 2 riporti uguali ⇒ no overflow (il risultato è fedele)

Il sommatore fa sempre la stessa cosa, siamo "noi" che interpretiamo il suo risultato in uno specifico modo.

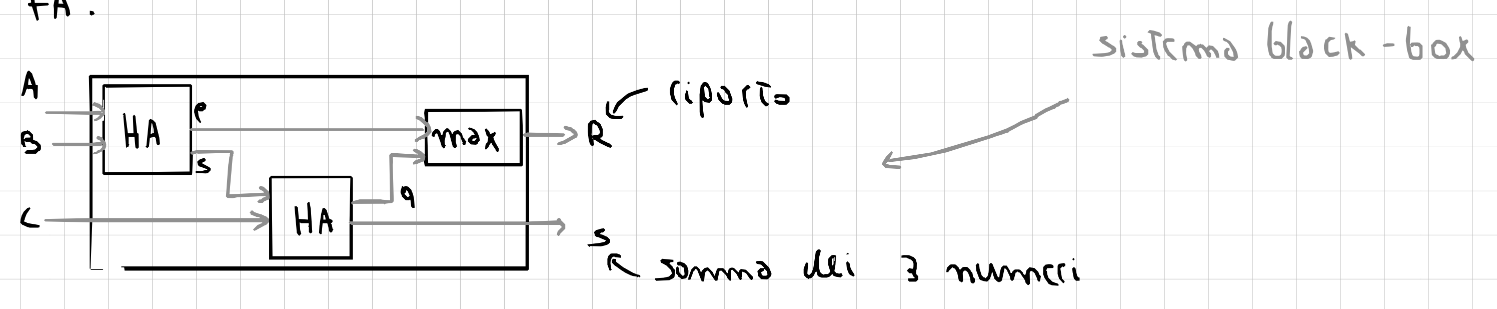
Lezione 24/04/25



Half-adder: rete che somma 2 bit

Full-adder: rete che somma 3 bit ⇒ formato da due HA

FA:



p e q riporti delle due somme ⇒ il resto è dato dal massimo tra i due riporti

ALTRI TIPI DI CODICE

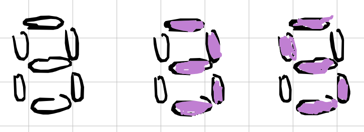
- Codice Gray: codice nel quale due configurazioni adiacenti hanno distanza 1. Ad esempio, 001, 101, 111, 011. Nel sist. binario "normale", la sequenza sarebbe 001, 010, ...

distanza 2 per queste due configurazioni adiacenti

Questa garanzia che ci dà il codice Gray risulta molto utile nella progettazione di alcune reti logiche, in quanto nel normale codice binario il passaggio (ad esempio) tra 001 e 010 implicherà un piccolissimo intervallo di tempo in cui ci sarà una configurazione errata, in quanto fisicamente è impossibile che i due bit cambino esattamente insieme. Codificando invece il segnale tramite codice Gray, evitiamo questo finto cambiamento simultaneo di bit, che in molti sistemi causa problemi (perché arriva un segnale in più, quello del primo bit che cambia, che implica un segnale indesiderato).

Si può realizzare una rete logica che presi n valori a n bit in codice Gray restituisce n valori a n bit in binario "normale" (o viceversa).

Codifica a 7 segmenti



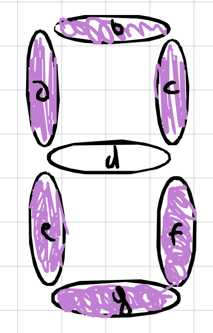
utilizzato nei vecchi display per rappresentare numeri binari sotto forma di numeri leggibili

		a	b	c	d	e	f	g
0	0000	1	1	1	0	1	1	1
1	0001							
2	0010							
3	0011							
4	0100							
5	0101							
6	0110							
7	0111							
8	1000							
9	1001							

configurazione binaria d'ingresso

configurazione binaria d'uscita

d spento =



Logica della rete logica che fa ciò molto difficile (deve descrivere la relazione tra i numeri binari da 0 a 10 e le corrispondenti stringhe rappresentative

Rete combinatoria = rete che trasforma una configurazione binaria in un'altra (non dipende dal tempo) (es. rete per la codifica a 7 segmenti, rete binario - Gray, ...)

Nota: nella cod. a 7 segmenti, il sistema binario non ha significato numerico

Cosa potrebbe chiedere all'esame di ciò fatto finora

Fare affidamento agli esami dell'ultimo anno, sono quelli fatti da lui, quelli più vecchi sono più tosti.

ESEMPIO DI ESERCIZI:

- conversione   PIÙ COMUNI:
  - complemento a 2 (2C) → decimale
  - segno - valore assoluto → decimale
  - esadecimale
- somma tra num. binari, Cout e overflow. Studia come vuole che venga compilata la tabella
- altro (domande aperte sul codice binario, prodotto/divisione per potenze di due, ...)

PROVA

1) binario → segno - valore assoluto

- 1111 = - 7
- 11111111 = - 127
- 0001 = 1
- 0000001 = 1
- 1000 = 0
- 1000000 =

2) Moltiplicazioni e si ricavi la regola generale

in 8 bit!

13 · 2	1101	26	00011010
13 · 4	1101	52	00110100
5 · 2	101	10	00001010
5 · 4	101	20	00010100
10 · 8	1010	80	01010000

Regola generale per la moltiplicazione per potenze di 2:

Aggiungere n zeri al numero N, con 2^n potenza di 2

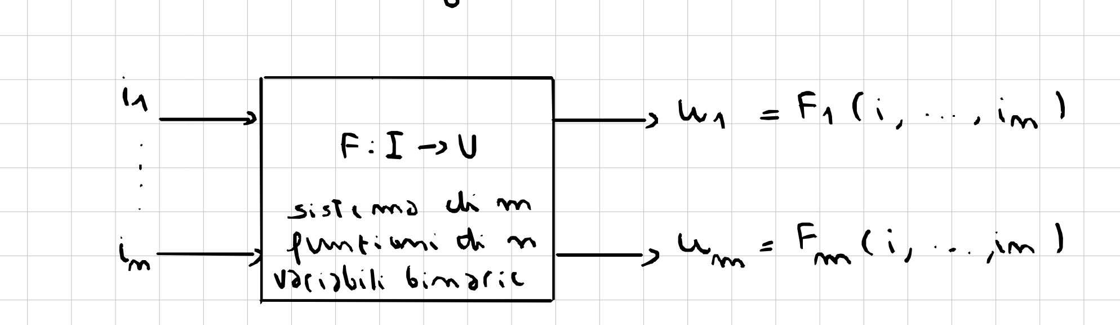
3) Si riempia la tabella sottostante, nell'ipotesi che le configurazioni a e b vengano mandati in ingresso a un sommatore a 4 bit che ne calcola la somma e genera anche i bit di Cout e overflow

1 1 1

a →	0 0 1 0	2	+ 2
b →	1 1 1 1	15	− 1
	1000 1		
	0 0 0 1	1	1
	1		
	0		
		NO	SI
		COUT	OVERFLOW

Lezione 30/04/25

Una rete logica combinatoria implementa una relazione tra variabili binarie in ingresso e in uscita. + Altro?



i_1
i_n
F : I → U
sistema di m funzioni di n variabili binarie
$u_1 = F_1(i, \dots, i_n)$
$u_m = F_m(i, \dots, i_n)$

Una funzione di n variabili (binarie) in ingresso avrà 2^n possibili valori d'uscita. Una funzione può essere descritta in vari modi, ad esempio tabelle della verità, mappe di Karnaugh, forme d'onda.
Descrivere una funzione significa enumerare le 2^n coppie di ingresso-uscita.

TABELLA DELLA VERITÀ

Associa a ogni coppia ingresso-uscita il valore desiderato a seconda della funzione che descrive

x_1	x_2	x_n	$F(x_1, \dots, x_n)$ ← es. OR, NOT, ...
⋮	⋮	⋮	⋮

↑ 2^n righe

Un sommatore a 4 bit ha 8 variabili d'ingresso: 4 bit del primo numero da sommare e 4 del secondo.

MAPPE DI KARNAUGH

Mappe bidimensionali, in cui le variabili sono divise tra i due assi

x_2 \ x_1x_2	00	01	11	10
1	0	1	1	0
0	0	0	1	1

← vanno obbligatoriamente elencate in codifica Gray

↖ valori di uscita

ab \ cd	00	01	11
00	0	1	0
01	1	0	1
11	0	1	0

Dunque la tabella di Karnaugh è come una tabella di verità "spalmata" su due dimensioni. Utili per ridurre il costo APPROFONDIRE
Celle adiacenti sono celle di configurazioni a distanza a 1, questo grazie alla codifica gray dei valori di ingresso.

ab \ cd	00	01	11
00	0	1	0
01	1	0	1
11	0	1	0

← anche le celle ai bordi sono adiacenti. FAI LA VERIFICA, DEVONO ESSERE A DISTANZA 1

abc \ de	00	01	11
000	1	0	1
001	0	1	0
010	1	0	1
011	0	1	0
010	1	0	0
110	0	1	0
111			
101	1	0	1
100	0	1	0

Ci sono anche adiacenze "simmetriche" rispetto agli assi che dividono a metà le tabelle, corrispondenti alla posizione della cella considerata specchiata rispetto all'asse

↖ asse che "divide a metà la tabella"

Notiamo che fino alle 4 variabili la tabella è molto intuitivo.
Un altro modo più intuitivo per trovare le celle adiacenti è dividere la tabella in 2^{n-4} mappe di 4 bit. Per far ciò si considera fisso, implicito, nascosto, il valore delle altre variabili

Es. 5 ingressi 2^{5-4} tabelle => 2 tabelle

ab \ cd	00	01	11	10
00	0	0	1	0
01	1	0	1	1
11	1	1	0	0
10	1	0	1	0

e = 0 Costruiamo la tabella nel caso e fosse = 0

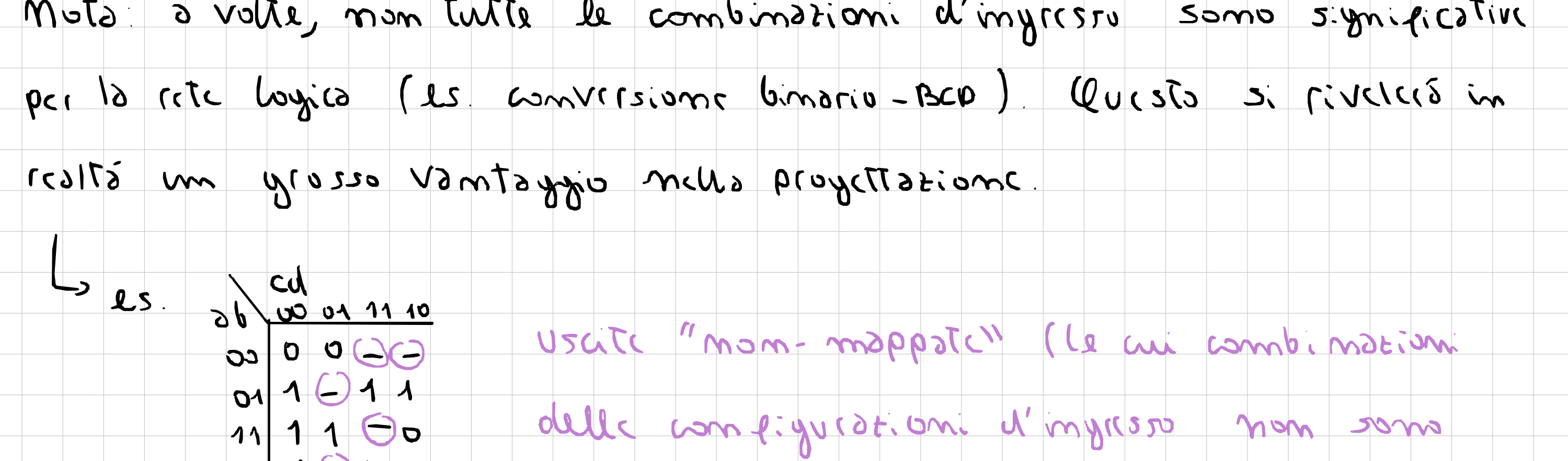
adiacenti a O

ab \ cd	00	01	11	10
00	0	0	1	0
01	1	0	1	1
11	1	1	0	0
10	1	0	1	0

e = 1 Costruiamo la tabella nel caso e fosse = 1

FORME D'ONDA

Consiste nel visualizzare le variabili come segnali binari. Grazie a ciò riusciamo a rappresentare delle tabelle il cui valore non è sempre lo stesso prestabilito (reti combinatorie) ma le cui uscite cambiano logica in funzione del tempo



↖ Anche se li ho rappresentati come regolari, gli intervalli di tempo possono non esserlo

Nota: a volte, non tutte le combinazioni d'ingresso sono significative per la rete logica (es. conversione binario-BCD). Questo si rivelerà in realtà un grosso vantaggio nella progettazione.

↳ es.

ab \ cd	00	01	11	10
00	0	0	-	-
01	1	-	1	1
11	1	1	-	0
10	1	-	1	0

uscite "non-mappate" (le cui combinazioni delle configurazioni d'ingresso non sono definite dalla nostra funzione descrittiva della rete => funzione incompleta)

Quando sviluppiamo una rete logica con funzione incompleta non può avere valori ≠ da 0 o 1, dunque ci saranno comunque degli 0 e 1 al posto dei trattini. Nella rappresentazione ad onda naturalmente non si possono rappresentare trattini.

ALGEBRA DI COMMUTAZIONE

Il numero di combinazioni possibili di n variabili binarie d'ingresso è, come già visto, 2^n (es. 4 bit → 4 v. binarie → 16 combinazioni).
=> per ogni configurazione, f potrà avere valore 0 o 1
=> per n ingressi ci saranno 2^n combinazioni (configurazioni) => 2^n uscite ottenute tramite la funzione f
=> sono possibili $2^{(2^n)}$ funzioni

Es.

- 1 var. di ingresso =>

I	f_0	f_1	f_2	f_3
0	0	1	0	1
1	0	0	1	1

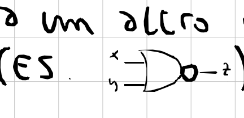
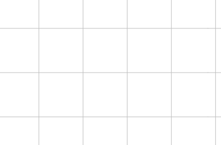
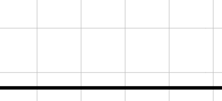
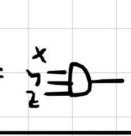
← 2^2 possibili funzioni in uscita

NOT logico ↗ ↖ funzione identità (= input)

- 2 variabili d'ingresso => $2^{2^2} = 16$ possibili funzioni in uscita

I_0	I_1	f_0	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8	f_9	f_{10}	f_{11}	f_{12}	f_{13}	f_{14}
0	0	0	1	0	1	0	1	0	1	0	1	0	1	0	0	1
0	1	0	0	1	1	0	0	1	1	0	0	1	1	0	1	1
1	0	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1
1	1	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1

Alcune di queste funzioni sono molto importanti:
f_8: minimo / AND logico => restituisce il minimo tra i due input
f_{14}: massimo / OR logico => restituisce il massimo tra i due input
f_6: XOR (funzione di parità pari) => restituisce 1 se I_0 e I_1 sono diversi
f_9: XNOR (funzione di parità dispari) => restituisce 1 se I_0 e I_1 sono uguali
Le operazioni AND, OR e NOT sono le tre operazioni logiche dell'algebra della commutazione (o Booleana), lo strumento più semplice per la sintesi delle reti logiche combinatorie.
Questa terna di operazioni si dice funzionalmente completa: combinandole tra di loro, è possibile realizzare qualsiasi funzione di un numero qualsiasi di variabili d'ingresso.

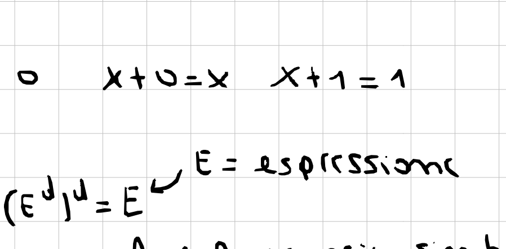
	SIMBOLO	POSTULATO	COSA FA	OPERATORE LOGICO (GATE)
AND	∨ (∘ ·)	x y z 0 · 0 = 0 0 · 1 = 0 1 · 0 = 0 1 · 1 = 0	restituisce "vero" se tutte le proposizioni sono vere (prodotto logico)	
OR	∧ (∘ +)	x y z 0 + 0 = 0 0 + 1 = 1 1 + 0 = 1 1 + 1 = 1	restituisce "vero" se almeno una delle proposizioni è vera (somma logica)	
NOT	¬ (∘ !)	x z 0¬ = 1 1¬ = 0	restituisce l'opposto di una proposizione (complementazione)	 o solo un pallino quando concatenato a un altro gate (Es. )

Possiamo anche rappresentare sotto forma di espressioni ingressi e uscite

Es. (x+y) · !z · !o

x	y	z	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

Esempio di conseguenza delle proprietà



PROPRIETÀ DELL'ALGEBRA DI COMMUTAZIONE

- Proprietà commutativa e associativa

$x \cdot y = y \cdot x$ $x+y = y+x$ $x \cdot y \cdot z = (x \cdot y) \cdot z$ $x+y+z = (x+y)+z$

- Proprietà distributiva

$x \cdot (y+z) = xy + xz$ $x+(y+z) = (x+y)+(y+z)$

- Altre

$x \cdot x = x$ $x+x = x$ $!(!x) = x$ $x \cdot 1 = x$ $x \cdot 0 = 0$ $x+0 = x$ $x+1 = 1$

$x \cdot !x = 0$ $x + !x = 1$

- Leggi di de morgan ← conseguenza della dualità $(E^d)^d = E$ ← E = espressione

$!(x+y) = !x + !y$ $!(x \cdot y) = !x + !y$

A e B generici simboli di un codice binario (equivalenti a 0 e 1)

- Teorema dell'assorbimento

$A + A \cdot X = A$ $(A) \cdot (A+X) = A$ $A + !A \cdot X = A+B$ $(A) \cdot (!A+X) = A+X$

- Teorema di espansione (caso particolare del teorema di Shannon)

$A = X \cdot A + !X \cdot A$ $A = (X+A) \cdot (!X+A)$

- Teorema del consenso

$x \cdot y + !x \cdot z + y \cdot z = x \cdot y + !x \cdot z$ $(x+y) \cdot (!x+z) \cdot (y+z) = (x+y) \cdot (!x+z)$

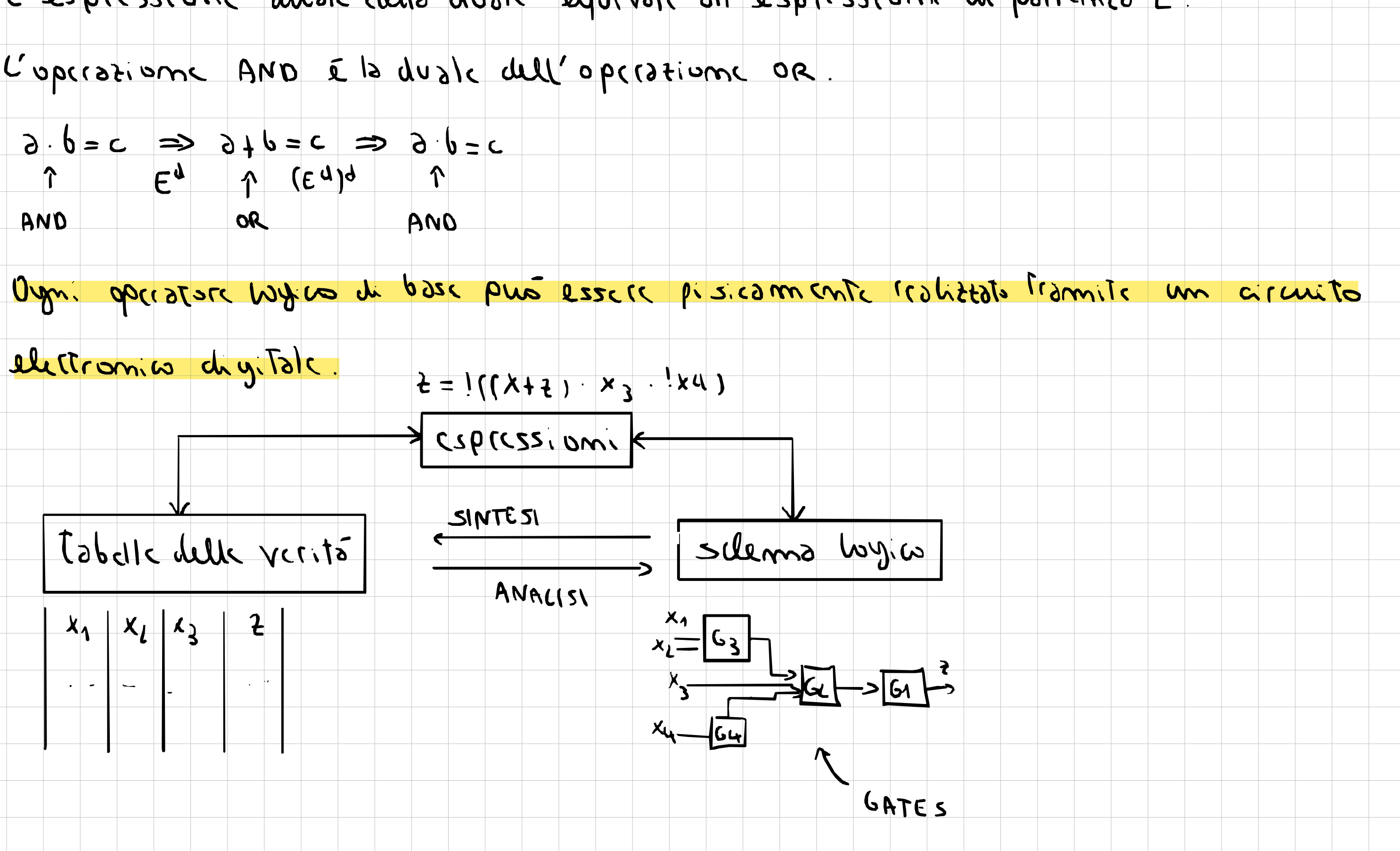
$x + !xy = x + y$

ESPRESSIONE DUALE

E^d espressione duale: espressione ottenuta a partire da un espressione E e scambiando fra loro somme e prodotti, zeri e uni.
L'espressione duale della duale equivale all'espressione di partenza E.
L'operazione AND è la duale dell'operazione OR.

$$\underset{\text{AND}}{a \cdot b = c} \xRightarrow{E^d} \underset{\text{OR}}{a + b = c} \xRightarrow{(E^d)^d} \underset{\text{AND}}{a \cdot b = c}$$

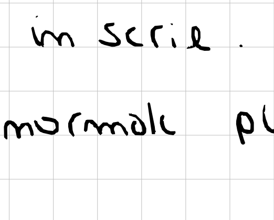
Ogni operatore logico di base può essere fisicamente realizzato tramite un circuito elettronico digitale.



Lezione 01/10/25

È possibile realizzare una rete logica a partire dalla sua tdv, attraverso le espressioni. (tabella della verità)

Si definisca espressione normale una espressione del tipo somma di prodotti (SP) o prodotto di somme (PS) ⇒ rappresentabile con al più 2 gate in serie.


da verificare

Una espressione normale può essere:

- canonica: compaiono tutti gli ingressi ⇒ ricavate tramite sintesi canonica
- irridondanti: la rimozione di un qualsiasi termine altera il risultato ⇒ ⇒ ricavate tramite semplificazione di espressioni (non per forza compaiono tutti gli ingressi)

Nota: sarebbe anche possibile ricavare reti logiche da espressioni non normali o anche senza ricavare espressioni (sintesi diretta) (richiede esperienza).

SINTESI CANONICA SP (il primo metodo di sintesi che studiamo)

Osservando le righe di una tdv, si possono identificare le configurazioni per le quali l'uscita della rete vale 1.

Consideriamo un'uscita alla volta (una riga alla volta).

Viene data una tabella. Guardiamo le configurazioni che ci danno 1 come uscita. Traduciamo gli 0 di tali configurazioni in 1 tramite NOT. Traduciamo poi ogni configurazione in un prodotto (AND) ⇒ mi darà effettivamente 1. Infine, faccio l'OR tra le espressioni.

x	y	z	F		
0	0	0	0		
0	0	1	1	⇒	!x !y z
0	1	0	1	⇒	!x · y · !z
0	1	1	0		
1	0	0	1	⇒	x !y !z
1	0	1	0	⇒	x !y z
1	1	0	1		x y !z
1	1	1	1		x · y · z

↳ detti **mintermini**

OR ⇒ (!x !y z) + (!x y z) + (x !y !z) + (x !y !z) + (x y !z) + x y z

⇓

schema logico ①

Essendo fatta tramite sintesi combinatoria ⇒ ogni espressione (e quindi ogni gate) usa tutti gli ingressi ⇒ può essere ridondante (come in questo caso).

Per semplificarla ⇒ possiamo applicare teoremi e proprietà dell'algebra booleana.

SINTESI CANONICA PS

Sfruttando il principio di dualità, posso fare "il contrario"

- Considero le configurazioni in cui l'uscita è 0, negando gli 1 in modo che la somma (OR) faccia 0.
- Faccio gli OR di ogni uscita ⇒ detti maxtermini
- Faccio l'AND finale

x	y	z	F		
0	0	0	0	⇒	x + y + z
0	0	1	1		
0	1	0	1		
0	1	1	0	⇒	x + !y + !z
1	0	0	1		
1	0	1	0	⇒	x + !y + z
1	1	0	1		
1	1	1	1		

↳ **maxtermini**

AND ⇒ (x + y + z) · (x + !y + !z) · (x + !y + z)

⇓

schema logico ②

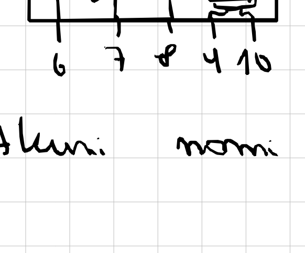
reti equivalenti

⇑

① e ② sono diversi ma descrivono entrambi la stessa logica. Applicando il principio di dualità potremmo passare da un SP a un PS (non per forza uguale all'SP/PS "nativa").

Notiamo che ② è più semplice di ①. Dunque per scegliere che approccio usare conviene guardare se ci sono più uscite 1 o più uscite 0 e usare SP o PS di conseguenza.

Una volta progettata la rete logica, possiamo creare fisicamente il circuito. Nella realtà, esistono dispositivi del tipo



in cui i gate sono realizzati tramite transistors opportunamente disposti, e ci sono pin di ingresso e uscita da cui prelevare/immettere segnali e corrente.

Alcuni nomi di questi circuiti: SN7404, SN7411, ...

In un sistema l'elaborazione richiede tempo ⇒ c'è ritardo. Per ora lo ignoriamo, poi lo prenderemo in considerazione. Il ritardo totale di un circuito è la somma dei ritardi delle sue componenti.

Facciamo ora la sintesi di qualche rete combinatoria interessante, utilizzando principalmente SP.

MULTIPLEXER (importantissimo)

MULTIPLEXER A DUE VIE:

tre ingressi: A, B (ingressi) e C (segnale di controllo), uscita z

logica: se C = 0 ⇒ z = A, altrimenti z = B

z = !ABC + A!B!C + AB!C + ABC ← giusta ma ridondante

Semplifichiamo

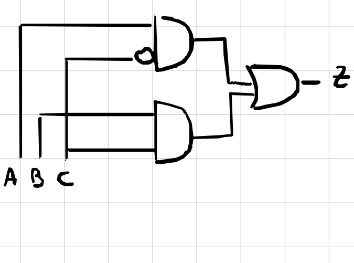
$A!C\underbrace{(!B + B)}_{=1 \text{ per proprietà}} \Rightarrow z = A!C \cdot 1 \Rightarrow z = A!C$

BC(!A + A) = BC

⇒ z = A!C + BC ← espressione minima (non semplificabile)

costo minimo (minimo n. di gates utilizzati)

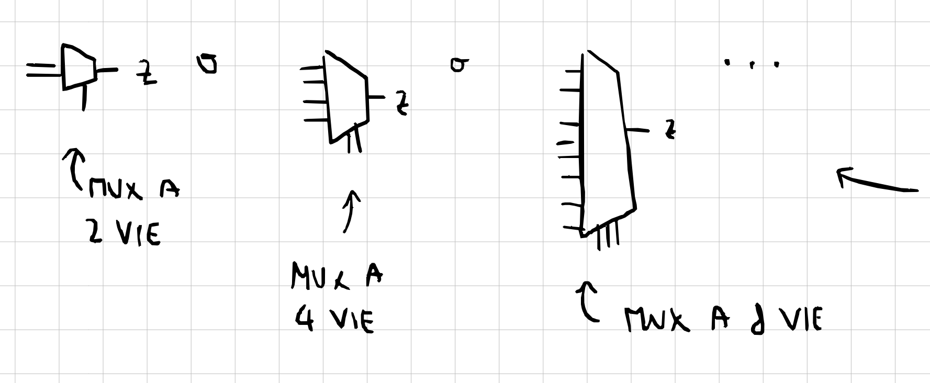
⇒ schema logico finale:



MULTIPLEXER A 2^n VIE:

Si avranno 2^n ingressi e n segnali di controllo.

In uno schematico, rappresentiamo (black box) un MUX (multiplexer) così:

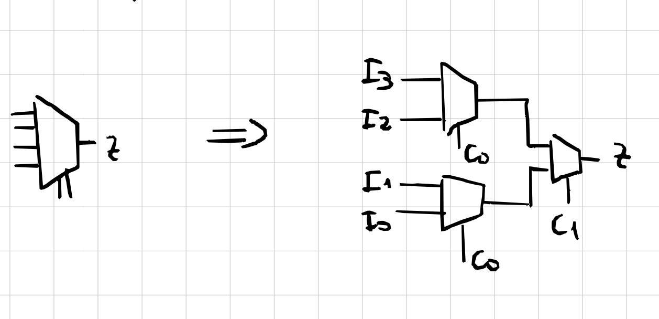


ne esce sempre un solo segnale

prova a fare il mux 8 con questo approccio

Guarda gli schematici di un MUX a più variabili (nota che c'è un ragionamento logico per non stare a rivalutare la TDV a più variabili) ① (usando i mintermini)

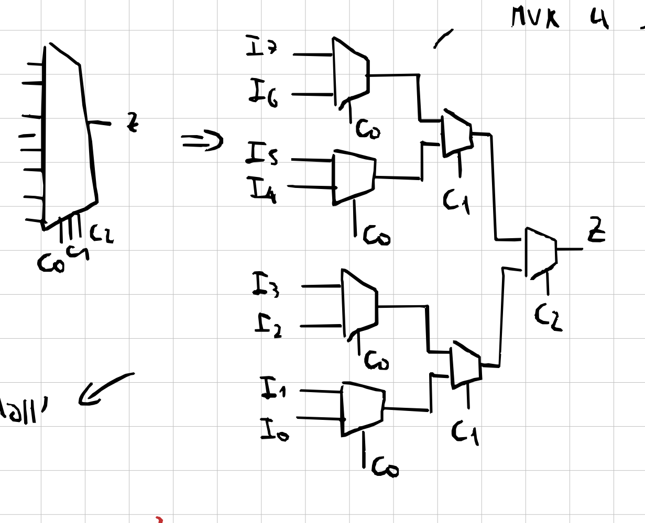
Posso anche realizzare un multiplexer a 2^n vie come composizione di più multiplexer a 2 vie ②



nota che partono dall'I0 dal basso

↳ è necessario? (non penso)

o anche con dei MUX 4 e dei MUX 2



Lo svantaggio dell'approccio ② è che aumento i livelli ⇒ aumento il ritardo. Conviene usare la realizzazione ①. (ogni livello produce ritardo)

A volte conviene usare un approccio tipo ①, altre volte un approccio tipo ② (composizione di componenti). Ad esempio, per il multiplexer conviene l'approccio ①, per un sommatore vedremo che conviene un approccio ②.

Nota: probabilmente i MUX sono così utilizzati perché "simulano un if"

DEMUX

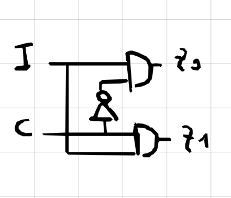
Rete con 2 ingressi: I (ingresso), C (segnale di controllo) e 2 uscite (z_0, z_1).

Il demux fa il contrario del MUX, sceglie quale uscita selezionare: se c = 0, I apparirà sull'uscita z_0, altrimenti sull'uscita z_1.

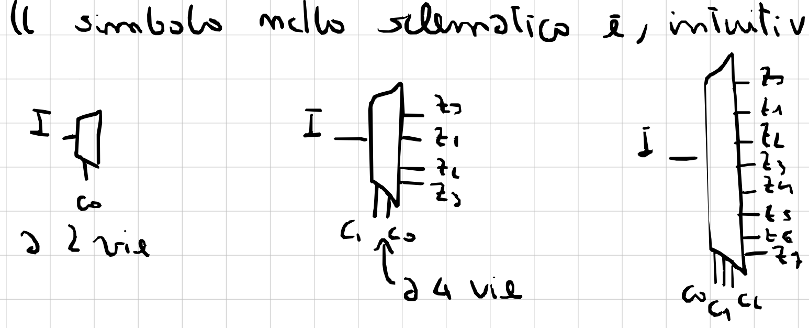
$z_0 = I!C$

$z_1 = IC$

schema logico

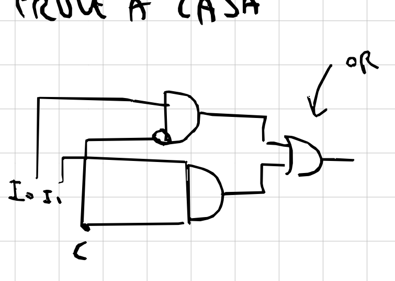


Il simbolo nello schematico è, intuitivamente,



Anche in questo caso possiamo ragionare, per la costruzione di un DEMUX a 2^n vie, per mintermini/maxtermini o per composizione di DEMUX più piccoli

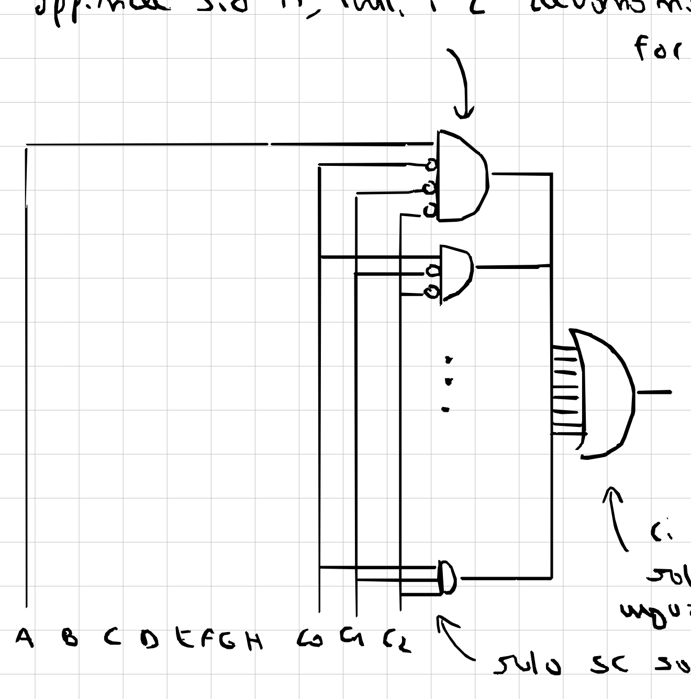
PROVE A CASA



	A	B	C	Z
m_0	0	0	0	0
m_1	0	0	1	0
.	0	1	0	0
	0	1	1	1

C = 0 ⇒ A
: B

affinché sia A, tutti i C devono essere 0 ⇒ negati sono 1 ⇒ unico modo per far passare A nell'AND



ci sarà un solo ingresso uguale a 1

solo se sono tutti 1 H passa dall'and

TDV MUX A 8 VIE

c2	c1	c0	
0	0	0	A
0	0	1	B
0	1	0	C
	...		

Lezione 14/10/25

DECODER

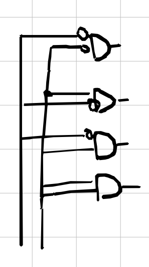
È equivalente a un $\overset{\text{a } 2^m \text{ vie}}{\text{demux}}$, ma in cui l'ingresso è sempre 1.

Dunque un ingresso costante attiverà una e una soltanto uscita, a seconda della t.d.v. È una rete che realizza una conversione da codice binario a codice decimale

t.d.v

c_1	c_2	z_0	z_1	z_2	z_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

⇒

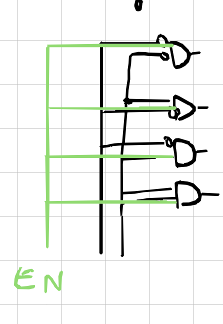


↑ come un mux, senza OR

Il decoder è un generatore di mintermini a m bit.

↳ Approfondire questo e perché si chiama decoder

Per spegnere una rete si può usare un segnale detto "enable", ovvero un segnale che forza l'uscita a essere 0. Ciò si fa mettendolo in AND il segnale di enable (0 per spegnerlo o 1 per farlo funzionare normalmente)

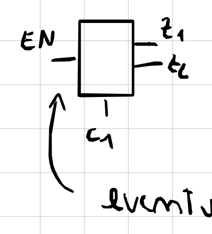


METTO UN SEGNALE ENABLE IN AND CON L'USCITA DI UN CIRCUITO

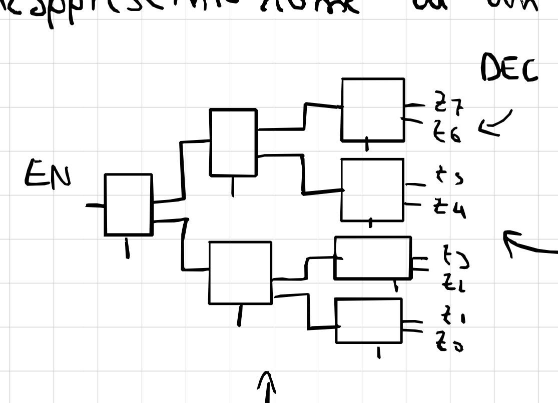
⇒ SE EN = 0 ⇒ AND = 0 ⇒ USCITA FINALE = 0

⇒ SE EN = 1 ⇒ AND ⇒ USCITA INALTERATA

Rappresentazione di un decoder:



eventuale segnale di enable



3 LIVELLI

Si usano anche per accedere a specifiche celle di memoria, con questo tipo di struttura

Per realizzare un DEC a 6 bit (DEC6) (6 bit $= 2^6$ uscite $= 64$) con dei DEC a 3 bit (DEC3) sono necessari $\frac{64}{2^3} = 8$ ⇒ sono necessari 8 DEC3.

Stesso discorso si applica anche per i multiplexer.

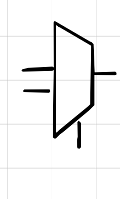
Dunque, in generale, per avere un DEC N tramite dei DEC M, con M<N, occorrono $\frac{N}{2^M}$ DEC (vale anche per i MUX)

si può completare il circuito con DEC più piccoli

Es. voglio costruire un DEC4 con dei DEC5 e DEC4.

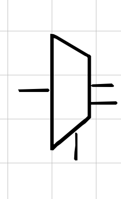
Riassumendo:

MUX:



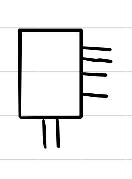
prende due input e restituisce un uscita, in funzione del valore del segnale di controllo

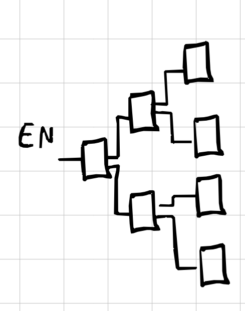
DEMUX:



prende un segnale in input e, a seconda del segnale di controllo, restituisce il segnale su una sola porta, lasciando le altre a zero

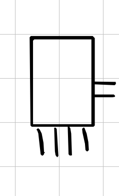
DEC





Si considera un segnale in ingresso come costante a 1 oppure come dato da un EN (quindi opzionalmente anche 0 per spegnere il decoder). I N segnali di controllo scelgono una e una sola via (uscita) che vada a 1, le altre restano a 0. Utilizzato ad esempio per raggiungere una certa cella di memoria. Per questo può essere considerato come un DEMUX a ingresso fisso 1.

ENC



Effettua l'operazione opposto del decoder. Entra un ingresso costante, es. quello dell'EN, e 2^N segnali di controllo, di cui solo uno alla volta può essere = 1. Sulla base di quale è uguale a 1, la t.d.v. indica la configurazione d'uscita (a N bit)

es. 3 bit

c_0	c_1	c_2	c_3	c_4	c_5	c_6	c_7	z_0	z_1	z_2
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
...								...		

"configurazione d'uscita (più usate)"

L'EN (ENABLE) è un collegamento in AND alle uscite di un componente, che può dare 1 (lascia invariato il comportamento della rete) o 0 (in AND diventa 0 ⇒ "spegne" la rete).

Anche nel caso di DEC e ENC, si possono avere a N vie. In generale, per i componenti visti (per lo meno per DEC e MUX) vale che per avere un componente a N bit, costruendolo con componenti a M bit, con M<N, sono necessari $\frac{2^N}{2^M}$ componenti. Si possono anche usare componenti di dimensione M diversa tra loro (es. 2 DEC3 e 1 DEC4 per formare un preciso DEC N)

SOMMATORE DI NUMERI BINARI

Ingressi: 2 numeri binari a 4 bit + carry in (riporto dalla somma precedente)

Uscite: Il numero binario a 4 bit corrispondente alla somma + carry out + overflow.

Come già visto, si può realizzare con un HA (a 2 bit) + N-1 FA (a 3 bit)

Half-adder:

TDV

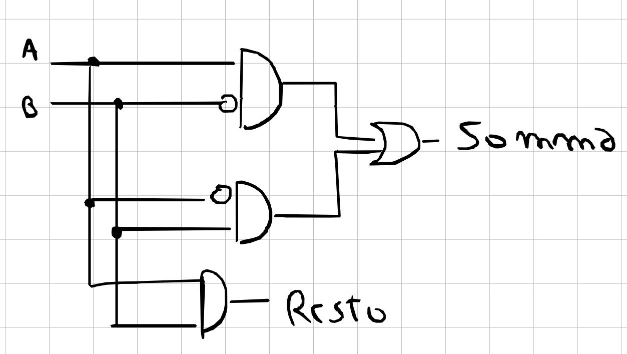
A	B	SOMMA	RIPORTO
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Tramite SP

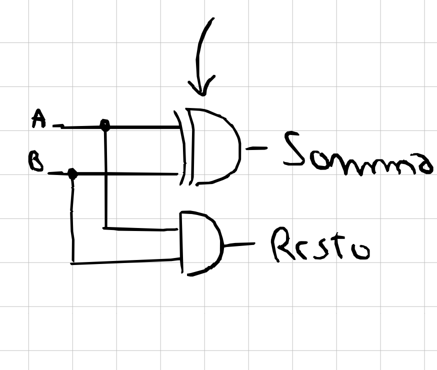
Somma: $A!B + !AB$

Resto: AB

⇒



≡



Full adder

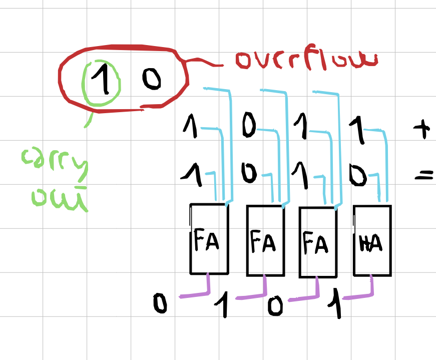
A	B	C	Somma	Resto
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Tramite SP

Somma: $!A!BC + !AB!C + A!B!C + ABC$

Resto: $!ABC + A!BC + AB!C + ABC$

Essendo che la prima somma non ha carry-in possiamo realizzarlo con un half adder, e il resto con full adders:



sintesi diretta

Lezione 15/10/25

SINTESI MINIMA

Sintesi che si occupa di ottimizzare, non solo in termini di livelli della rete ma anche di numero di componenti che la compongono, una rete logica.

RIASSUNTO SU MINTERMINI E MAXTERMINI

A ogni riga della TDV è associato un termine.

	A	B	C	F(A,B,C)		
m_0	0	0	0	0	$x+y+z$	●
m_1	0	0	1	1	$!x!yz$	●
m_2	0	1	0	1	$!xyz$	●
m_3	0	1	1	0	$x+!y+!z$	●
m_4	1	0	0	1	$x!y!z$	●
m_5	1	0	1	0	$!x+y+!z$	●
m_6	1	1	0	1	$xy!z$	●
m_7	1	1	1	1	xyz	●

mintermini: prodotto logico che rappresenta una configurazione in cui l'uscita è 1, gli ingressi vengono presi come negati se valgono 0

maxtermine: somma logica che rappresenta una configurazione, in cui l'uscita è 0, gli ingressi vengono presi come negati se valgono 1

Una rete logica può essere sintetizzata (descritta) mediante somma dei mintermini (SP) o tramite prodotto di maxtermini.

Sia F una qualsiasi espressione.

Sintesi canonica ⇒ ottimizzazione in termini di livelli

Per ricavare espressioni irridondanti da espressioni canoniche, si sfruttano le adiacenze tra le configurazioni per cui F vale 1 (0). Per definizione, due configurazioni adiacenti si differenziano tra di loro per un solo bit

⇒ si può sfruttare il principio di adiacenza per semplificare F

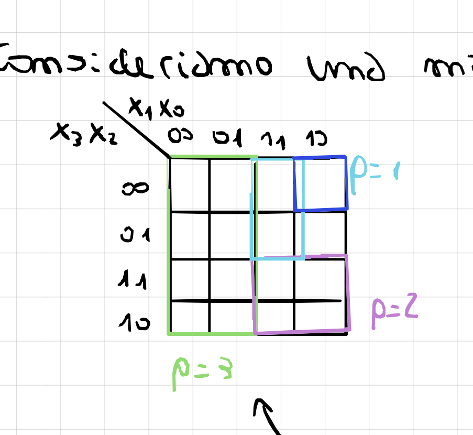
Es.

$$F = \underbrace{!ABC + A!B!C + AB!C + ABC}$$ ⇒ $F = A!C + BC$

A!C

BC

Consideriamo una mappa di Karnaugh

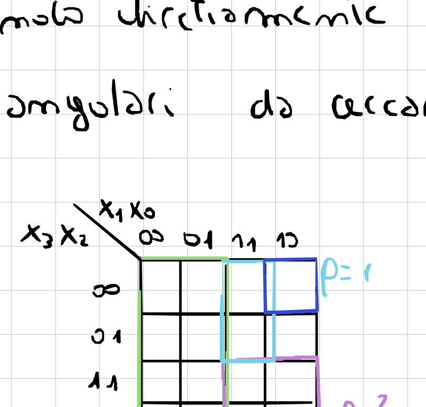


↖ Raggruppamenti rettangolari

↳ Ogni cella all'interno di un raggruppamento di ordine p ha p celle adiacenti

In ogni raggruppamento c'è solo una variabile che non cambia (per adiacenza) ⇒ si può semplificare (sintesi minima) l'espressione agendo direttamente sulla tabella di Karnaugh. I raggruppamenti rettangolari da cercare sono quelli di celle contenenti solo 1 (o 0)

es.



↖ in p_2 x_1 vale sempre 1 ⇒ x_0 cambia ⇒ possiamo rimuovere x_0 per principio di adiacenza

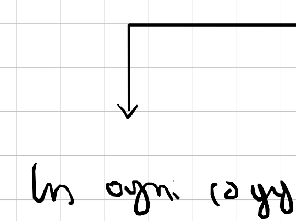
↳ più grande è il raggruppamento, maggiore il numero di variabili eliminabili

Nota: i raggruppamenti DEVONO contenere solo un numero di celle che sia una potenza di 2 ($p_3 = 2^3$ celle, $p_2 = 2^2$ celle, ...)

Si nota una variabile che è sempre 1, e si semplifica come le altre

Raggruppamenti di dimensione massima individuano implicanti primi. Se inoltre tale raggruppamento contiene almeno una cella non copribile da altri raggruppamenti, allora esso individua gli implicanti primi essenziali.

Es ①



C \ AB	00	01	11	10
0	0	0	1	1
1	0	1	1	0

↙ mappa di Karnaugh di un MUX a 2 vie

$F = A!C + BC + AB$

sempre 1

sempre 0 ⇒ negato

sempre 1

sempre 1

sempre 1

sempre 1

ridondante, copre celle già coperte

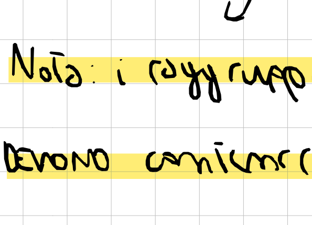
Implicante essenziale ⇒ copre uni che non possono essere coperti da altri implicanti ⇒ raggruppamento verde non essenziale, gli altri due sì

Dunque

→ cerchiamo le uscite = 1 ⇒ identifichiamo i rettangoli di uni adiacenti ⇒ su questi rettangoli, teniamo le variabili che per tali uscite rimangono costanti: se un ingresso resta 1 non si nega nella espressione, altrimenti si scrive negato ⇒ si eliminano per ogni rettangolo gli ingressi che variano ↖ SP

Si può fare una cosa analoga cercando le uscite = 0, dove però i termini che non variano saranno negati se = 1, non negati se restano 0, e si invertono + e · ↖ PS

Es. ②



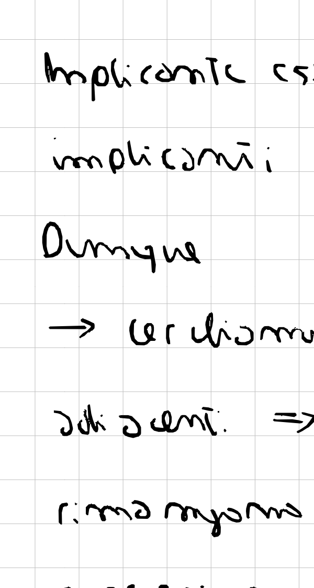
C \ AB	00	01	11	10
0	0	0	1	1
1	0	1	1	0

→ $F = (A+C) \cdot (!C+B) \cdot \underbrace{(A+B)}$

↑ ridondante

In casi più complessi, identificare sempre i raggruppamenti di ordine massimo.

ESEMPI:



ab \ cd	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	1	1	0	0
10	1	1	0	0

	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	1	1	0	0
10	1	1	0	0

raggruppamenti di ordine 1

$F = a!c!d + a!c!d + !b + !cd + \dots$

non abbiamo identificato i raggruppamenti massimi ⇒ F è ridondante (non è di costo minimo)

$F = a!c + !cd + !ad$

↖ (anche se !cd non è essenziale)

↓ $F = a!c + !ad$

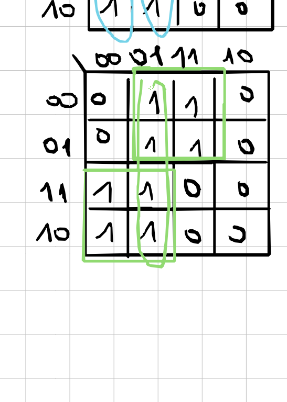
forse lo era anche qua

abbiamo identificato raggruppamenti massimi ⇒ F è di costo minimo

Nel caso in cui ci siamo celle di uscita incomplete, posso interpretare tali celle come 0 o 1 se sono utili a formare un gruppo.

ES (con analisi PS) → PS ⇒ si cercano gli 0

↳ SP ⇒ si cercano gli uni



1	1	0	0
1	-	0	-
1	1	0	1
1	1	0	1

⇝ $F = (!c+!d)(a!+c)$ ⇐ rete di costo minimo

sfruttiamo l'uscita incompleta considerandola come 0, in modo da poter sfruttare questo raggruppamento

questo lo consideriamo come 1, perché è inutile come 0 (non comporta coprire 0 non coperti da altri raggruppamenti)

↑ (-) non conta come 0 da coprire

Dunque bisogna cercare

- raggruppamenti di ordine massimo
- raggruppamenti che coprono ALMENO una cella non coperta da altri raggruppamenti.

Se non ci sono mintermini adiacenti (se non ci sono 0 e 1 adiacenti nella mappa di Karnaugh) non si può usare la sintesi minima ma solo la sintesi canonica.

⇝ o maxtermini nel caso PS

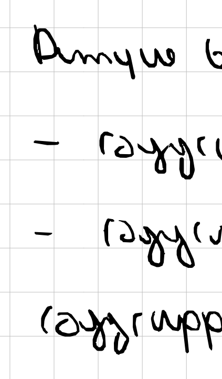
Implicante (SP ⇒ ricerca degli 1): termine prodotto che vale 1 solo per configurazioni in cui F ≠ 0 (1 o -) ⇒ prod. che rappresenta un gruppo di uni

Implicato (PS ⇒ ricerca degli 0): termine della somma che vale 0 solo per configurazioni in cui F ≠ 1 (0 o -) ⇒ somma che rappresenta un gruppo di zeri

Implicante (implicato) primo: termine non più tale rimuovendo una qualsiasi variabile ⇒ prodotto (somma) che rappresenta il più grande gruppo di uni (zeri) possibile

Implicante (implicato) primo essenziale: unico ad assumere valore 1 (0) per alcune configurazioni in cui F vale 1 (0) ⇒ prodotto (somma) che rappresenta il più grande gruppo di uni (zeri) possibile e che contiene almeno un 1 (0) che non può essere raggiunto da altri raggruppamenti

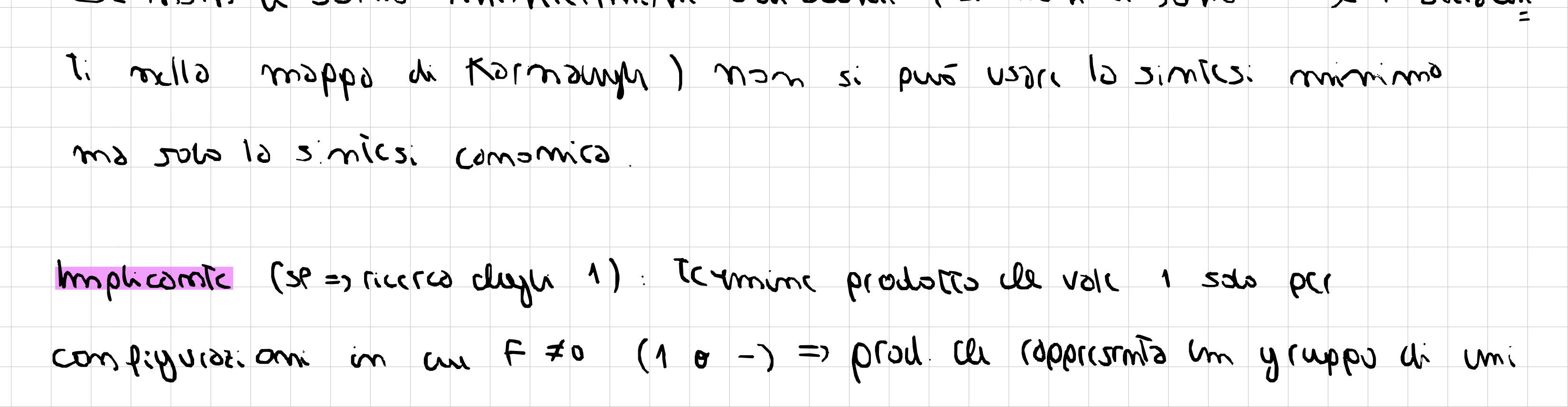
7 SEGMENTI REALIZZATO CON SINTESI MINIMA



Serve una mappa di Karnaugh per ogni segmento

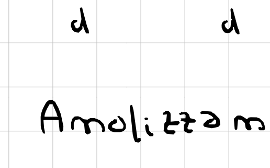
Ingresso: configurazione binaria a 4 bit

Uscita: configurazione binaria a 7 bit corrispondente ai 7 segmenti.



Analizzando come deve comportarsi ogni segmento, comportamento descritto dalla mappa di Karnaugh, possiamo ricavare F per ogni segmento, e dunque realizzarne il circuito

Nota: se tutta la tabella è un gruppo, allora F=1



-	1	-	1
-	1	1	1
1	1	-	1

⇒ F=1 (nessuna variabile resta sempre lo stesso valore in tutte le caselle)

Per riassumere

La sintesi canonica si può sempre fare ma non dà il costo minimo.

Per ottenerlo ⇝ semplificazioni dall'algebra booleana

↳ sintesi minima (non sempre possibile)

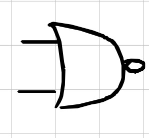
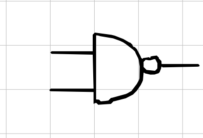
Lezione 28/10/25

Oltre all'algebra booleana esistono altri tipi di algebra utilizzate.

ALGEBRE BINARIE

Riprendendo le possibili funzioni di due variabili binarie, abbiamo:

I_1 I_0	NOR (f_1)	NAND (f_7)
0 0	1	1
0 1	0	1
1 0	0	1
1 1	0	0

NOR:  NAND: 

ALGEBRA DEL NAND (NOR)

L'operatore NAND (NOR) è funzionalmente completo, ovvero è in grado di realizzare qualsiasi funzione di variabili binarie senza l'uso di altri operatori.

Condizione sufficiente affinché un operatore sia funzionalmente completo è che attraverso esso si possano realizzare i gate dell'algebra di computazione (AND, OR, NOT)

L'algebra del NAND è definita dai valori {0,1} e dall'operatore NAND {↑}, l'algebra del NOR da {0,1}{↓}.

OPERATORE	REALIZZAZIONE TRAMITE NAND	REALIZZAZIONE TRAMITE NOR
NOT: !A	A ↑ A	A ↓ A
AND: AB	(A ↑ B) ↑ (A ↑ B)	(A↓A) ↓ (B↓B)
OR: A+B	(A ↑ A) ↑ (B ↑ B)	(A↓B) ↓ (A↓B)

Lo XOR non è invece funzionalmente completo (richiede anche l'AND).

Da un punto di vista circuitale, NAND e NOR richiedono un minor numero di componenti (transistor) per essere realizzati.

SINTESI A NAND

È possibile effettuare una sintesi a NAND (↑) a partire da un espressione SP. Se l'espressione SP è irridondante, lo sarà anche quella a NAND.

Nota: il not si scrive anche con "'" ⇒ !A = A

Algoritmo

$F = a \cdot b + c' \cdot d + e \cdot f' + g$

↓ Si sostituisce · con ↑ e si complementano i prodotti *

$F = (a \uparrow b)' + (c' \uparrow d)' + (e \uparrow f')' + g$

↓ II l. di de Morgan: $A + B = \overline{\overline{A} \cdot \overline{B}}$ (e si complementano le variabili singole (es. g)

$F = ((a \uparrow b) \cdot (c' \uparrow d) \cdot (e \uparrow f') \cdot g')'$

↓ Si sostituisce · con ↑ e si complementa il tutto *

$F = (a \uparrow b) \uparrow (c' \uparrow d) \uparrow (e \uparrow f') \uparrow g'$

* questo passaggio è la definizione dell'operatore ↑ (NAND) (approfondire)

SINTESI A NAND DI UNO XOR

Operazione di XOR: $A \oplus B = A!B + !AB$

$U = ab' + a'b$

⇓ passaggi

$U = (a \uparrow (a \uparrow b)) \uparrow (b \uparrow (a \uparrow b))$

SINTESI A NOR

Notazione NOR: ↓

Si può effettuare la sintesi a NOR a partire da una espressione PS. Anche in questo caso, se la PS è irriducibile allora lo è anche in NOR.

I passaggi sono analoghi a quelli del NAND:

$F = (a' + b' + c) \cdot (d' + e) \cdot f' \cdot g$

↓ definizione dell'operatore ↓

$F = (a' \downarrow b' \downarrow c)' \cdot (d' \downarrow e)' \cdot f' \cdot g$

↓ I° l. di de Morgan

$F = ((a' \downarrow b' \downarrow c) + (d' + e) + f + g')'$

↓ definizione dell'operatore ↓

$F = (a' \downarrow b' \downarrow c') \downarrow (d' \downarrow e) \downarrow f \downarrow g'$

SINTESI A NOR DI UNO XNOR

Definizione di uno XNOR: $A \odot B = (A!B) + (!AB)$

$U = (a + b') \cdot (a' + b)$

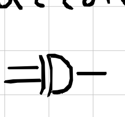
⇓ passaggi

$U = (a \downarrow (a \downarrow b)) \downarrow (b \downarrow (a \downarrow b))$

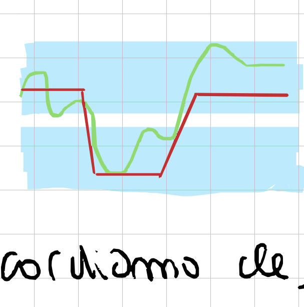
XOR

Come detto, è un operatore che restituisce 1 solo se i due ingressi sono diversi. Lo XOR non è funzionalmente completo, richiede anche l'AND e l'uso della costante 1 (per realizzare il NOT).

Proprietà interessanti:

- $a \oplus 1 \rightarrow$ NOT a
- $a \oplus 0 \rightarrow a$
- si può utilizzare per realizzare l'uscita somma dell'Half-Adder
- simbolo: 

APPLICAZIONE (ESEMPIO) DELLO XOR: PROTEZIONE DELL'INFORMAZIONE



- segnale analogico in ingresso
- corrispondente segnale digitale binario

Ricordiamo che, sebbene il segnale binario sia più robusto ai disturbi, il disturbo potrebbe essere abbastanza forte da alterare il valore da H a L (o viceversa).

Si suppone di trasmettere 3 bit: x_0, x_1, x_2 (ad esempio, numeri binari da 0 a 7. Se durante la trasmissione del valore "7" un disturbo altera x_2, allora al posto che arrivare "7" (111) arriva "3" (110). Il ricevitore non può sapere che questo è un errore.

x_2 \ $x_1 x_0$	00	01	11	10
0	0	1	3	2
1	4	5	7	6

Per ovviare al problema, introduciamo un bit "p". Imponiamo che le configurazioni lecite siano a distanza (almeno) 2 l'una dall'altra, e supponiamo che il mittente invii solo configurazioni lecite. In questo modo, l'alterazione di un solo bit farà arrivare al ricevitore una configurazione illecita

P x_2 \ $x_1 x_0$	00	01	11	10
00	0	E	3	E
01	E	5	E	6
11	4	E	7	E
10	E	1	E	2

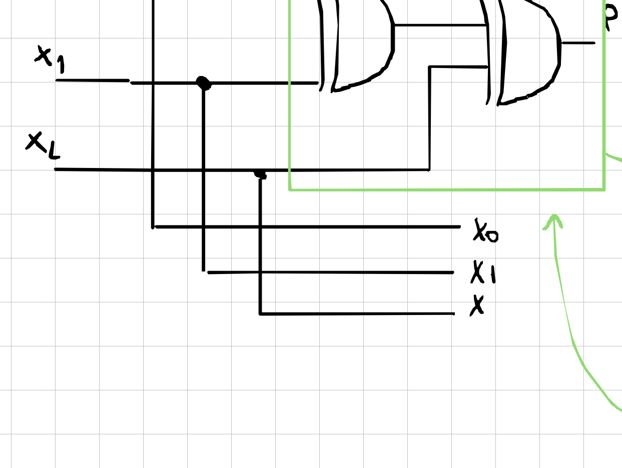
Es. il mittente invia solo configurazioni lecite (0000, 1100, ...). Se un bit arriva sfalsato (e questo rilevatore è in grado di identificare solo un eventuale bit sfalsato) allora la configurazione avrà distanza uno dalla configurazione inviata e di conseguenza sarà illecita.

Es. voglio mandare 1100 (3) ⇒ si sfalsa x_2 ⇒ 1110 è configurazione illecita

Una rete combinatoria di questo tipo, che restituisce 1 in caso di combinazione illecita, o altrimenti, può essere realizzato attraverso degli XOR. Tale tipo di rete prende il nome di CHECKER.

Notiamo dalla mappa di Karnaugh che c'è l'errore solo quando il numero di 1 nella configurazione d'ingresso è dispari

Il bit "p" non viene "direttamente" mandato ma deve essere generato in modo tale da rendere pari il numero di 1 della configurazione



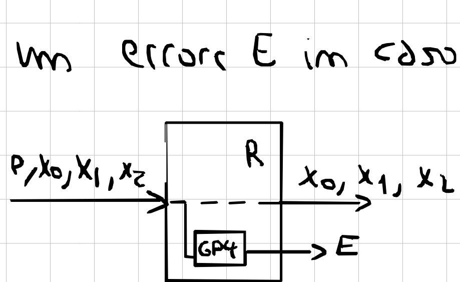
GPm ⇒ gen. di parità a m bit

GP generatore di parità

⇒ restituisce 1 se la configurazione d'ingresso ha un numero dispari di 1

GP3

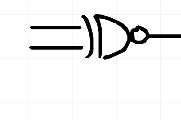
Il ricevitore processerà x_0, x_1, x_2 e e, e restituirà il segnale originale e un errore E in caso sia sfalsato



XNOR

Operatore che restituisce 1 solo quando i due ingressi sono uguali.

Può essere utilizzato per confrontare configurazioni binarie.

XNOR = "XOR negato" 

Lezione 29/10/25

ALTRI TIPI DI SINTESI

SINTESI DIRETTA

Sintesi realizzata tramite ragionamento e intuito. Per far ciò il progettista utilizza componenti già esistenti per "comporre" le reti, vedremo come effettuare la sintesi mediante decoder e mediante multiplexer

SINTESI CON DECODER

Decoder => rete combinatoria con n bit di controllo e 2^n uscite.
Una sola uscita alla volta vale 1 (a seconda della configurazione che arriva).
Un mintermine da "1" solo per una specifica configurazione in ingresso
=> il decoder è un generatore di mintermini

esempio: FULL-ADDER

a b c	S	R
0 0 0	0	0
0 0 1	1	0
0 1 0	1	0
0 1 1	0	1
1 0 0	1	0
1 0 1	0	1
1 1 0	0	1
1 1 1	1	1

S = !A!BC + !AB!C + A!B!C + ABC
R = !ABC + A!BC + AB!C + ABC
Basta far entrare tutti gli ingressi in un decoder =>
=> per ogni configurazione si accende solo 1 delle uscite
=> l'OR tra i mintermini mi da S e R (da approfondire)

guarda la slide per capire meglio

Nota: ricorda che la sintassi canonica ci da il costo minimo solo quando non ci sono adiacenze nella tabella di Karnaugh

SINTESI CON MULTIPLEXER

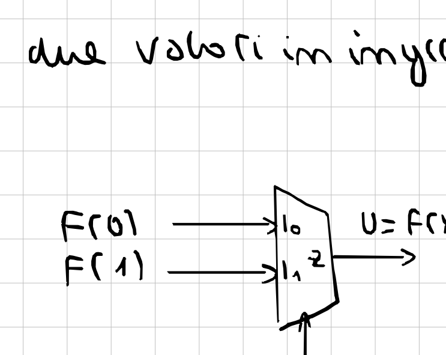
La sintesi canonica si basa sulle cosiddette espressioni canoniche (prodotto tra mintermini, somma tra maxtermini). Con i MUX possiamo generare una espressione detta "generale", più generica della sintesi canonica, che contiene sia mintermini sia maxtermini.
A tale scopo si può utilizzare il MUX in due modi:
- MUX come reti programmabili
- Alberi di MUX, utile nel caso di funzioni con tante variabili d'ingresso, ci permette di semplificare la rete (ha il contro che la semplificazione causa una non-ottimizzazione del costo). Questo processo si basa sul teorema di Shannon.

Nota: ricorda che un mintermine è un prodotto logico in cui compaiono tutti gli ingressi.

MUX A 2 VIE COME RETE COMBINATORIA PROGRAMMABILE

Il mux, presi due valori in ingresso, è in grado di portarne solo uno sull'uscita.

x	F(x)
0	F(0)
1	F(1)



Definendo gli ingressi del multiplexer tramite funzioni e usando il segnale di controllo x come segnale d'ingresso. In tal senso si parla di MUX programmabile.
In un MUX, ogni ingresso è collegato in AND a un mintermine di controllo.
Grazie a ciò si possono introdurre le espressioni generali.

	X_1	X_0	F
m_0	0	0	F(0)
m_1	0	1	F(1)
m_2	1	0	F(2)
m_3	1	1	F(3)

Nella sintesi SP, conoscendo la funzione, andremo a selezionare i mintermini per cui F(i) vale 1.
Qui però la funzione è incognita (programmabile).
Non conoscendo a priori F(i) possiamo considerare in AND i mintermini per ogni possibile valore della funzione, tanto ne verrà selezionato solo 1, che è esattamente quello che fa un MUX. Tale espressione che comprende tutti i mintermini a prodotto con un valore di una i-esima funzione è appunto detta espressione generale. Si può fare una cosa analoga, con OR al posto degli AND, per la sintesi PS.

APPROFONDIAMO CON UN ESEMPIO QUANTO APPENA VISTO

Dunque l'espressione generale (nella forma SP) è una somma di tutti i mintermini di n ingressi.

$$\Rightarrow U = \sum_{i=0}^{2^n-1} m_i \cdot F(i)$$

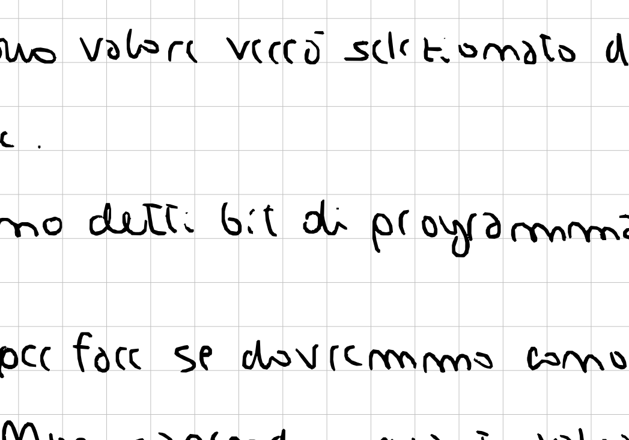
es. da m_0 a m_3

Nota: in un MUX programmabile:
- Le n variabili d'ingresso fungono da segnali di controllo ($x_1, x_2, \dots$)
- i 2^n bit di programmazione che ne definiscono il comportamento fungono da ingressi del MUX

Per riassumere
F(X) = F(0), F(1), … funzione fissa (es. somma)
$X_0, X_1, \dots$ ingresso variabile (es. 001)
Il mux restituisce una specifica F(i) a seconda della configurazione $X_0, X_1, \dots$ in ingresso.

ES.

$X_1 X_2$	F
00	F(00)
01	F(01)
10	F(10)
11	F(11)



=> Posso mandare qualsiasi funzione in ingresso, a prescindere da quale sia ogni suo valore verrà selezionato da $X_1 X_2$ => si parla di MUX programmabile.
F(0,0), F(0,1), … sono detti bit di programmazione

	$X_1 X_2$	F
m_0	00	F(00)
m_1	01	F(01)
m_2	10	F(10)
m_3	11	F(11)

per fare SP dovremmo conoscere quando F vale 1.
Non sapendo quanto valga a priori F(i), possiamo mettere in and tutti i m_i con F_i, e poi sommarli
=> Se F_i vale 0 allora semplicemente non compare il certo m_i nell'espressione (che è giusto da definizione di SP).

$$U = \sum_{i=0}^{2^n-1} m(i) \cdot F(i) \Rightarrow \text{Sintesi generale}$$

Stiamo dunque selezionando un F_i in ingresso mediante i segnali di controllo => Si attua con i MUX

ALBERI DI MUX

Quando si hanno molte variabili la funzione può diventare molto complicata. Ci si conduce al t. di Shannon, che permette di scomporre una funzione di n variabili in due funzioni di n-1 variabili nel seguente modo:

$$F(X_1, \dots, X_n) \begin{cases} \to SP: & !X_n F(X_1, \dots, X_{n-1}, 0) + !X_n F(X_1, \dots, X_{n-1}, 1) \\ \to PS: & !X_n + F(X_1, \dots, X_{n-1}, 0) \cdot !X_n + F(X_1, \dots, X_{n-1}, 1) \end{cases}$$

Le $F(X_1, \dots, X_{n-1}, 0)$ e $F(X_1, \dots, X_{n-1}, 1)$ sono, come nel caso del MUX programmabile, funzioni di cui non conosciamo a prescindere i valori.
=> Si può realizzare il T. di Shannon con un MUX a 2 vie, con $F(X_1, \dots, X_{n-1}, 0)$ e $F(X_1, \dots, X_{n-1}, 1)$ ingressi e X^n come unico segnale di controllo

Verifichiamolo
- Prendiamo la funzione $F(x_0, x_1, x_2, x_3)$
- t. di Shannon usando x_0 (SP)
  $\Rightarrow F(x_0, x_1, x_2, x_3) = !x_0 \cdot F(1, x_1, x_2, x_3) + x_0 \cdot F(0, x_1, x_2, x_3)$
- chiamiamo per semplicità le due F e_0 e e_1
  $\Rightarrow F(x_0, x_1, x_2, x_3) = !x_0 \cdot e_0 + x_0 \cdot e_1$ ← è esattamente la descrizione di un MUX a due vie.

Riapplicando nuovamente il T. di Shannon:

$$e_0 = !x_1 \underbrace{(0, 0, x_2, x_3)}_{e_{00}} + x_1 \underbrace{(0, 1, x_2 x_3)}_{e_{01}}$$
$$e_1 = !x_1 \underbrace{(1, 0, x_2, x_3)}_{e_{10}} + x_1 \underbrace{(1, 1, x_2 x_3)}_{e_{11}}$$

$\Rightarrow e_0 = !x_1 e_{00} + x_1 e_{01}$ $e_1 = !x_1 e_{10} + x_1 e_{11}$
$\Rightarrow F(X_0 X_1 X_2 X_3) = !x_0 e_0 + x_0 e_1$
$= \underbrace{!x_0 !x_1 e_{00} + !x_0 x_1 e_{01} + x_0 !x_1 e_{10} + x_0 x_1 e_{11}}$
↑ definizione di un albero di MUX a 2 vie di due livelli / MUX a 4 vie!

Da qui posso realizzare la t.d.v come 2 sotto-t.d.v (nel caso di un MUX2, o 2^n sotto-t.d.v nel caso di 2^n vie)

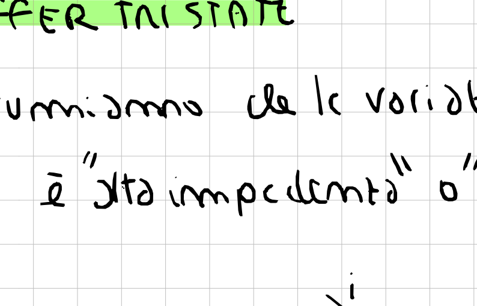
	Cin a b	Cout
Cin = 0	0 0 0	0
	0 0 1	0
	0 1 0	0
	0 1 1	1
Cin = 1	1 0 0	0
	1 0 1	1
	1 1 0	1
	1 1 1	1

- scelgo il bit d'ingresso più significativo da usare come segnale di controllo del MUX (es. Cin)
- divido la tabella in due sottotabelle nel punto in cui cambia Cin
- Effettuo la sintesi canonica delle due sottotabelle e le faccio entrare nel MUX

⇓

$$C_{out} = !C_{in}(a \cdot b) + C_{in}(a + b)$$

↑ conviene la PS nella II sotto tabella

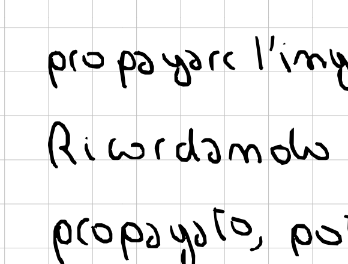


RETI COMBINATORIE ALLA BASE DI SISTEMI DIGITALI PIÙ COMPLESSI

- Buffer tristate
- Memorie
- ALU (arithmetic and logic unit)

BUFFER TRISTATE

Assumiamo che le variabili possano assumere valore 0, 1, Hiz, dove Hiz è "alta impedenza" o "tristate"

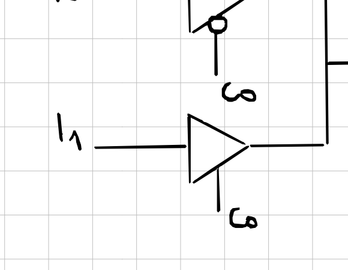


c \ i	0	1
0	Hiz	Hiz
1	0	1

variabile di controllo
=> per c=0 l'uscita è scollegata
Hiz è in realtà un valore, ma è sconosciuto

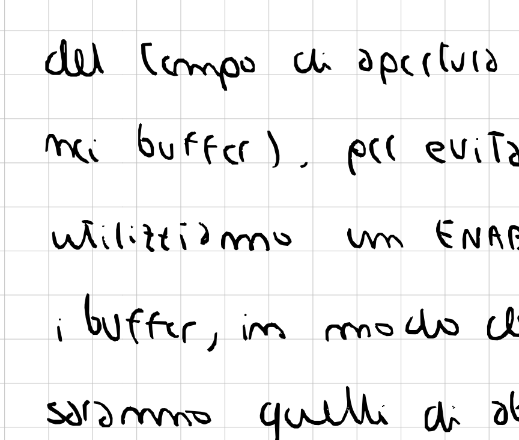
Implementazione alternativa ("con c attivo basso")

{0, 1, Hiz} = variabili ternarie



c \ i	0	1
0	0	1
1	Hiz	Hiz

I buffer tristate sono ampiamente utilizzati in quanto permettono di di propagare l'ingresso oppure no (scollegando l'uscita).
Ricordando che il MUX a 2 vie decide quale fra due ingressi viene propagato, potremmo realizzare il MUX2 con il seguente circuito:

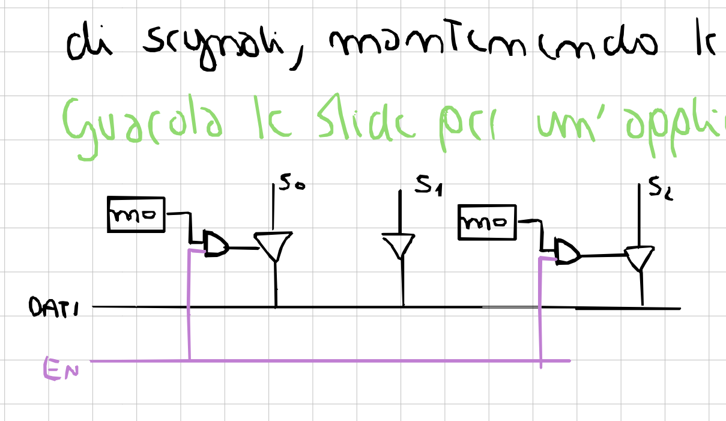


Nella pratica non si fa mai in questo modo, in quanto bastano anche solo due segnali ≠ Hiz e si crea un cortocircuito (mix di due segnali, non ammesso in logica binaria, che è poi quella che si usa nei sistemi digitali. Bisogna quindi far sì che non si verifichi mai cortocircuito. Possiamo selezionare uno solo dei due segnali tramite decoder, e siccome il tempo di chiusura (Hiz → 1/0) è maggiore del tempo di apertura (1/0 → Hiz) (passaggi di stato in risposta agli ingressi nei buffer), per evitar cortocircuiti dovuti a questa differenza di tempistiche utilizziamo un ENABLE: prima del cambio segnale, disabilitiamo tutte le i buffer, in modo che poi gli unici passaggi di stato che avverranno saranno quelli di abilitazione (Hiz → 0/1) (le chiusure), evitando quindi i cortocircuiti dovuti alle tempistiche tecniche.

Nota: si parla di
- abilitazione quando l'uscita del buffer passa da Hiz a 0/1 (chiusura del circuito)
- disabilitazione quando l'uscita del buffer passa da 0/1 a Hiz (chiusura del circuito)

Un MUX costruito tramite buffer è detto "MUX distribuito". Tali MUX sono componenti importanti in sistemi dove un elevato numero di componenti deve accedere a un canale comune
Es: nella CPU viene usato un unico canale di comunicazione (bus dati) per trasferire le informazioni tra memorie e componenti hardware.
Tale tipo di MUX consente inoltre di ridurre drasticamente il numero di segnali, mantenendo le prestazioni invariate

Guarda le slide per un'applicazione pratica



Lezione 04/11/25

Per riassumere l'utilità delle reti combinatorie (che non tengono conto del passato) nei processori avviene in diversi modi, tra cui

- Buffer tristate: gate logico che consente il passaggio o meno dell'ingresso tramite un segnale di controllo, basandosi sulla logica ternaria ({0, 1, Hiz}). Consente il collegamento di più componenti a un unico canale, a patto che un solo tristate alla volta sia abilitato (altrimenti si ha cortocircuito)
- Memorie: dispositivi in grado di immagazzinare informazioni. Tali informazioni sono accessibili tramite indirizzi, che sono selezionati mediante dei decoder, che vanno a selezionare gli indirizzi
- ALU: componente della CPU (è il cuore della CPU) in grado di eseguire sia operazioni aritmetiche sia operazioni logiche. Realizzabile a partire da un ADDER

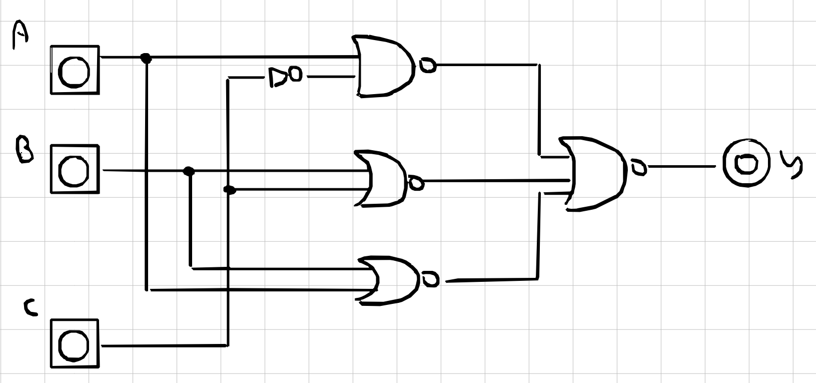
GUIDA ALL'ESAME PT. 2

Questa parte, sulle reti combinatorie, sommato all'esercizio sui codici (parte 1), copre il 50% del voto (7/14 della parte di teoria)

1) Realizzare un half-adder usando
- il n° minimo di gate (si possono usare anche operazioni non-booleane)
- SP di costo minimo

2) Disegnare lo schema logico di un full-adder usando
- un MUX come rete programmabile
- due HALF-ADDER opportunamente connessi
- espressione SP di costo minimo

PROVA X CASA



NOR

0 0	1
0 1	0
1 0	0
1 1	0

ab \ c	0	1
00	0	0
01	1	0
10	1	1
11	0	1

$z = ac' + bc$

GUARDARE LA CONTINUAZIONE

Lezione

FENOMENI TRANSITORI NELLE RETI COMBINATORIE

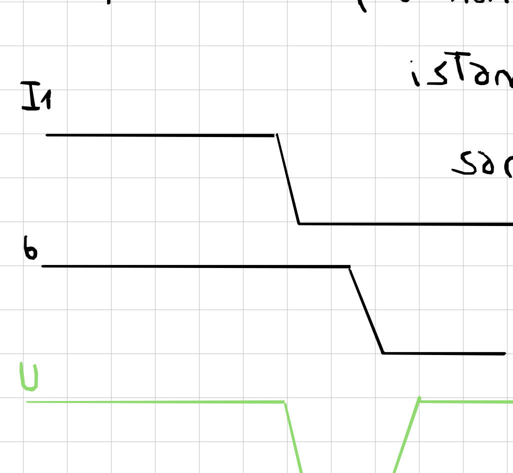
Le reti combinatorie non dipendono dalle elaborazioni precedenti. Tuttavia sono soggette a ritardi. Questo è dovuto al tempo di elaborazione.

Transitorio = intervallo di tempo in cui la rete potrebbe non fare quello che vogliamo ⇒ può avere effetti indesiderati su tutto il sistema

Si ricorda che la simultaneità di due eventi è impossibile. Due ingressi non possono cambiare nello stesso istante. Il cambiamento di due variabili è una configurazione a distanza due ⇒ nella realtà sono due spostamenti di distanza 1 uno dopo l'altro. Tale serie di spostamenti è detta "corsa". Questo può essere un bene o un effetto indesiderato ("glitch"). Non si può prevedere quale dei due ingressi cambi prima.

Es:

I1 \ ab	00	01	11	10
0	0	0	0	1
1	0	1	1	1



Sono in $a=1, b=1, I_1=1 \rightarrow U=1$. Se $b=0$ e $I_1=0$ cambiamo, e I_1 cambia per primo, ho un istante in cui $a=1, I_1=0, b=1$ (b non è ancora arrivato I_1 aggiornato) ⇒ $U=0$ per un istante. Nota che se fosse cambiato prima b non ci sarebbero stati problemi (U non sarebbe diventato 0 neanche per un istante)

⇓

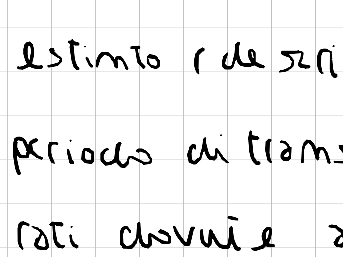
una corsa può dare dei problemi o può anche non darne

nelle reti sequenziali questo è un problemone, in quanto possono andare a "sporcare" la memoria (la storia) della rete sequenziale.

Dunque questo può essere un imprevisto dovuto alla natura dei segnali in ingresso. Tuttavia c'è anche un altro problema legato ai gate e al loro tempo di elaborazione, che si presenta anche nelle reti combinatorie:

Transitorio = intervallo di tempo in cui la rete potrebbe non fare quello che vogliamo = somma dei ritardi dei gate

Ritardo = ritardo fra la ricezione dei valori in ingresso e l'aggiornamento dell'uscita



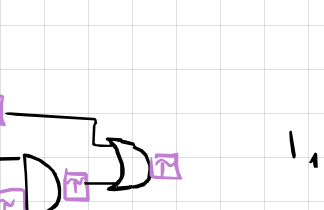
τ = ritardo

└ si conta come ritardo del sistema il ritardo della serie di componenti più lunga

⇒ le tabelle e le mappe di K. viste finora descrivono un caso di transitorio estinto (descrivono come è effettivamente l'uscita finale) ma nel periodo di transitorio si possono verificare dei glitch. Gli effetti indesiderati dovuti ai ritardi di elaborazione sono detti ALEE

ALEE STATICHE: durante il transitorio, l'uscita assume temporaneamente un altro valore

es:



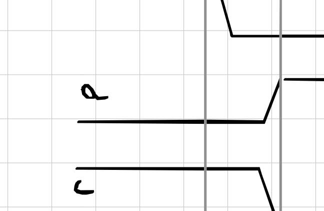
← metto 1 → ritardo → 1

cambio l'ingresso in 0 → ritardo → 0

↑

durante il ritardo è ancora 1!

r m



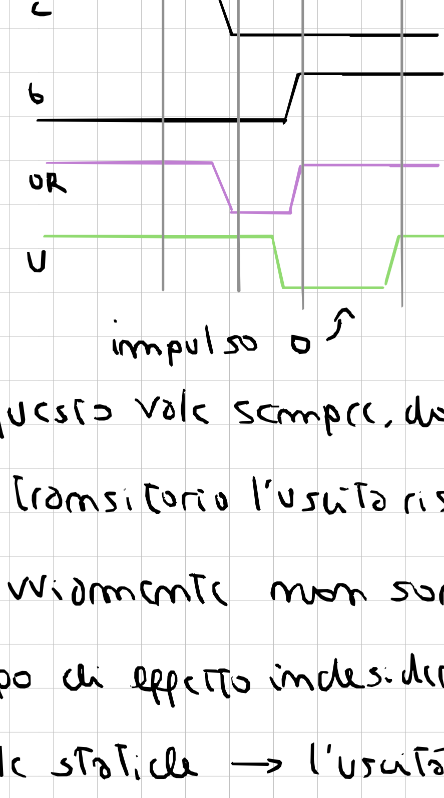
I1 \ bA	00	01	11	10
0	0	0	0	1
1	0	1	1	1

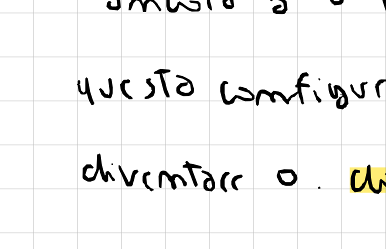
parto da 111. Cambio A da 1 a 0.

a = uscita not

b = uscita and

c = uscita dell'altro and





dopo un ritardo τ a→1, c→0 ma b rimane ancora a 0 (si aggiornerà dopo 2τ) ⇒ dopo 2τ questa configurazione errata arriva in uscita, facendola diventare 0. Dopo 3τ il transitorio sarà estinto e la uscita sarà effettivamente quella desiderata (quella della tabella)

questo vale sempre, dopo il transitorio l'uscita rispecchia quella della T.D.V.

Ovviamente non sono sempre presenti per ogni variazione di ingresso questo tipo di effetto indesiderato.

Alee statiche → l'uscita "glitcha" una sola volta.

Alee dinamiche → l'uscita "glitcha" più volte ⇒ avvengono solo in reti combinatorie con più livelli. Non avremo mai a che fare con alee dinamiche perché cercheremo sempre di realizzare reti di al massimo 2 livelli tramite sintesi. Sintesi SP e PS non presentano mai alee dinamiche (se è un livello di AND, completamente in un secondo livello di un unico OR)

Alea di 1: impulso 0 indesiderato $\overset{\text{durante il transitorio}}{\surd}$ non presente nella t.d.v. (quando l'uscita dovrebbe essere 1)

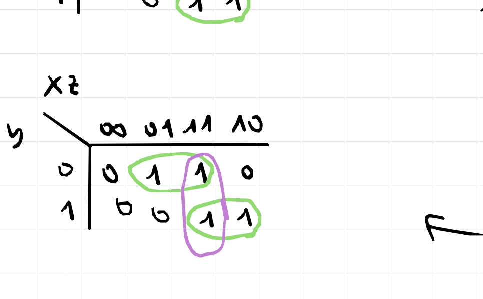
Alea di 0: impulso 1 indesiderato durante il transitorio non presente nella t.d.v. (quando l'uscita dovrebbe essere 1)

Una rete S.P. soffre di alee di 0, una P.S. soffre di alee di 1.

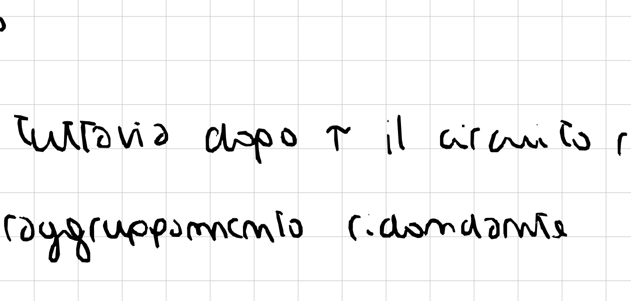
Questo perché nella S.P. gli AND del primo livello $\overset{\text{possono andare}}{\surd}$ a 0 per un certo istante ⇒ entrano nell'OR ⇒ l'uscita vale 0 per un attimo

L'alea di tipo 1 si presenta sulle reti SP e causa un impulso "0" indesiderato

gli AND si scambiano di ruolo



⇒



y \ xz	00	01	11	10
0	0	1	1	0
1	0	0	1	1

↑ ~→ U = z!y + xy

forse il prof ha sbagliato

il passaggio di y da 1 a 0 significa passare da un gruppo all'altro

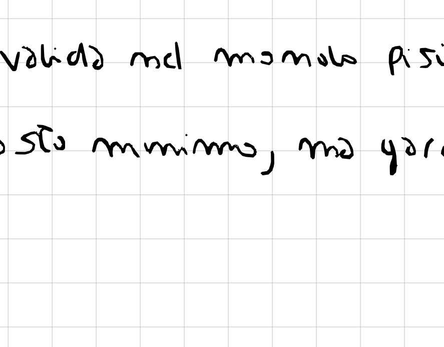
↳ tuttavia dopo τ il circuito rispecchia il raggruppamento ridondante

y \ xz	00	01	11	10
0	0	1	1	0
1	0	0	1	1

Aggiungendo tale raggruppamento ridondante all'espressione otteniamo un circuito che evita l'alea

U = z!y + xy + xz ⟹

└ termine anti-alea



Questa soluzione è valida assumendo che due ingressi non cambino simultaneamente ⇒ assunzione molto valida nel mondo fisico (reale). Ovviamente la rete ottenuta non è di costo minimo, ma garantisce il funzionamento desiderato

Nota che il problema nell'SP si ha nel passaggio 1→0

Si ha un caso duale per i circuiti realizzati mediante P.S. (alee statiche di 0)

ES.

ab \ cd	00	01	11	10
00	1	1	0	0
01	1	1	0	1
11	0	1	1	1
10	0	1	1	0

- raggruppamenti "fondamentali" (di costo minimo)
- termini anti-alee

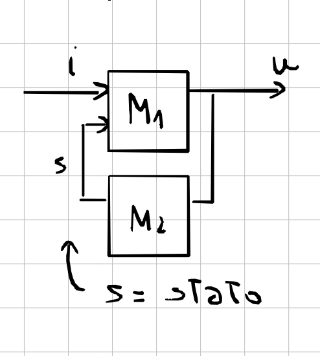
Abbiamo visto il problema dei ritardi dei gate, presenti anche nelle reti combinatorie. Mancano da vedere le corse (i ritardi dovuti al fatto che due ingressi non cambiano mai all'unisono)

Lezione 11 / 11 / 25

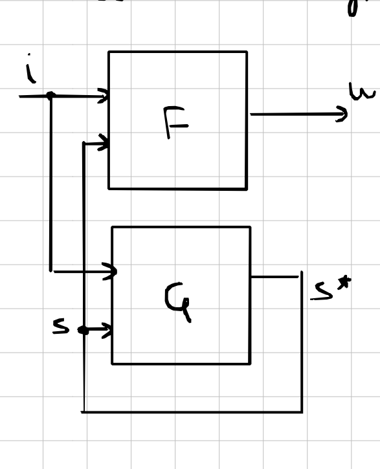
RETI SEQUENZIALI

Reti il cui comportamento dipende dalla storia delle elaborazioni precedenti (esempio lampada, esempio cassaforte).

Per memorizzare si realizza una combinazione di reti logiche in retroazione. Prima di iniziare l'elaborazione t+1, è necessario che l'elaborazione t sia conclusa, in modo da ottenere e mettere in retroazione il risultato di tale elaborazione.



Se una rete ha retroazioni, allora è sequenziale. Il comportamento della rete è descritto da funzioni delle variabili di ingresso E di stato.

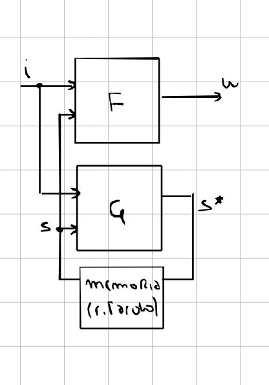


Una qualsiasi rete sequenziale può essere scomposta in due funzioni principali:

- F: definisce il comportamento dell'uscita u della rete. Dipende dallo stato e optionalmente dall'ingresso
- G: definisce come lo stato corrente s commuta ("come si trasforma") nello stato futuro s*. Dipende dallo stato attuale e dall'ingresso.

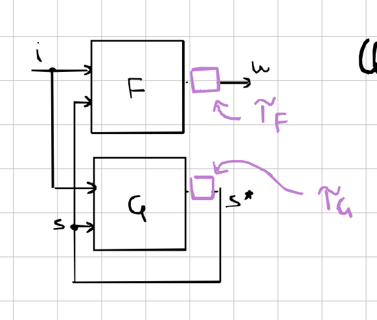
Rete sequenziale asincrona: oltre a quello di elaborazione, non c'è ritardo nella retroazione ⇒ la rete non è in grado di riconoscere configurazioni di ingresso (es: in ingresso arriva 0101 → la retroazione lo elabora (t. di elaborazione) e restituisce il risultato in ingresso → 0101 è ancora uguale perché magari l'ingresso non è ancora cambiato → la retroazione ri-elabora lo stesso ingresso, introducendolo in memoria come se fosse un nuovo ingresso)

Rete sequenziale sincrona: viene introdotto un ritardo esplicito T nella retroazione, il che permette di distinguere tra ingressi differenti)



↙ reti sequenziali sincrone

Si definiscono τ_F e τ_G i transitori delle funzioni F e G, dovuti alle elaborazioni.



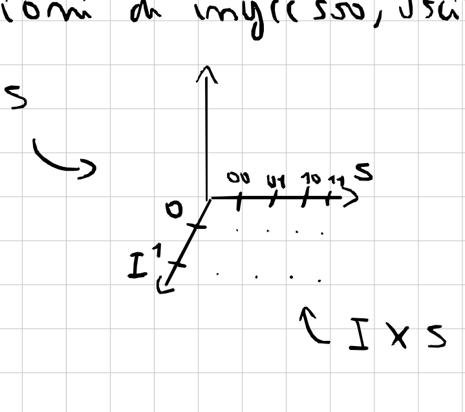
Questo impatta le reti sequenziali asincrone, ma non le reti sequenziali sincrone se T (ritardo esplicito) >> τ_G.

RETI SEQUENZIALI ASINCRONE

Il comportamento di una qualsiasi rete sequenziale può essere descritto da un modello, detto automa a stati finiti, descritto da una specifica pentupla {I, U, S, F, G} con:

I, U, S insiemi delle possibili configurazioni di ingresso, uscita e stato

F, G funzioni il cui dominio è I × S ↪



F: S × I → U

S \ I	I_1	I_2
S_1	$F(S_1, I_1)$	$F(S_1, I_2)$
S_2	$F(S_2, I_1)$	$F(S_2, I_2)$

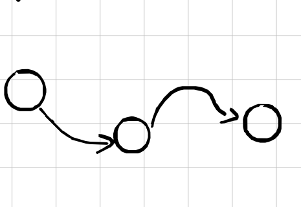
G: S × I → S

S \ I		
	$G(S_1, I_1)$ $F(S_1, I_1)$	…
	…	…

Per ogni coppia ingresso-stato, G definisce uno stato futuro e di conseguenza una uscita.

GRAFO ORIENTATO

Rappresentazione del comportamento di un automa



↝ archi / rami O = nodi

A ogni stato si associa un nodo, a ogni ingresso si associa un ramo uscente da ciascun nodo. Per ogni stato, ogni ramo uscente indica quale sarà lo stato futuro in seguito all'elaborazione della rete e il valore di uscita. Tale grafo è detto grafo degli stati o diagramma degli stati (dds).

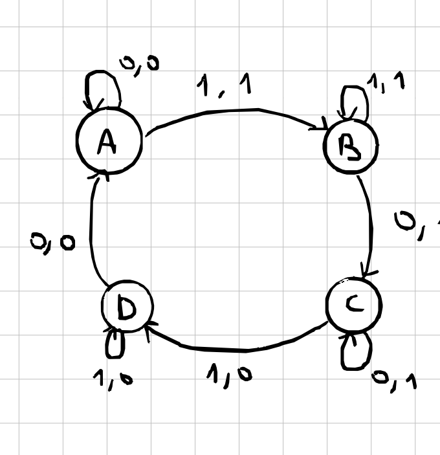
esempio: lampada

pulsante premuto ⇒ x (ingresso) = 1

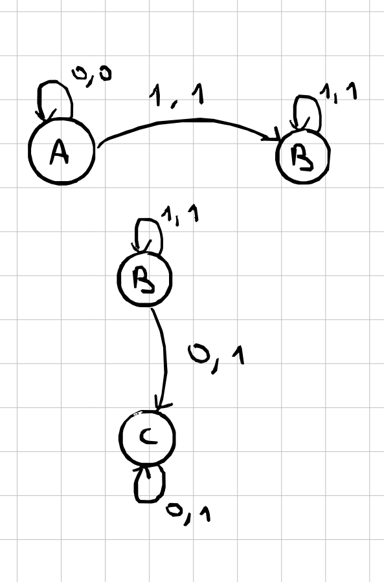
pulsante rilasciato ⇒ x (ingresso) = 0

lampada accesa ⇒ u (uscita) = 1

lampada spenta ⇒ u (uscita) = 0



stato iniziale: lampada spenta, pulsante rilasciato (x=0, u=0)

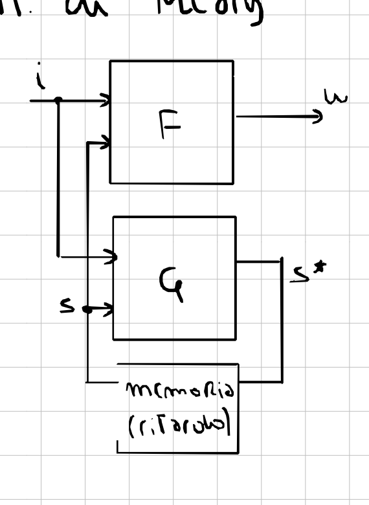


Sono nello stato iniziale (0,0). Se premo il pulsante (1, ·) l'uscita va a 1 (·, 1) entro nello stato B in cui ho pulsante premuto e luce accesa. Se mantengo (1,1) (tengo premuto mentre la luce è accesa) rimango in B, altrimenti rilascio (0, ·) e la luce rimane accesa (·, 1) → vado nello stato C, ...

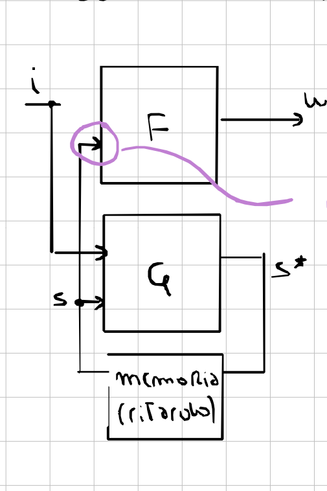
Una rete sequenziale può aggiornare l'uscita in due modi:

- macchina di Mealy: può cambiare ad una qualsiasi variazione dell'ingresso o dello stato interno ⇒ F: S × I → U
- macchina di Moore: può cambiare solo se cambia lo stato interno ⇒ F: S → U

M. di Mealy

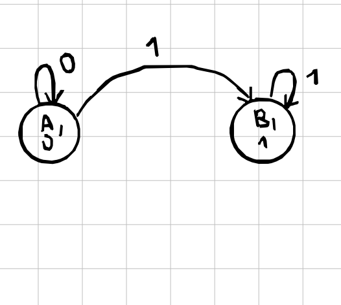


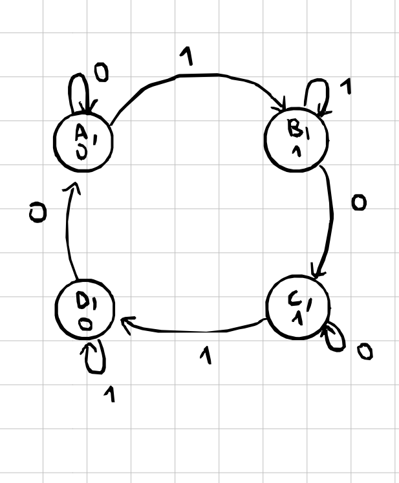
Macchina di Moore



direttamente implementata solo dallo stato interno

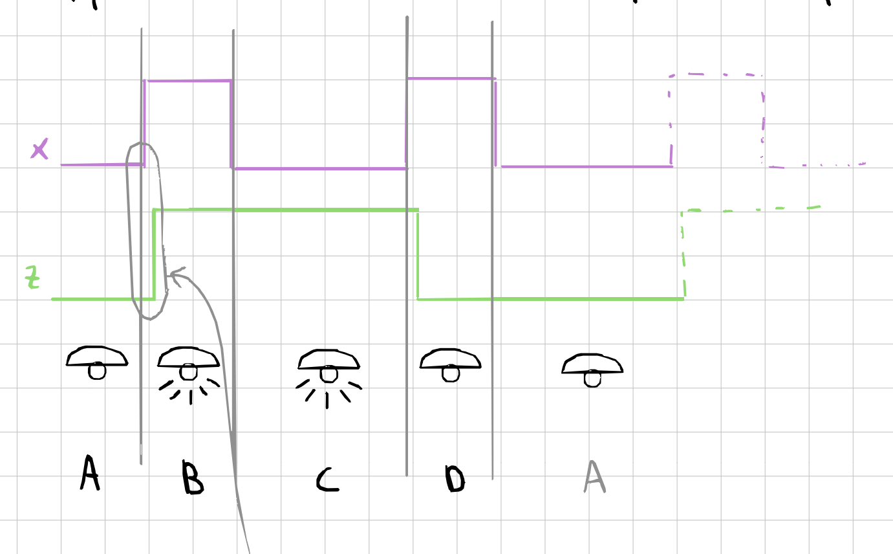
La rappresentazione della lampadina vista poc'anzi è una macchina di Mealy, in cui le uscite sono rappresentate sugli archi. Vediamo come si rappresenta nel caso della macchina di Moore (uscite rappresentate sui nodi)





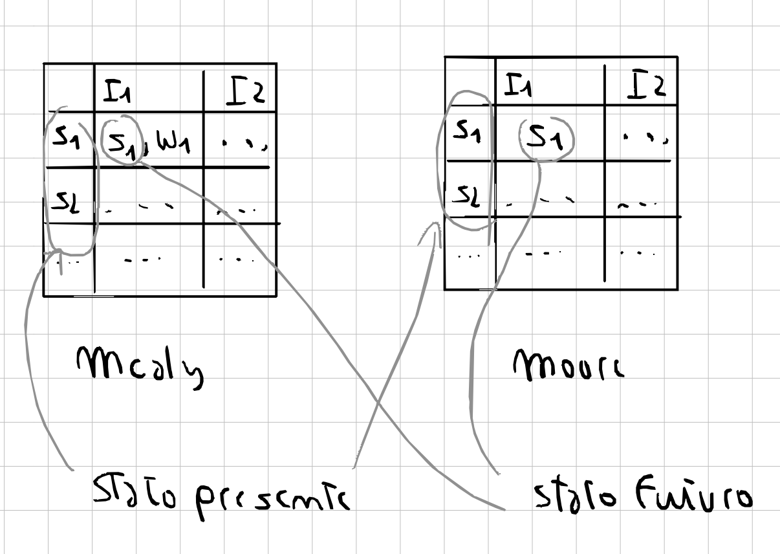
Se l'ingresso va a 1 (premo il pulsante) entro in uno stato in cui x è 1 e l'uscita vale 1, ci resto finché x rimane 1, e vado in C (in cui uscita = 1 e x=0) solo se l'ingresso cambia in 0. ⇒ non c'è dipendenza diretta dall'ingresso

Rappresentazione dell'esempio lampadina tramite forme d'onda:

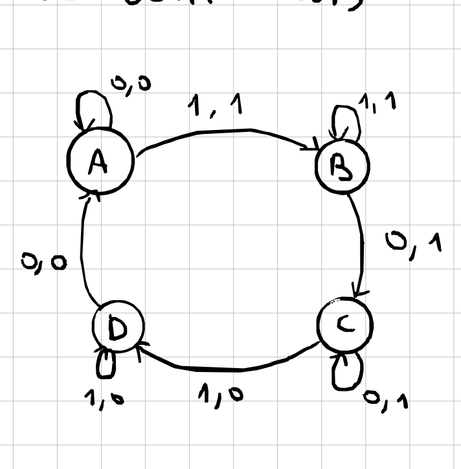


ritardo dovuto all'elaborazione

Analogamente, nelle tabelle di flusso quella di Mealy conterrà coppie uscita-stato, quella di Moore solo stati

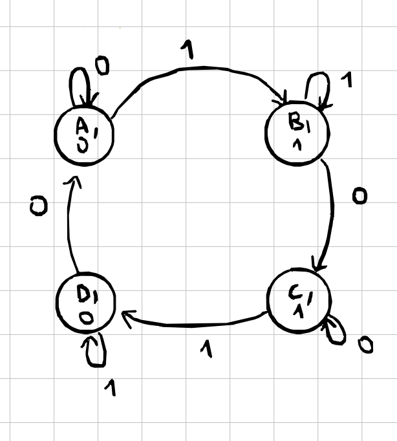


ES. con Mealy



	0	1
A	A,0	B,1
B	C,1	B,1
C	C,1	D,0
D	A,0	D,0

ES. con Moore



	0	1	z
A	A	B	0
B	C	B	1
C	C	D	1
D	A	D	0

Quando lo stato attuale è A → l'uscita vale

STATI FUTURI

Sorge ora la necessità di codificare gli stati tramite variabili binarie.

ES. codifichiamo A, B, C, D mediante $\log_2$ variabili (A, B, C, D = 4 ⇒ $\log_2 4 = 2$) y_1 e y_0 ⇒ 00, 01, 11, 10

	0	1
A	A,0	B,1
B	C,1	B,1
C	C,1	D,0
D	A,0	D,0

⇒

y_1y_0	0	1
00	00,0	01,1
01	11,1	01,1
11	11,1	10,0
10	00,0	10,0

notiamo anche il percorso per passare da uno stato all'altro

	0	1
A	A,0	B,1
B	C,1	B,1
C	C,1	D,0
D	A,0	D,0

Si può fare anche l'analogo per Moore.

SINTESI DI RETI SEQUENZIALI ASINCRONE

esempio (Mealy)

y_1y_0	0	1
00	00,0	01,1
01	11,1	01,1
11	11,1	10,0
10	00,0	10,0

x = ingressi

scompongo →

y_1y_0	0	1
00	0	0
01	1	0
11	1	1
10	0	1

mappa di y_1

y_1y_0	0	1
00	0	1
01	1	1
11	1	0
10	0	0

mappa di y_0

y_1y_0	0	1
00	0	1
01	1	1
11	1	0
10	0	0

mappa di U

nota: presentano alee statiche, pericolose nelle reti con retroazione

individuo i raggruppamenti

y_1y_0	0	1
00	0	0
01	1	0
11	1	1
10	0	1

y_1y_0	0	1
00	0	1
01	1	1
11	1	0
10	0	0

y_1y_0	0	1
00	0	1
01	1	1
11	1	0
10	0	0

$y_1 = y_0 !x + y_1 x$

$y_0 = y_0 !x + !y_1 x$

$u = y_0 !x + !y_1 x$

eliminiamo le alee statiche aggiungendo la ridondanza (termini anti-alea)

y_1y_0	0	1
00	0	0
01	1	0
11	1	1
10	0	1

y_1y_0	0	1
00	0	1
01	1	1
11	1	0
10	0	0

y_1y_0	0	1
00	0	1
01	1	1
11	1	0
10	0	0

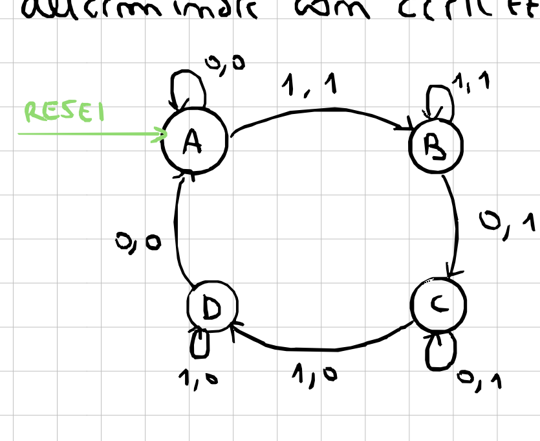
$y_1 = y_0 !x + y_1 x + y_1 y_0$

$y_0 = y_0 !x + !y_1 x + !y_1 y_0$

$u = y_0 !x + !y_1 x + !y_1 y_0$

Le alee statiche sono un problema nelle reti sequenziali asincrone perché la configurazione errata che generano viene processata dalla retroazione e quindi considerata nello storico delle elaborazioni, rendendo lo stato attuale diverso rispetto a quello che dovrebbe essere.

Una volta realizzata a livello pratico una rete, non è possibile determinare con certezza quale stato sarà quello iniziale. Per questo



Viene aggiunto un circuito RESET in grado di forzare la rete a un determinato stato (es. A (00)) (guarda schematico sulle slide)

È necessario, anche se dato per scontato fino ad ora, che nodi adiacenti siano codificati con configurazioni adiacenti

	0	1
A	A,0	B,1
B	C,1	B,1
C	C,1	D,0
D	A,0	D,0

✓ ⇒

y_1y_0	0	1
00	00,0	01,1
01	11,1	01,1
11	11,1	10,0
10	00,0	10,0

	0	1
A	A,0	B,1
B	C,1	B,1
C	C,1	D,0
D	A,0	D,0

✗ ⇒

y_1y_0	0	1
00	00,0	01,1
01	10,1	01,1
10	10,1	11,0
11	00,0	11,0

distanza 2

cambia sempre prima un bit poi l'altro ⇒ ci porterà in uno stato indesiderato ⇒ corsa critica

y_1y_0	0	1
00	00,0	01,1
01	10,1	01,1
10	10,1	11,0
11	00,0	11,0

Se cambia prima y_1,

stato 01 → stato 11 → stato futuro 00

⇒ l'ingresso però è sempre 0 ⇒

la rete passa direttamente allo stato (si dice posizione instabile)

3✓. Se invece i nodi fossero adiacenti allora distarebbero di solo 1 bit ⇒ non si passerebbe per stati indesiderati

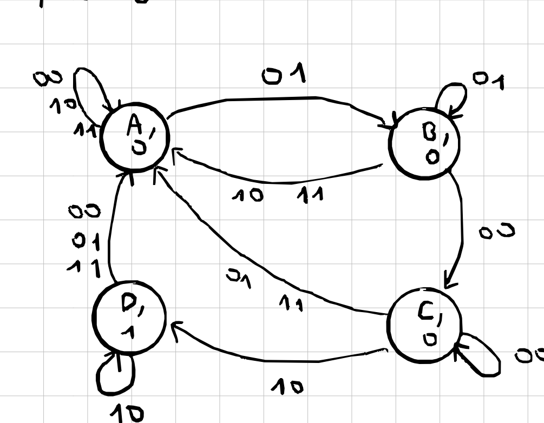
La corsa è critica quando il passaggio da una configurazione all'altra può portare a stati / configurazioni indesiderate, dovute al naturale cambiamento non-simultaneo di entrambi gli ingressi.

Vedremo nella prossima lezione che cosa fare quando non si può garantire l'adiacenza delle configurazioni.

Lezione 12/11/25

CASSAFORTE ASINCRONA

Si vuole realizzare una cassaforte che si apra con la sequenza di due ingressi x_1 e x_0 01 - 00 - 10



Costruisco la tabella

x_1x_0	00	01	11	10	z
A	A	B	A	A	0
B	C	B	A	A	0
C	C	A	A	D	0
D	A	A	A	D	1

codifico
A=00 B=01
C=11 D=10 ⟶

x_1x_0	00	01	11	10	z
00	00	01	00	00	0
01	11	01	00	00	0
11	11	00	00	10	0
10	00	00	00	10	1

percorso da fare (sequenza)

x_1x_0	00	01	11	10	z
00	00	01	00	00	0
01	11	01	00	00	0
11	11	00	00	10	0
10	00	00	00	10	1

Problema: nel grafo A e C sono adiacenti (connessi direttamente) ma non sono codificati adiacenti (A = 00, C = 11)

Supponiamo infatti di essere in C, codificato 11. Arriva un ingresso 01 → si dovrebbe andare in A, poi direttamente in B.

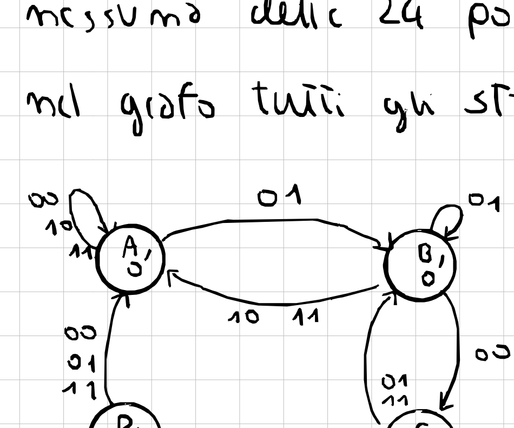
11 ⟶ 00 ⟶ 01

↑ x_1 e x_0 non cambieranno insieme

⟶ se cambia prima x_1 11 ⟶ 01 ⟶ va direttamente in B (non è un problema)

⟶ se cambia prima x_0 11 ⟶ 10 ⟶ va in D (glitch: in D z=1 ⇒ la cassaforte si apre per un istante) → poi cambia e va in 00 → l'ingresso è sempre 01 → va in B

Possiamo provare a cambiare codifica (es A=01, B=00, C=10, D=11) ma nessuna delle 24 possibili codifiche può risolvere il problema, perché nel grafo tutti gli stati sono adiacenti ad A.

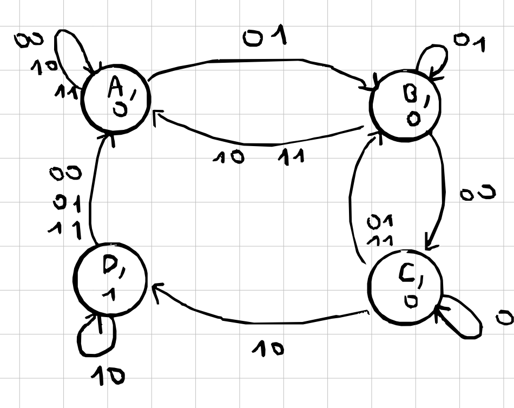


L'unico modo per risolvere il problema è a intuito: se in C arriva 11 lo passo in B, che mi riporta in A, se arriva 01 passo direttamente in B.

Questo passaggio è difficile da intuire e non sempre possibile

Un altro modo per risolvere il problema è introducendo stati ridondanti (vedi slide per esempio). Ciò però aggiunge del ritardo (ci sono più stati per cui passare).

Nel nostro circuito risolto a intuito notiamo delle criticità:



B stabile per ingresso 01 → arriva ingresso 10

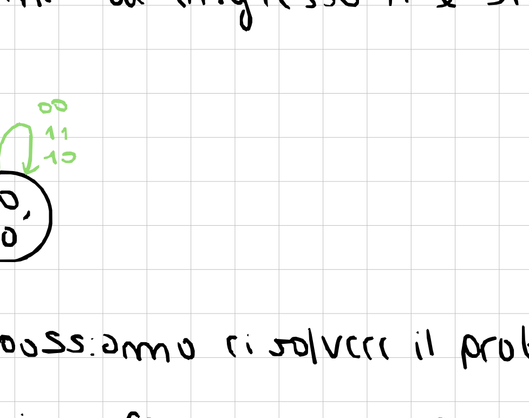
C stabile per ingresso 00 → arriva ingresso 11

D stabile per ingresso 10 → arriva ingresso 01

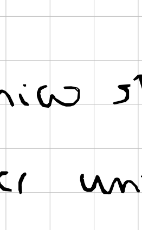
Sappiamo che se in B c'è 01, che lo rende stabile, i bit di tale ingresso non cambieranno mai insieme → B non andrà mai nella cella in cui dovrebbe andare se arrivasse 10. Stessa cosa per C e D, dunque possiamo sostituire tali stati futuri nella tabella con delle indifferenze.

⇒

x_1x_0	00	01	11	10	z
00	00	01	00	00	0
01	11	01	00	—	0
11	11	00	—	10	0
10	00	—	00	10	1



Notiamo ora che per tre configurazioni di ingresso A (00 è stabile (rimane A (00))

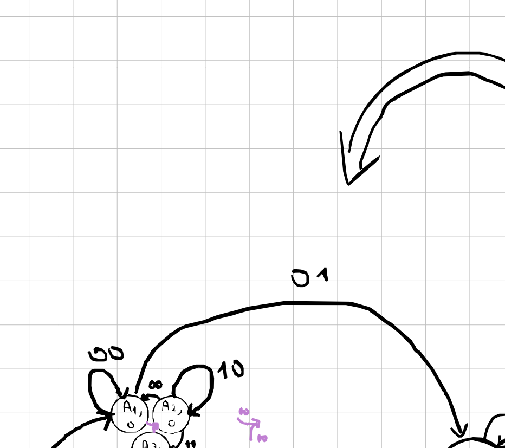


x_1x_0	00	01	11	10	z
00	00	01	00	00	0
01	11	01	00	—	0
11	11	00	—	10	0
10	00	—	00	10	1

⇒ Non possiamo risolvere il problema attraverso indifferenze (dovremmo mettere indifferenza per 01) (?)

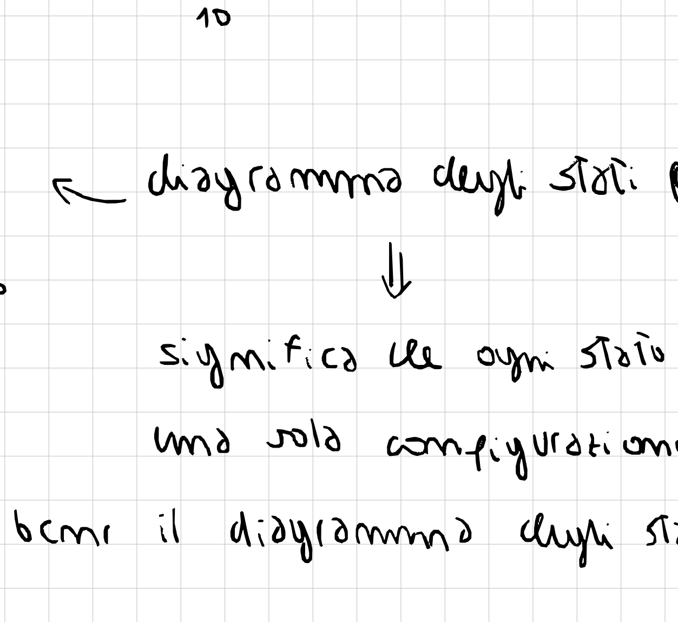
⇒ notiamo comunque che A è l'unico stato stabile per 3 configurazioni di ingresso (le altre solo per una) (es. B per 01, C per 00, ...)   (stabile per un ingresso = per quell'ingresso lo stato futuro = stato presente)

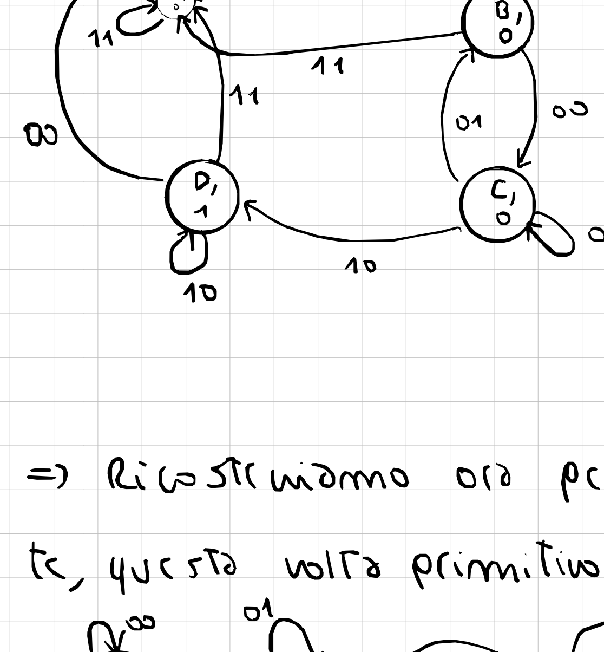
⇒ Suddividiamo A in tre sotto-stati, in cui ogni stato è stabile solo per una delle 3 configurazioni d'ingresso "di stabilità"



Colleghiamo poi D-A_1 per l'ingresso 00, D-A_3 per l'ingresso 11, B-A_3 per l'ingresso 11

⇓



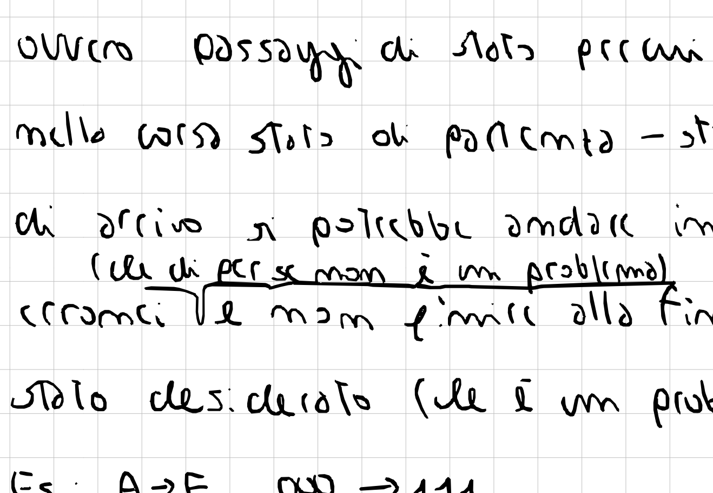


← diagramma degli stati primitivo

⇓

significa che ogni stato è stabile per una sola configurazione di ingresso

⇒ Ricostruiamo ora per bene il diagramma degli stati della cassaforte, questa volta primitivo



⇒

x_1x_0	00	01	11	10	z
A	A	B	-	F	0
B	C	B	E	—	0
C	C	B	-	D	0
D	A	-	E	D	1
E	—	B	E	F	0
F	A	-	E	F	0

⇓ codifico A=000, B=001, ...

	$y_2y_1y_0$ \ x_1x_0	00	01	11	10	z
A	000	000	001	–	111	0
B	001	011	001	110	—	0
C	011	011	001	–	010	0
D	010	000	–	110	010	1
E	110	—	001	110	111	0
F	111	000	—	110	111	0
	101	—	—	–	–	—
	100	–	–	–	–	–

Notiamo quattro corse critiche, ovvero passaggi di stato per cui nella corsa stato di partenza - stato di arrivo si potrebbe andare in stati erronei (che di per se non è un problema) e non finire alla fine nello stato desiderato (che è un problema).

Es: A→F   000 → 111

Es. cambia prima y_1 ⇒ 010 ⇒ ingresso ancora 10, siamo in D ⇒ D è stabile per 10 ⇒ resta in D ⇒ corsa critica

cambia prima y_0 ⇒ 001 ⇒ ingresso 10, siamo in B ⇒ a 10 in B corrisponde una indifferenza ⇒ problema (corsa critica)

↳ corse critiche:

- corsa F→A
- corsa A→F

corse risolte

↓

analogamente si possono risolvere anche le altre due corse critiche (B→E, E→B)

	$y_2y_1y_0$ \ x_1x_0	00	01	11	10	z
A	000	000	001	–	100	0
B	001	011	001	110	—	0
C	011	011	001	–	010	0
D	010	000	–	110	010	1
E	110	—	001	110	111	0
F	111	101	—	110	111	0
	101	100	001	100	111	–
	100	000	–	110	101	–

Risolviamo le corse indirizzando non direttamente allo stato ma prima ai due "stati non utilizzati" che a loro volta porteranno senza problemi allo stato voluto

Posso ora formulare le mappe di Karnaugh. Costruiamo una mappa per ogni bit dello stato di uscita

Es. per y_2 Spezzo la tabella in due sottotabelle

	$y_2y_1y_0$ \ x_1x_0	00	01	11	10
A	000	0	0	–	1
B	001	0	0	1	–
C	011	0	0	–	0
D	010	0	–	1	0

	$y_2y_1y_0$ \ x_1x_0	00	01	11	10
	100	0	–	1	1
	101	1	0	1	1
F	111	1	1	1	1
E	110	–	1	1	1

↙ es. stato A (000) ⇒ arriva 10 ⇒ devo andare in 100 (y_2) ⇒ per 10, dallo stato iniziale A, y_2 vale 1

$$y_2 = x_1 x_0 + x_1 !y_1 + y_2 y_1 + y_2 y_0 !x_0$$

identiche a quelle della tabella precedente ⇒ non le consideriamo

Mappa di z (Tabella di Karnaugh dell'uscita)

y_1y_0 \ y_2	0	1
00	0	–
01	0	–
11	0	–
10	1	0

⇒ $z = !y_2 y_1 !y_0$

Nelle reti viste, una volta che lo stato si è stabilizzato, all'ingresso che era entrato è corrisposta una sola uscita. Questo comportamento è detto "di tipo 1": a un cambiamento dell'ingresso corrisponde un solo cambiamento dell'uscita

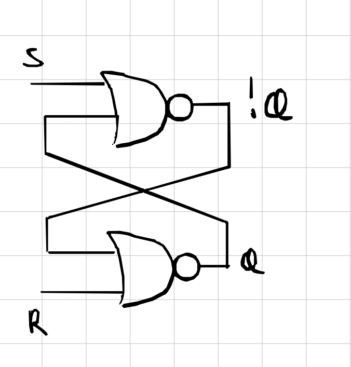
Tipo 2: cambiamento ingresso → numero limitato di cambiamenti dell'uscita

Tipo 3: cambiamento ingresso → continui cambiamenti dell'uscita ⇒ l'uscita oscilla all'infinito senza raggiungere mai un valore stabile

Con reti sequenziali asincrone è possibile realizzare solo comportamenti di tipo 1. Per ottenere una rete sequenziale di tipo 1 è sufficiente (ma non necessario) partire da un diagramma degli stati primitivo

Lezione 18/11/25

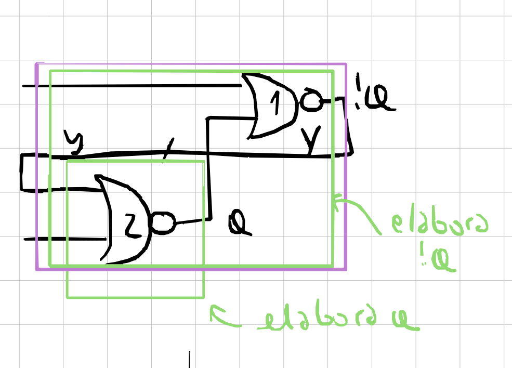
LATCH SR



S: suppone che S,R non siano mai 11 (che S·R = 0 sempre) e che S e R cambino uno alla volta

Ingressi: S, R   Uscite: Q, !Q

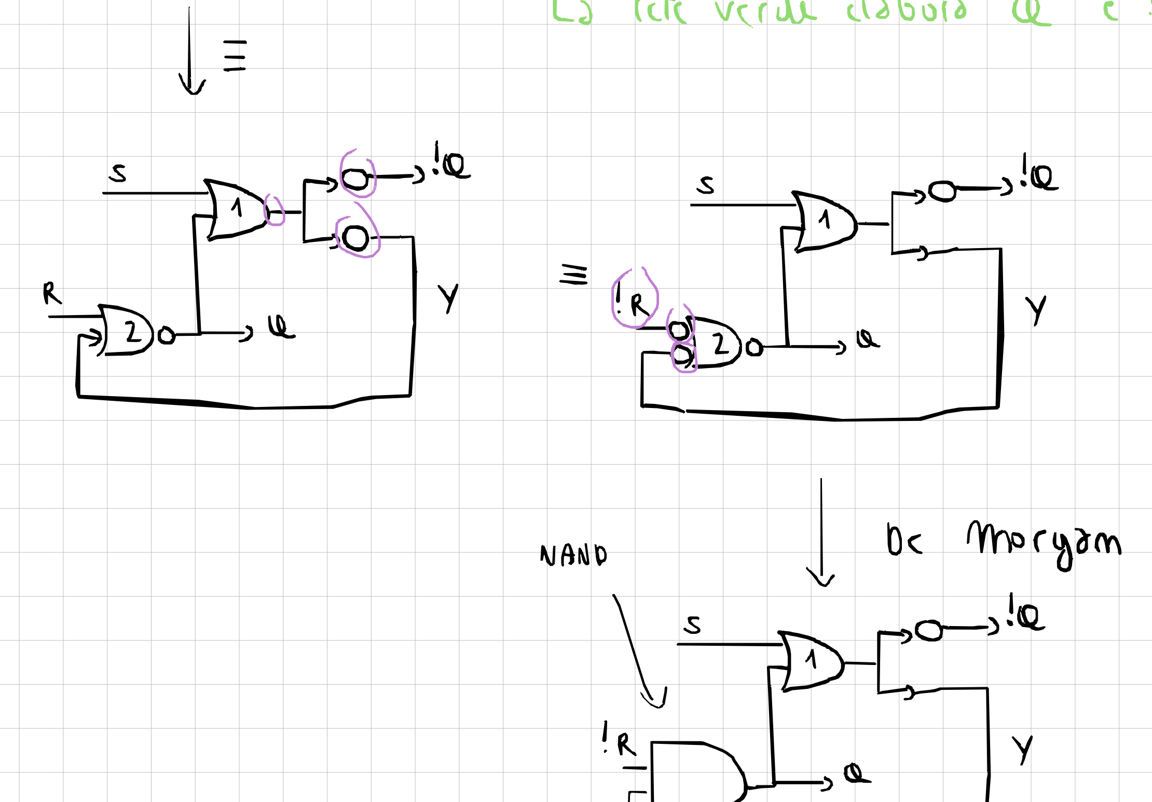
Possiamo interpretare i 2 NOR in serie e !Q in retroazione:



Y = stato futuro
y = stato presente
(Y diventa y)

La rete viola elabora lo stato Y: rete G

La rete verde elabora Q e !Q: rete F



Espressione di G: $Y = S + !Ry$

Espressione di F: $!Q = !(S + !Ry)$ $Q = !Ry$

⇒ Rete di Mealy

Da qui posso ricavare le t.d.v. (assumendo per ipotesi: SR mai 11)

y\SR	00	01	11	10
0	0	0	-	1
1	1	0	-	1

mappa di Y

y\SR	00	01	11	10
0	0	0	-	0
1	1	0	-	1

mappa di Q

y\SR	00	01	11	10
0	1	1	-	0
1	0	1	-	0

mappa di !Q

Ricaviamo la tabella di flusso (t.d.t (tabella delle transizioni))

y\SR	00	01	11	10
0	0,01	0,01	-	1,00
1	1,10	0,01	-	1,10

⇒

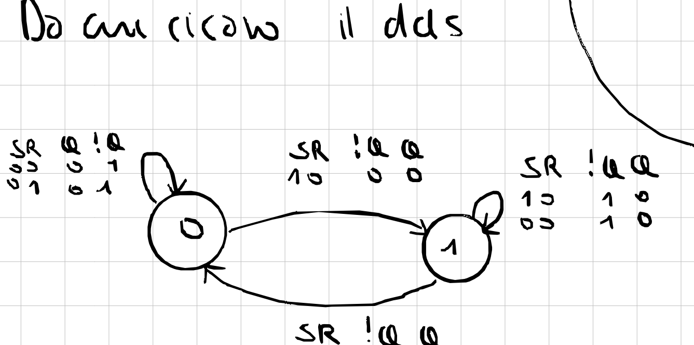
y\SR	00	01	11	10
0	0,01	0,01	-	1,00
1	1,10	0,01	-	1,10

condizioni di stabilità

↳ Y primo bit ⇒ resto nello stato 0 o nello stato 1

es. 1,10 (Y, Q, !Q)

Da cui ricavo il dds



Notiamo che SR = 10 ⇒ lo stato viene impostato a 1

SR = 01 ⇒ lo stato viene impostato a 0

⇒ Il Latch SR è una memoria a un bit

⇒ Quando S=1 porta lo stato a 1 ⇒ Set

⇒ Quando S=1 porta lo stato a 0 ⇒ reset

⇒ Quando S=0 e R=0 lo stato rimane invariato

Studiamo ora l'effetto sul solo Y:

y\SR	00	01	11	10
0	0,01	0,01	-	1,00
1	1,10	0,01	-	1,10

→ solo Y

y\SR	00	01	11	10
0	0	0	-	1
1	1	0	-	1

y\SR	00	01	11	10
0	0	0	-	1
1	1	0	-	1

stabile

Stato 1 (Y=1) ⇒ arriva 01

y\SR	00	01	11	10
0	0	0	-	1
1	1	0	-	1

stabile

Stato 0 (Y=0) ⇒ arriva 10

⇒ si può notare in generale che ogni cambiamento della configurazione SR porta in uno stato stabile ⇒ comportamento di tipo 1

Guardiamo ora alle uscite (Q e !Q). Come visto, possiamo mettere delle indifferenze sulle uscite durante i transitori

y\SR	00	01	11	10
0	0,01	0,01	-	1,00
1	1,10	0,01	-	1,10

- posizioni stabili
- transitori

y\SR	00	01	11	10
0	0,01	0,01	-	1,--
1	1,10	0,--	-	1,10

ragioniamo sulle sole uscite →

y\SR	00	01	11	10
0	0	0	-	-
1	1	-	-	1

mappa di Q

Espressione: $Q = y$

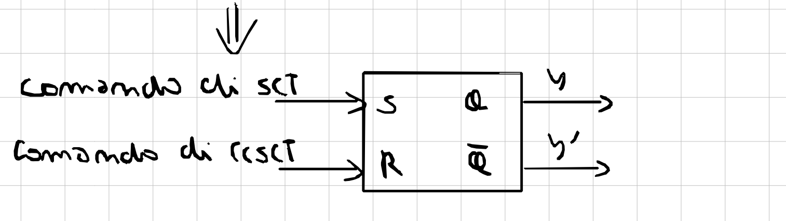
y\SR	00	01	11	10
0	1	1	-	-
1	0	-	-	0

mappa di !Q

Espressione: $!Q = !y$

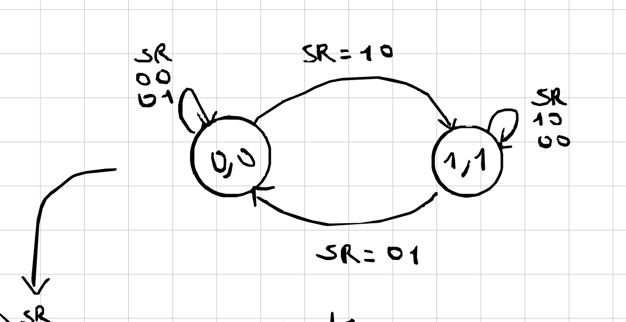
In condizioni di stabilità, Q e !Q sono uguali allo stato interno e allo stato interno negato

⇓



Il comportamento della rete segue dunque il modello di Moore

costruiamo il modello di Moore

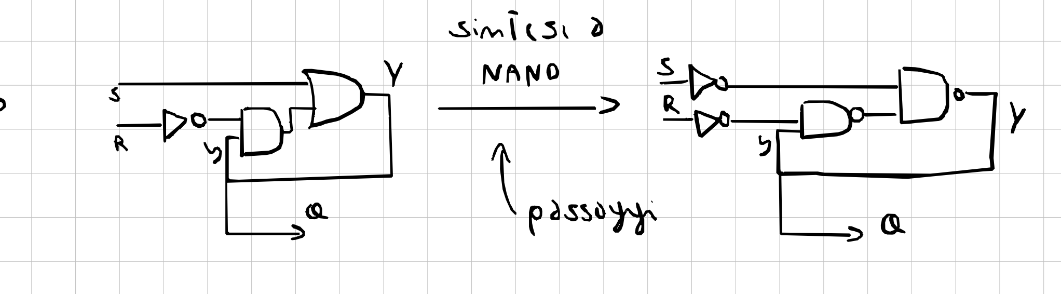


y\SR	00	01	11	10
0	0	0	-	1
1	1	0	-	1

Q	!Q
0	1
1	0

↳ $Q = Y$
$!Q = !Y$

↳ $Y = S + y!R$

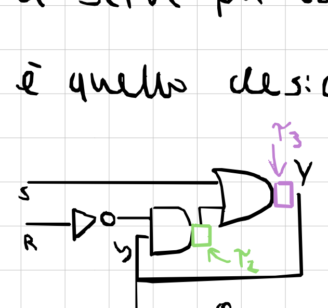


Dunque, il latch SR è una rete sequenziale asincrona la cui struttura segue il modello di Mealy, ma a transitori estinti si comporta secondo il modello di Moore

⇐ equivalente al Latch SR a NOR a cui abbiamo effettuato l'analisi

Stima della durata del transitorio del Latch SR

Ci serve per capire sotto quali ipotesi il comportamento del Latch SR è quello desiderato



Supponiamo inizialmente y=0 e SR = 00.

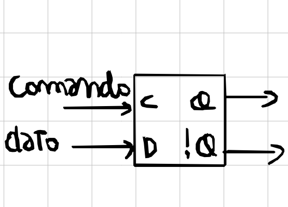
Arriva $\overset{SR}{10}$ → al termine del transitorio Y=1

⇒ $\overbrace{\text{L'OR elabora y}}^{\tau_3}$ → $\overbrace{\text{AND di R e y (retroazione)}}^{\tau_2}$ → $\overline{\text{OR elabora}}$
$\underbrace{\text{il nuovo !Q . S}}_{\tau_3}$

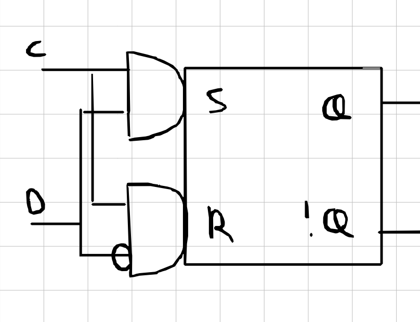
⇒ $\tau_3 + \tau_2 + \tau_3$

Latch CD

Rete in grado di memorizzare un segnale D. Si può realizzare a partire da un Latch SR   C=1 → campiona D   C=0 → mantiene il valore salvato in memoria



← Q e !Q, le uscite, procuriamo una vista sul bit in memoria



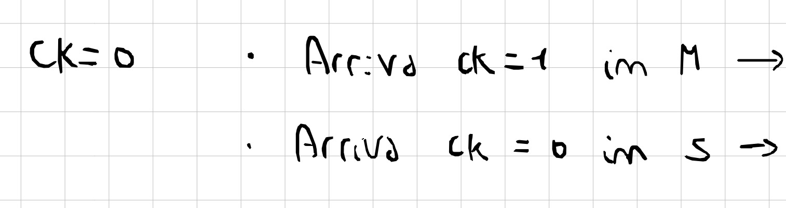
$Y = CD + y!C + yD$

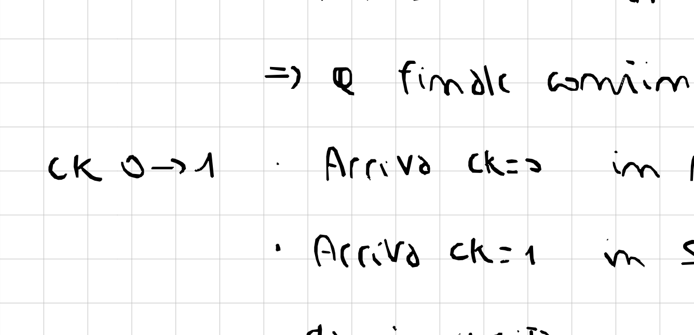
Se C, il segnale di controllo, vale 0, allora S e R varranno zero. Altrimenti, se C vale 1 il latch SR memorizza il valore di D. È un meccanismo analogo a un segnale di Enable.

Flip-Flop D

Memoria a un bit caratterizzata da un segnale di clock e un ingresso D. Quando il segnale di clock va da 0 a 1 (fronte di salita) il FFD memorizza (campiona) il valore di D. Tale valore rimane invariato fino al successivo fronte di salita.

Questo comportamento può essere realizzato con un doppio Latch SDR in cascata Master-Slave.





CK=0
- Arriva CK=1 in M → M memorizza D
- Arriva CK=0 in S → S non memorizza D

⇒ Q finale continua ad essere il valore precedente

CK 0→1
- Arriva CK=0 in M → M non memorizza D, rimane lo stato precedente
- Arriva CK=1 in S → S memorizza lo stato precedente e lo da in uscita

CK=1
- Continua ad essere la situazione precedente ⇒ S continua a fornire lo stesso valore di uscita. Stesso caso se CK torna a 0, si ripete il passaggio iniziale (Q continua ad essere l'ultimo valore memorizzato

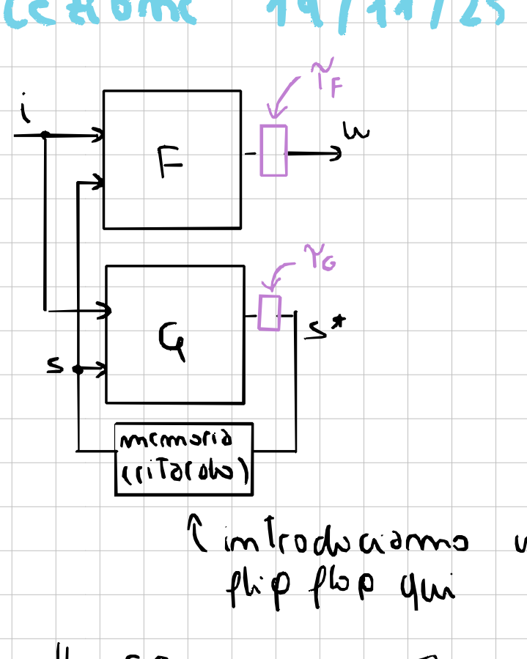
CK (clock) è un segnale con periodo costante T.

Essendo il FFD internamente composto da Latch SD, a loro volta composti da Latch SR, affinché non avvengano corse critiche il segnale D deve essere stabile

Tempo di setup τ_{SU} = Tempo per cui il segnale D deve essere stabile prima del fronte

Tempo di Hold τ_H = Tempo per cui il segnale D deve essere stabile dopo il fronte

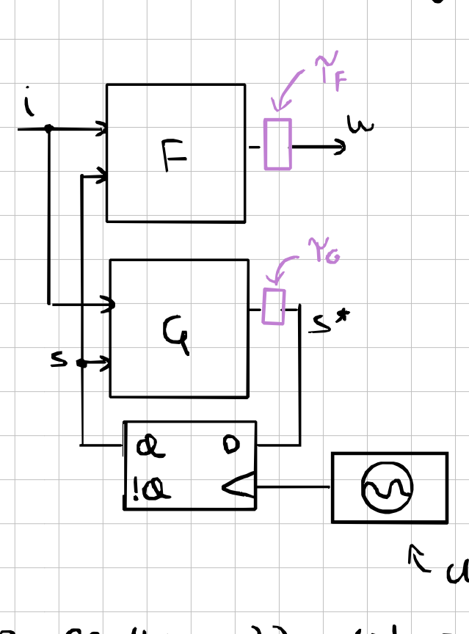
Lezione 19/11/25



↑ introduciamo un flip flop qui

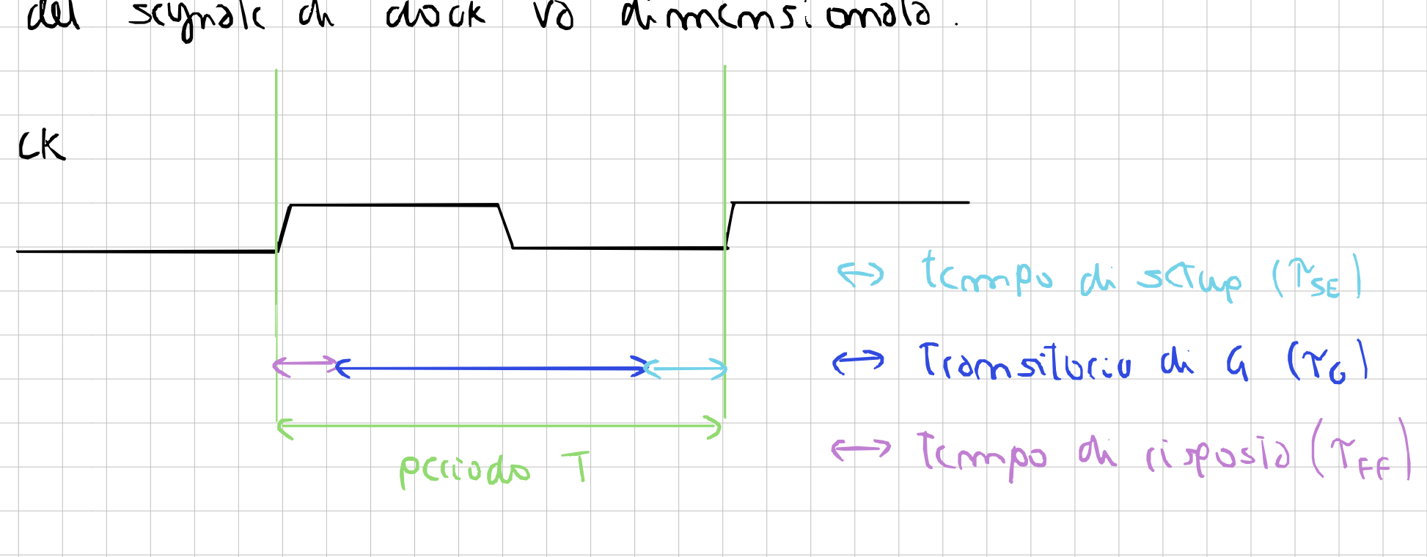
τ_G è un ritardo imprevedibile.
Il flip flop agisce come barriera di sincronizzazione. Essendo che esso trasmette il segnale solo quando clock è 1, e clock è periodico, sappiamo con precisione ogni quanto "sale" lo stato (ogni ciclo di clock)

Il FFD memorizza lo stato iniziale s. s^* (lo stato futuro) entra nel FFD MA non arriva allo slave → rimane nel master. Quando CK=1, viene rilasciato lo stato presente s, che va a finire nella rete F. FFD memorizza ora l'ex-stato futuro s^*, che ora è lo stato presente, e lo rilascerà al successivo ciclo di clock. Così per ogni fronte di salita.



In questo modo G e F rielaboreranno solo lo stato attuale fedele, evitando alee statiche e corse critiche.
Il ritardo introdotto dal clock deve essere $> \tau_G$.
Maggiore la frequenza di clock, più operazioni riescono ad essere compiute da F e G.

La frequenza del segnale di clock indica il numero di fronti di salita che si presentano per unità di tempo (MHz o GHz nei sistemi digitali moderni).
Dunque per garantire il corretto funzionamento della rete la frequenza del segnale di clock va dimensionata.


CK
↔ tempo di setup (τ_{SE})
↔ Transitorio di G (τ_G)
↔ Tempo di risposta (τ_{FF})
periodo T

⇒ T_{CK} = periodo di CK deve essere $> \tau_{SE} + \tau_G + \tau_{FF}$

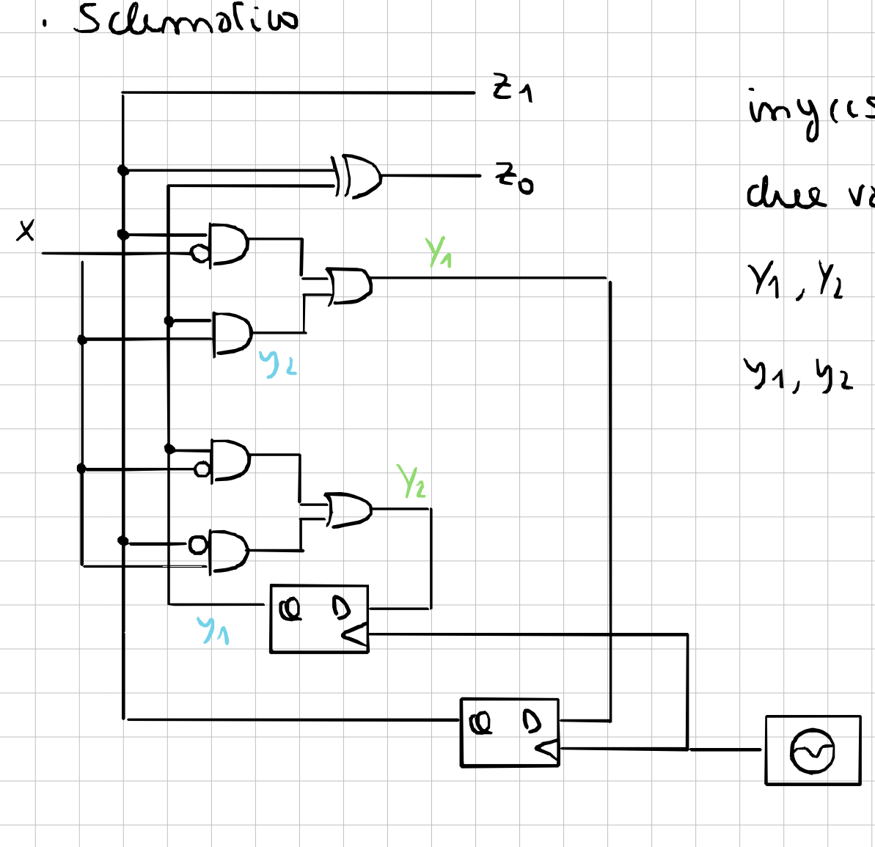
$f_{CK} = \frac{1}{T}$ $\quad f_{CK}$ deve essere $\leq \frac{1}{\tau_{FF}+\tau_G+\tau_{SE}}$

↑ frequenza di clock

Ovviamente un FFD è in grado di memorizzare (e far passare) un solo bit ⇒ per memorizzare stati a n bit occorrono n FFD in retroazione. Questi FFD devono ovviamente avere lo stesso segnale di clock. Eventuali gate sulla retroazione introdurrebbero ritardo ⇒ per questo l'elaborazione del clock all'ingresso di un FFD è una pratica sconsigliata. Cerchiamo allora di capire come strutturare questo meccanismo.

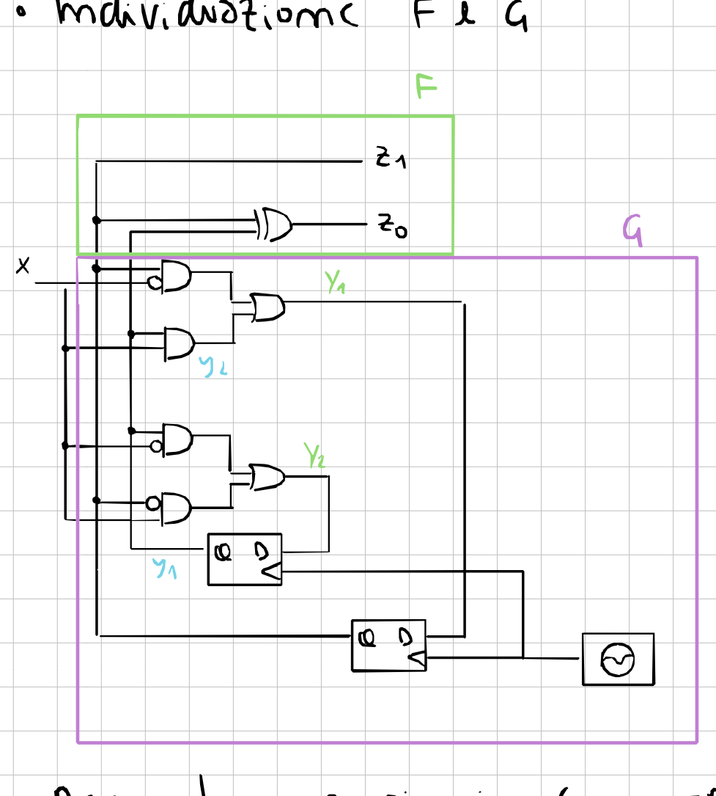
ANALISI DI RETI SINCRONE (la metodologia è pressoché identica a quella per l'analisi delle reti asincrone, a meno della ricerca delle alee statiche di cui non dobbiamo preoccuparci visto che i FFD sono sincronizzati e dunque i bit di stato "cambiano insieme")

- Schematico



ingresso x, uscite z_0 e z_1,
due variabili di stato ⇒ due FFD
Y_1, Y_2 stati futuri
y_1, y_2 stati presenti

- Individuazione F e G



- Ricavo le espressioni (es. SP)

$z_1 = y_1$
$z_0 = y_1 y_0$
$Y_1 = y_1 !x + y_0 x$
$Y_0 = y_0 !x + !y_1 x$

⇒ le uscite dipendono solo dagli stati ⇒ rete di Moore

- Ricavo le mappe e la TdT

y_1y_0 \ x	0	1
00	0	0
01	0	1
11	1	1
10	1	0

Y_1

y_1y_0 \ x	0	1
00	0	1
01	1	1
11	1	0
10	0	0

Y_0

y_1y_0	z_1
00	0
01	0
11	1
10	1

z_1

y_1y_0	z_1
00	0
01	1
11	0
10	1

z_1

TdT (metto assieme le mappe) rete di Moore ⇒ TdT di Moore

y_1y_0 \ x	0	1	$z_1 z_0$	
00	00	01	0	0
01	01	11	0	1
11	11	10	1	0
10	10	00	1	1

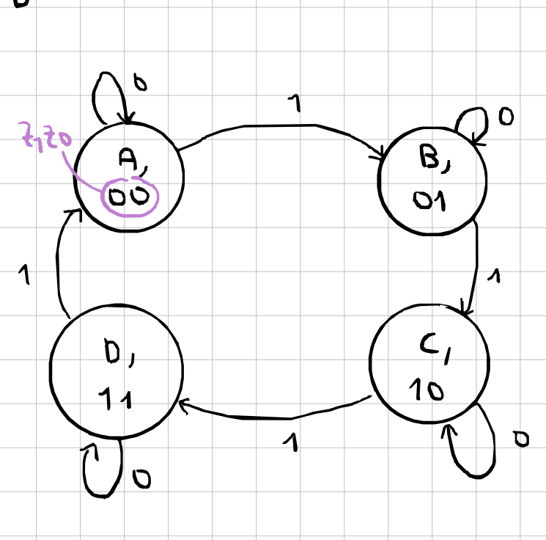
- Assegno dei nomi agli stati e ricavo il dds

00 = A 01 = B 11 = C 10 = D

⇒

	y_1y_0 \ x	0	1	$z_1 z_0$	
A	00	A	B	0	0
B	01	B	C	0	1
C	11	C	D	1	0
D	10	D	A	1	1

⇒



- Traduciamo inoltre il comportamento in forme d'onda

clock	T0	T1	T2	T3	T4	T5	T6	T7	T8	T9
x	0	1	1	0	1	0	1	0	1	0
y_1, y_0	00	00	01	11	11	10	10	00	00	01
Y_1, Y_0	00	01	11	11	10	10	00	00	01	01
$z_1 z_0$	00	00	01	10	10	11	10	00	00	01

← cicli di clock
← ingressi scelti a caso in ogni istante
stato futuro — stato attuale

SINTESI DI UNA RETE SINCRONA (analoga alla sintesi di una rete asincrona ma i FFD eliminano il problema dei transitori)
Progettiamo una rete che apra una cassaforte
Due ingressi: x_1, x_0 uscita: z
z = 1 cassaforte aperta in seguito alla ricezione 01 - 00 - 00
Scegliamo il modello di Moore

- Forme d'onda

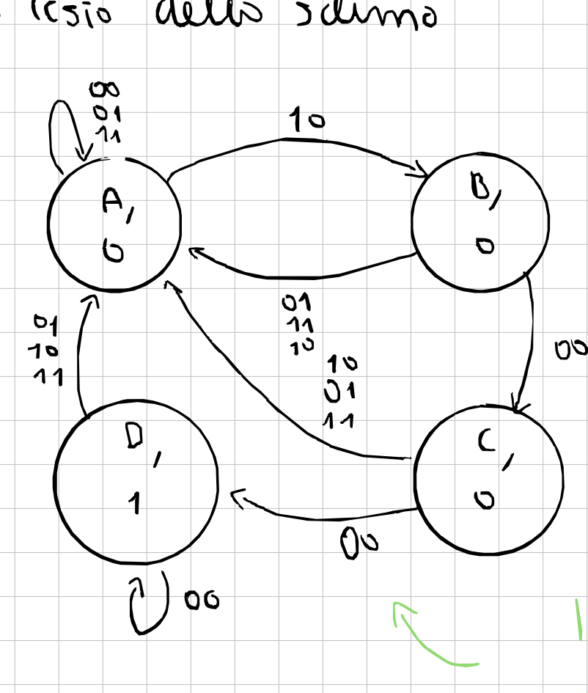
clock	T0	T1	T2	T3	T4	T5	T6	T7
x_1, x_0	00	01	10	01	00	00	00	00
Z	0	0	0	0	0	0	1	1

← ingressi scelti a caso in ogni istante

↑ quando arriva questo 00, l'uscita è ancora in 0
questo perché la rete è di Moore, dove l'uscita dipende unicamente dallo stato (che non è ancora stato aggiornato)

- Realizzazione dds

Assumiamo lo stato iniziale A, con cassaforte chiusa, e costruiamo il resto dello schema



← non torniamo diretto in B dopo l'ingresso 10, andiamo in A che porterà in B

la rete sequenziale è in grado di distinguere due ingressi uguali consecutivi (grazie ai FFD)

Inoltre le reti sequenziali non hanno problemi legati a determinate configurazioni d'ingresso (cambio di più bit contemporaneamente)

* perché l'uscita dipende dagli stati, e lo stato rimane C finché non arriva un altro 00 ⇒ l'uscita rimane 0 fino a che non si passa in D (se cambia nel frattempo la configurazione x_0x_1, lo stato torna in A, non va diretto in D)

- Costruisco la tabella

	00	01	11	10	z
A	A	B	A	A	0
B	C	A	A	A	0
C	D	A	A	A	0
D	D	A	A	A	1

- Codifico A, B, C, D

A = 00 B = 01 C = 11 D = 10 (non occorre verificare le adiacenze)

	00	01	11	10	z
A	A	B	A	A	0
B	C	A	A	A	0
C	D	A	A	A	0
D	D	A	A	A	1

⇒

		00	01	11	10	z
A	00	00	01	00	00	0
B	01	11	00	00	00	0
C	11	10	00	00	00	0
D	10	10	00	00	00	1

- Mappe di Karnaugh

		00	01	11	10	z
A	00	00	01	00	00	0
B	01	11	00	00	00	0
C	11	10	00	00	00	0
D	10	10	00	00	00	1

⇒

y_1y_0 \ x_1x_0	00	01	11	10
00	0	0	0	0
01	1	0	0	0
11	1	0	0	0
10	1	0	0	0

Y_1

y_1y_0 \ x_1x_0	00	01	11	10
00	0	1	0	0
01	1	0	0	0
11	0	0	0	0
10	0	0	0	0

Y_0

y_1y_0	z
00	0
01	0
11	0
10	1

- Sintesi minima

y_1y_0 \ x_1x_0	00	01	11	10
00	0	0	0	0
01	1	0	0	0
11	1	0	0	0
10	1	0	0	0

y_1y_0 \ x_1x_0	00	01	11	10
00	0	1	0	0
01	1	0	0	0
11	0	0	0	0
10	0	0	0	0

y_1y_0	z
00	0
01	0
11	0
10	1

$z = y_1 !y_0$

$Y_1 = y_0 !x_0 !x_1 + y_1 !x_0 !x_1$
$Y_0 = !y_1 !y_0 !x_1 x_0 + !y_1 y_0 !x_1 !x_0$

- Schematico (slide 47 foglio 3.4)

Ricorda: su ogni retroazione (2 in questo caso, Y_1 e Y_0) va messo un FFD

- Si può ora finire la forma d'onda (slide 53)

Nota:

clock	T0	T1	T2	T3	T4	T5	T6	T7
x_1, x_0	00	01	10	01	00	00	00	00
Z	0	0	0	0	0	0	1	1

↳ con Mealy sarebbe stato

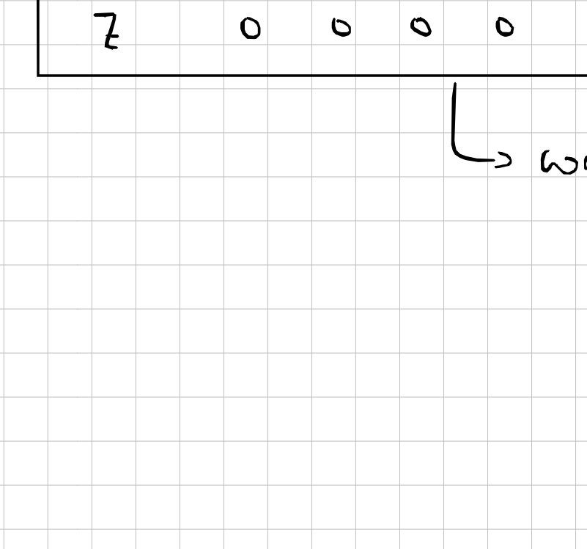
clock	T0	T1	T2	T3	T4	T5	T6	T7
x_1, x_0	00	01	10	01	00	00	00	00
Z	0	0	0	0	0	1	1	1

ovviamente la rete sarebbe sintetizzata diversamente da come fatto

RETE CON COMPORTAMENTO DI TIPO 2

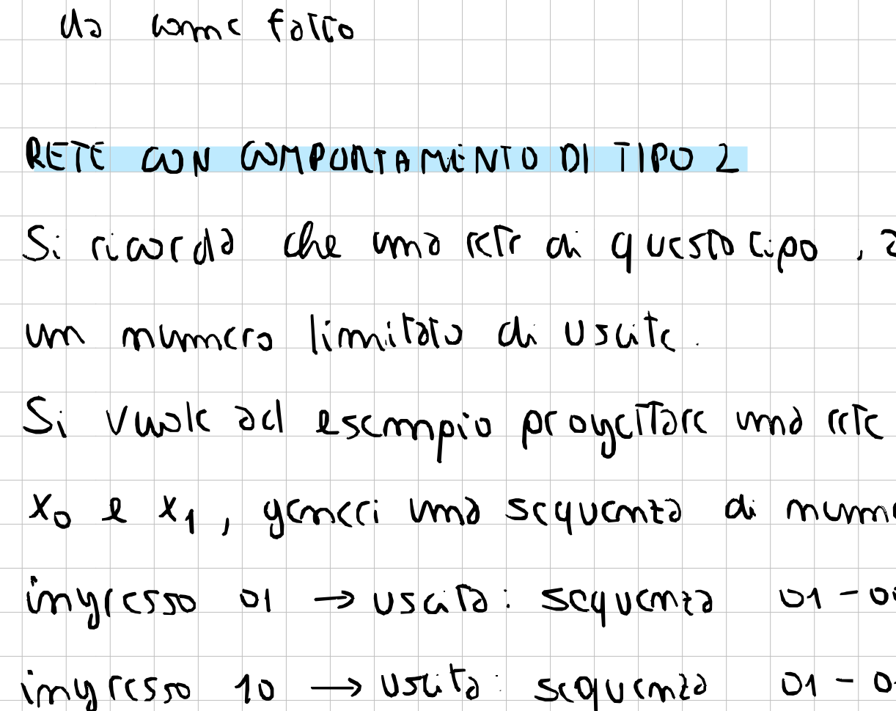
Si ricorda che una rete di questo tipo, all'arrivo di un ingresso, produce un numero limitato di uscite.
Si vuole ad esempio progettare una rete di Moore che, pres. due ingressi x_0 e x_1, generi una sequenza di numeri a 2 bit. Nello specifico
ingresso 01 → uscita: sequenza 01 - 00 - 00
ingresso 10 → uscita: sequenza 01 - 01 - 00 - 00
L'ingresso 11 non è consentito. Durante la generazione, ogni ingresso viene ignorato, eccetto 00 che annulla la sequenza. Al termine della sequenza, la rete genera 00 fino alla ricezione di ingressi 01 e 10

- dds (prima sequenza) (11 non può arrivare)



← a prescindere da cosa arrivi (eccetto 0 che annulla la sequenza) al ciclo di clock successivo B va in C, e lo stesso fa C.

- dds (prima più seconda sequenza)



- Creo la tabella

	00	01	11	10	z_1z_0
A	A	B	-	D	00
B	A	C	-	C	01
C	A	A	-	A	00
D	A	E	-	E	01
E	A	F	-	F	01
F	A	A	-	A	00

noto che due stati sono uguali →

	00	01	11	10	z_1z_0
A	A	B	-	D	00
B	A	C	-	C	01
C	A	A	-	A	00
D	A	E	-	E	01
E	A	F	-	F	01
F	A	A	-	A	00

posso dunque rimuoverne 1 e reindirizzare F a C

	00	01	11	10	z_1z_0
A	A	B	-	D	00
B	A	C	-	C	01
C	A	A	-	A	00
D	A	E	-	E	01
E	A	C	-	C	01
~~F~~	~~A~~	~~A~~	~~-~~	~~A~~	~~00~~

← ora però altre due sono uguali

	00	01	11	10	z_1z_0
A	A	B	-	D	00
B	A	C	-	C	01
C	A	A	-	A	00
D	A	E	-	E	01
E	A	C	-	C	01
~~F~~	~~A~~	~~A~~	~~-~~	~~A~~	~~00~~

↓ ri-semplifico

	00	01	11	10	z_1z_0
A	A	B	-	D	00
B	A	C	-	C	01
C	A	A	-	A	00
D	A	B	-	B	01
~~E~~	~~A~~	~~C~~	~~-~~	~~C~~	~~01~~
~~F~~	~~A~~	~~A~~	~~-~~	~~A~~	~~00~~

Ho semplificato la tabella, e di conseguenza il d.d.s (che potrei ri-disegnare, vedi slide 63 foglio 3.4)

Esiste un modo algoritmico per la semplificazione. Questo processo è detto minimizzazione degli stati

- Costruisco la tabella delle implicazioni

	00	01	11	10	z_1z_0
A	A	B	-	D	00
B	A	C	-	C	01
C	A	A	-	A	00
D	A	E	-	E	01
E	A	F	-	F	01
F	A	A	-	A	00

B					
C					
D					
E					
F					
	A	B	C	D	E

ogni coppia è univoca (c'è AB, ma non c'è BA, ...)

	00	01	11	10	z_1z_0
A	A	B	-	D	00
B	A	C	-	C	01
C	A	A	-	A	00
D	A	E	-	E	01
E	A	F	-	F	01
F	A	A	-	A	00

B					
C					
D					
E					
F					
	A	B	C	D	E

B e A hanno uscite diverse non possono essere compatibili ⇒ metto una X

B	X				
C					
D					
E					
F					
	A	B	C	D	E

Teniamo a mente A e B incompatibili

	00	01	11	10	z_1z_0
A	A	B	-	D	00
B	A	C	-	C	01
C	A	A	-	A	00
D	A	E	-	E	01
E	A	F	-	F	01
F	A	A	-	A	00

B	X				
C					
D					
E					
F					
	A	B	C	D	E

A e C hanno la stessa uscita, dunque potrebbero essere compatibili. Affinché lo siano, gli stati futuri a parità d'ingresso devono essere compatibili.

Per l'ingresso 01, A → B, C → A ⇒ stati futuri A e B incompatibili (c'è la x messa prima nella tabella) ⇒ A e C incompatibili ⇒ metto una x
Posso ripetere il passaggio per ogni coppia di stati e completare la tabella.
Ad esempio, per BD noto che potrebbero essere compatibili se lo fossero C ed E ⇒

B	X				
C	X	X			
D	X	CE			
E	X				
F	X				
	A	B	C	D	E

⇒ vado avanti, in CE noto che C ed E non sarebbero compatibili ⇒ non lo sono neanche B e D ⇒ tolgo CE e metto una X

⇒

B	X				
C	X	X			
D	X	X	X		
E	X	CF	X		
F	X				
	A	B	C	D	E

⇒ noto che BE compatibili se lo sono CF ⇒ CF lo sono ⇒ metto dei trattini

⇒

B	X				
C	X	X			
D	X	X	X		
E	X	--	X	X	
F	X	X	--	X	X
	A	B	C	D	E

↙ tutti gli altri, si può verificare, non lo sono

ritroviamo che C e F e B ed E si possono "semplificare", come visto prima ←
⇒ si sceglie come semplificarli (si "identificano delle classi di compatibilità")

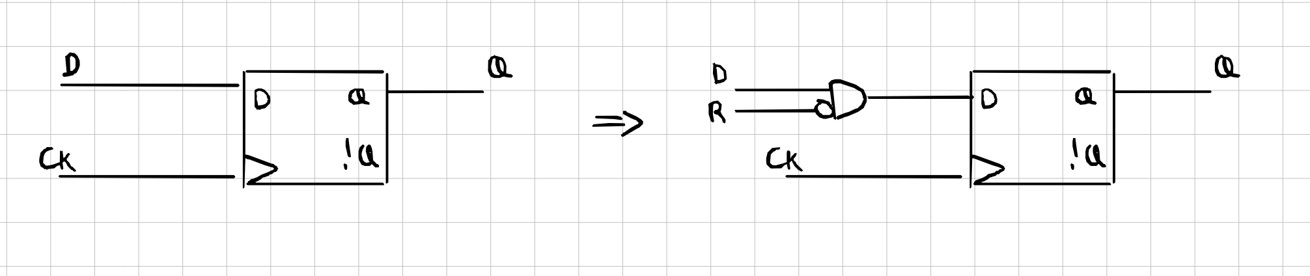
[A] ⟶ A
[B-E] ⟶ B
[C-F] ⟶ C
[D] ⟶ D

⇒ da qui si può poi ricostruire il d.d.s.

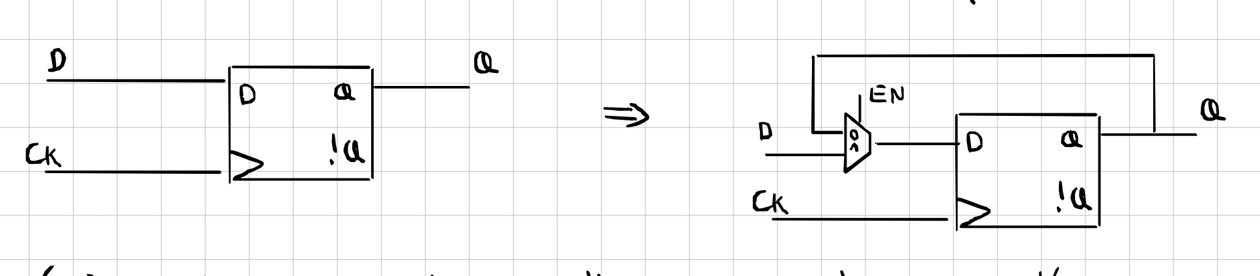
Lezione 21/11/25

Al fine di studiare sistemi più complessi, aggiungiamo delle funzionalità al FFD

RESET: se 1, consente di riportare FFD a un valore iniziale (solitamente 0)



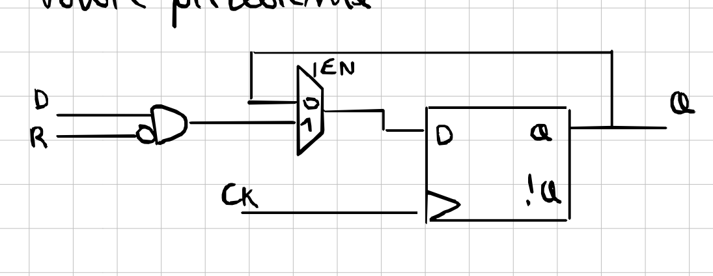
ENABLE: abilita il campionamento del FFD solo quando vale 1 (altrimenti il FFD mantiene in memoria il valore precedente)



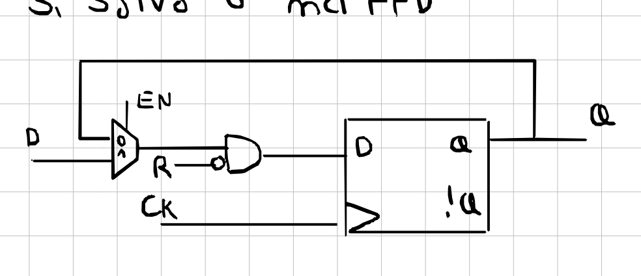
Grazie al MUX, se EN=1 allora viene selezionato l'ingresso D, altrimenti entra il vecchio ingresso (preso dalla retroazione)

Aggiungendo entrambe le funzionalità, possiamo creare due tipi di FFD:

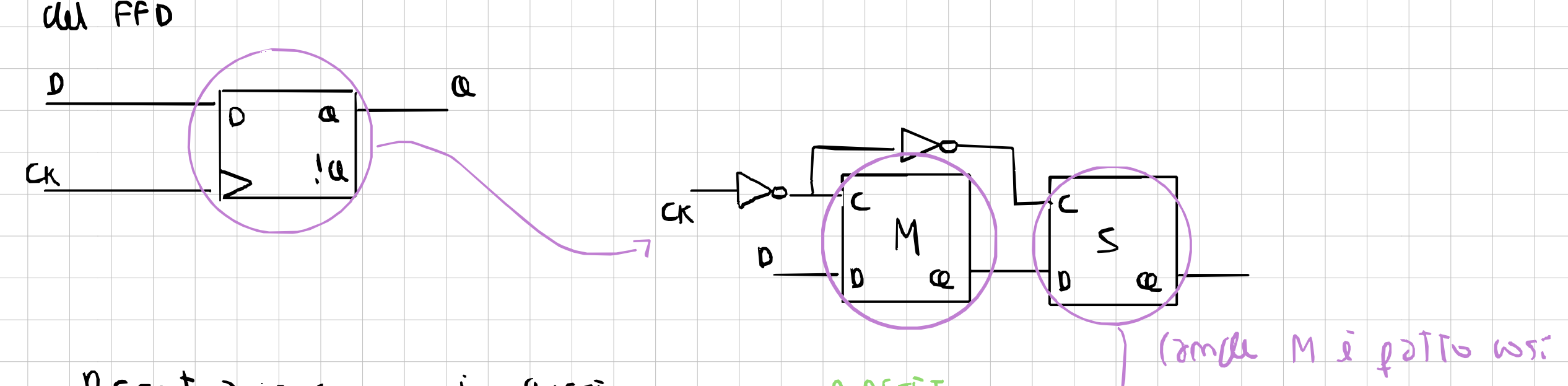
- ENABLE prioritario: anche se R=1, se ENABLE=0 il FFD mantiene il valore precedente



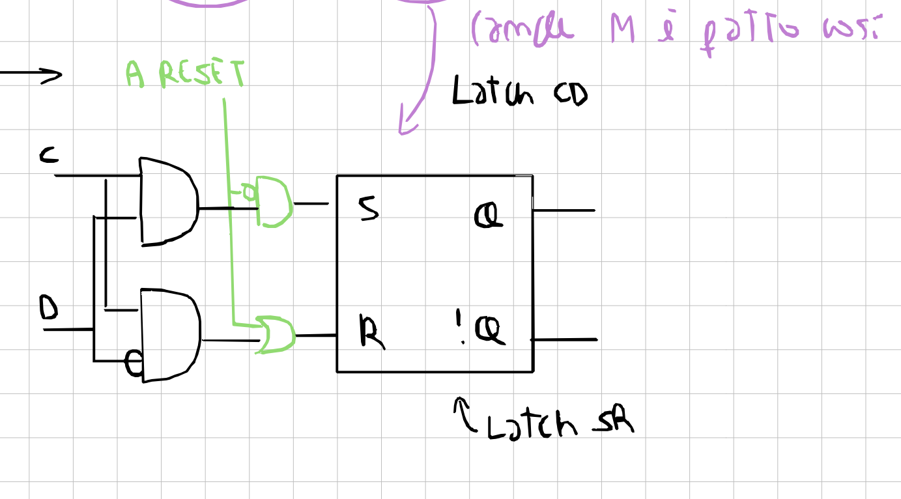
- RESET prioritario: anche se ENABLE=1, se RESET=1 D viene ignorato e si salva 0 nel FFD



Il RESET visto agisce sul fronte di salita, ovvero il segnale viene resettato alla prima salita del clock dopo il RESET. In alcuni casi è necessario applicare il RESET immediatamente, senza aspettare la salita del clock. Dunque serve un RESET asincrono. Per realizzarlo occorre modificare la struttura interna del FFD



Reset asincrono, in questo modo se il RESET=1 forza S a 0 e R a 1 ⇒ resetta il segnale (ignorando il clock in quanto agisce direttamente sul Latch) →

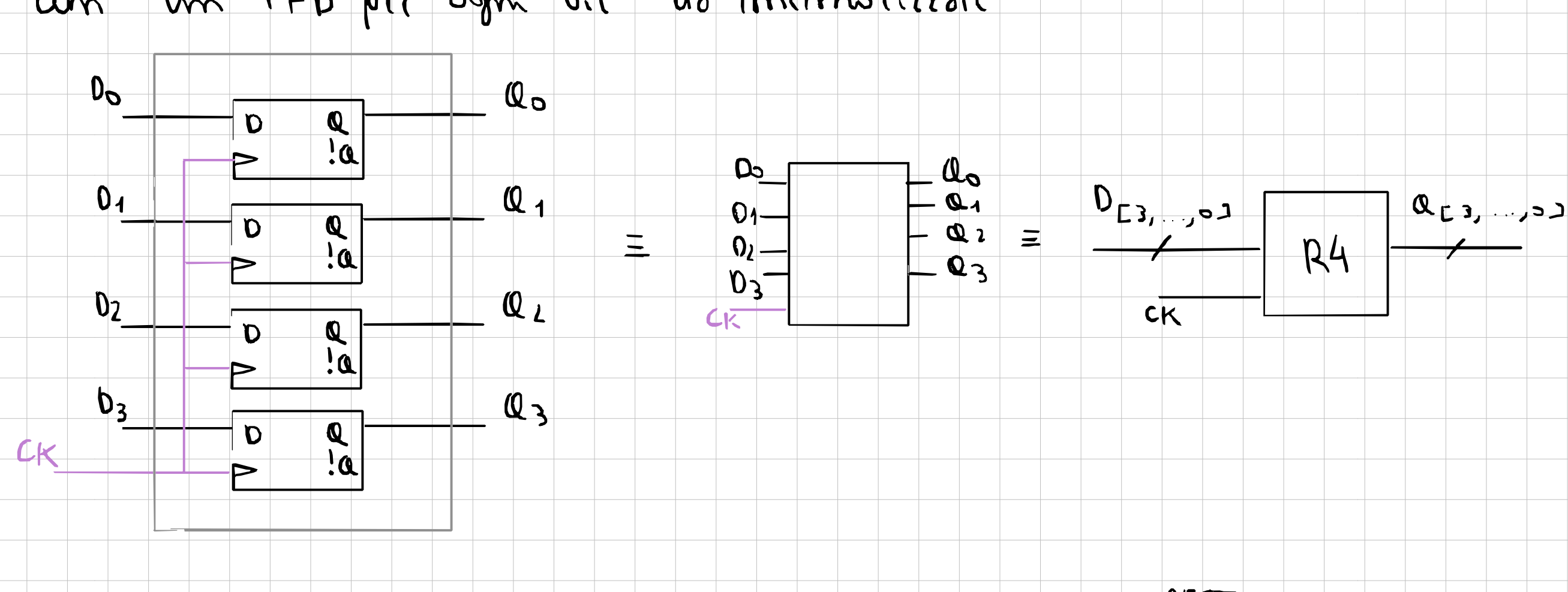


REGISTRO EDGE-TRIGGERED

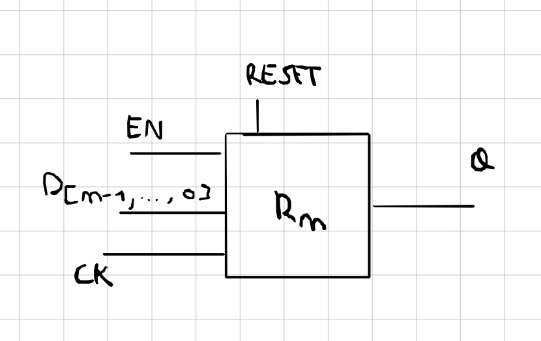
Dal momento che nei calcolatori l'unità minima di informazione considerata è il byte (8 bit) è necessario studiare componenti che operino con n bit in parallelo (n bit che "arrivano insieme").

Il registro edge-triggered è una rete che consente la memorizzazione di n bit in parallelo.

Viene intuitivamente realizzato con una "batteria di FFD" sincronizzati, con un FFD per ogni bit da memorizzare



È anche possibile progettare registri con ENABLE, RESET e RESET ASINCRONO (vedi slide 14 pagina 3.5)



È possibile dimostrare tramite sintesi e analisi (slides 20, ..., 30) quanto visto prima, ovvero che aggiungere un segnale di ENABLE a un registro edge-triggered si traduce nell'aggiungere un MUX2 all'ingresso dei FFD del registro, e aggiungere un segnale di RESET si traduce nell'aggiungere un AND all'ingresso dei FFD del registro.

Questo è vero e applicabile in qualsiasi rete, anche più complessa

REGISTRI A SCORRIMENTO (SHIFT REGISTER)

Supponiamo di voler trasmettere 256 bit.

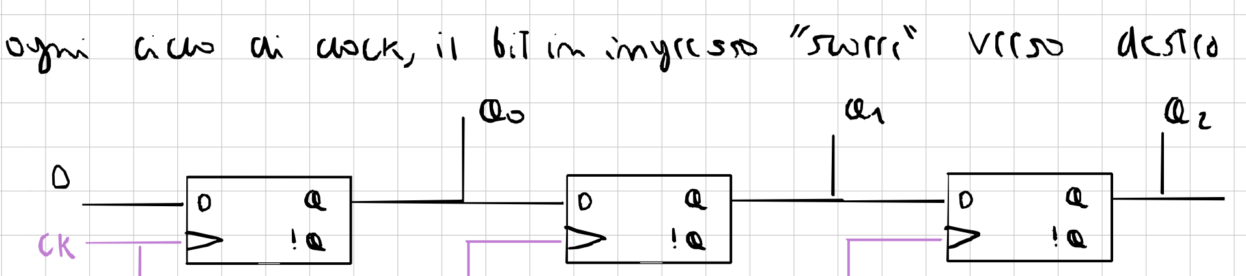
① Se li trasmetto in parallelo devo essere in grado di gestire 256 bit insieme
⇒ tanto spazio richiesto ma veloce

② Se li trasmetto uno alla volta (256 cicli di clock) devo allora gestire un solo bit alla volta (poco spazio richiesto) ma ci metto molto più tempo

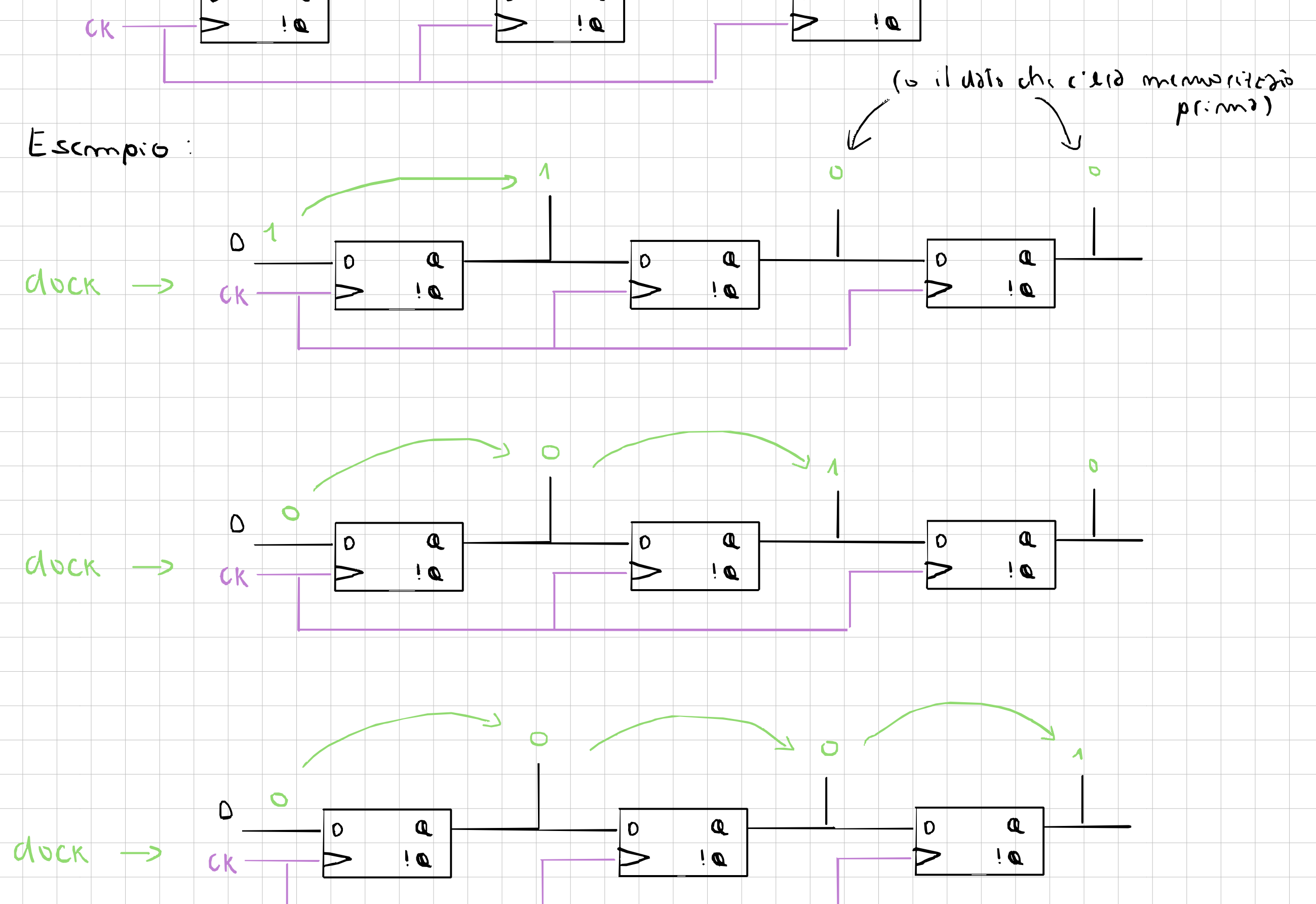
Un compromesso spazio-tempo potrebbe essere trasmettere 8 bit in parallelo in 8 cicli di clock. Lo shift register riproduce il comportamento 2.

Un registro a scorrimento si presenta come una catena di FFD in cascata.

A ogni ciclo di clock, il bit in ingresso "scorre" verso destra



Esempio:



A ogni il clock il segnale scorre verso destra. Se ad esempio arriva un segnale a 4 bit (es. 1011) in 4 cicli di clock le uscite restituiranno tale segnale, che può essere letto da un processore che lavora con dati da 4 bit. Questo appunto ottimizza lo spazio (abbiamo usato un solo filo) ma richiede tempo (4 cicli di clock). Uno registro a scorrimento è dunque in grado a tutti gli effetti di memorizzare dati seriali (esempio di gemini del cavo ethernet e del processore)

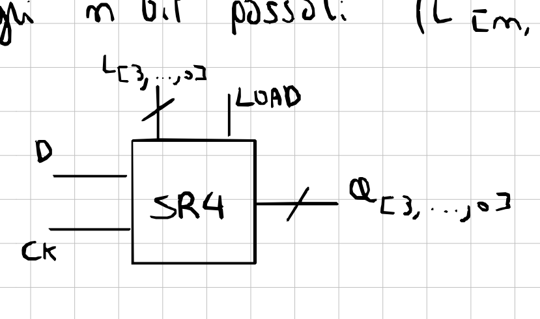
Simbologia: —/— $X_{[3,\dots,0]}$ = dati in parallelo

x —— = un dato alla volta

Uno shift-register può essere costruito in diversi modi (SISO, PIPO, PISO, SIPO).

Inoltre, si può anche

- Aggiungere un segnale di LOAD, che abilita il caricamento di n bit alla volta, rispettivamente negli n FFD ⇒ salvataggio in parallelo (carica gli n bit passati ($L_{[n,\dots,0]}$) dentro agli n FLIP-FLOP



- Aggiungere un segnale L/R (Left/Right). Se tale segnale è 0, lo spostamento avviene verso sinistra, altrimenti verso destra.

Dunque i registri sono reti che permettono la memorizzazione di più di un bit alla volta. Ad esempio, un processore potrebbe voler trattare 8 bit alla volta ⇒ si può usare un registro edge-triggered. Se i bit vengono ad esempio uno dietro l'altro (es. da un cavo ethernet) per essere usati dalla CPU dovremmo arrivare 8 alla volta ⇒ in 8 cicli di clock, uno shift-register può memorizzare 8 bit e poi trasmetterli insieme volendo.

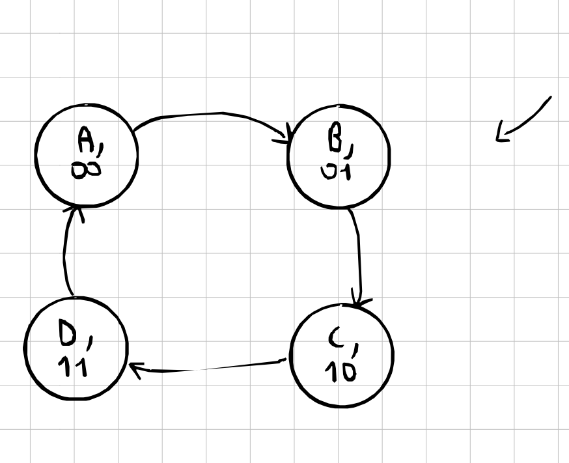
RETI SINCRONE E COMPORTAMENTI DI TIPO 3

Ricordiamo che questo comportamento descrive una rete che, all'arrivo di un ingresso, produce un numero illimitato di uscite.

Un esempio sono i contatori binari, reti di Moore che:

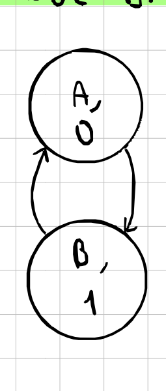
- hanno un diagramma degli stati ad anello, in cui ogni stato codifica un valore del conteggio ES: 00, 01, 10, 11, 00, 01, ...
- nella loro forma base non hanno ingressi ma iterano all'infinito automaticamente (detta rete free-running)
- il numero di stati determina la capacità massima del contatore (es. contatore da 0 (00) a 3 (11) ⇒ 4 bit ⇒ 4 stati.

ES. a 4 bit

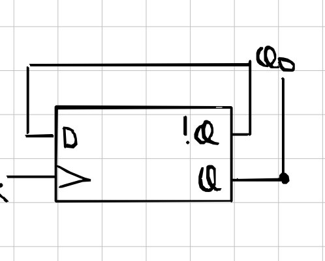


no ingressi, itera all'infinito

A DUE BIT (BC2)



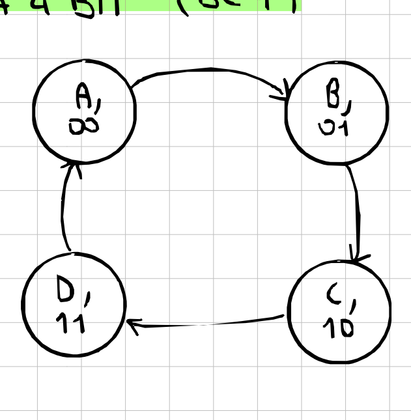
Schematico:



$Y_0 = !y_0$

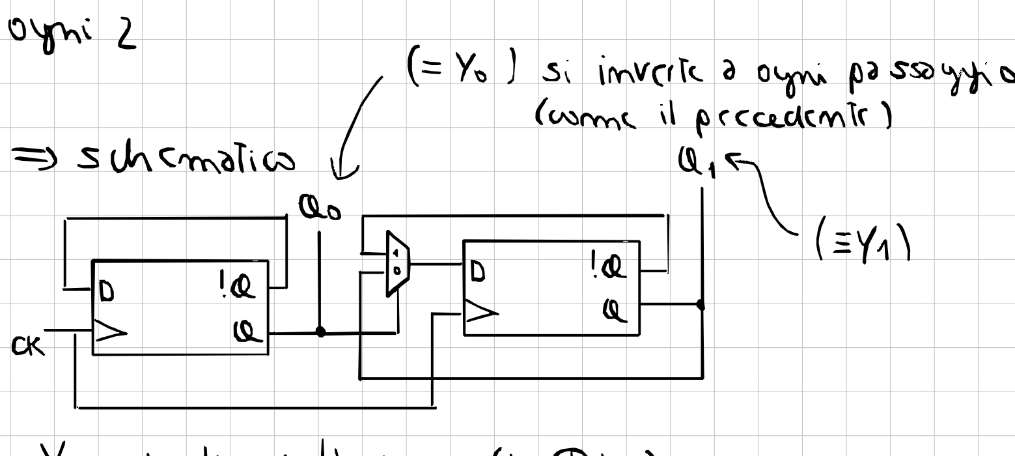
← Ad ogni ciclo di clock la variabile di stato si inverte (0→1→0→...)

A 4 BIT (BC4)



y_0 si inverte in ogni passaggio di stato, y_1 si inverte ogni 2

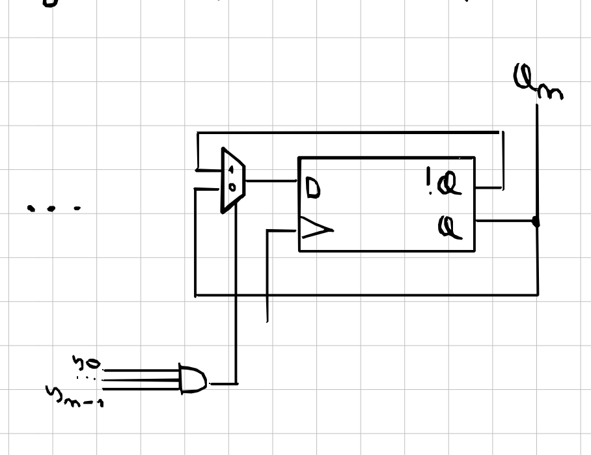
⇒ schematico



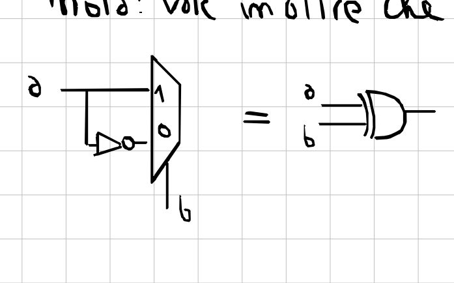
$Y_1 = y_1 !y_0 + !y_1 y_0$ ($y_1 \oplus y_0$)

BC2^n

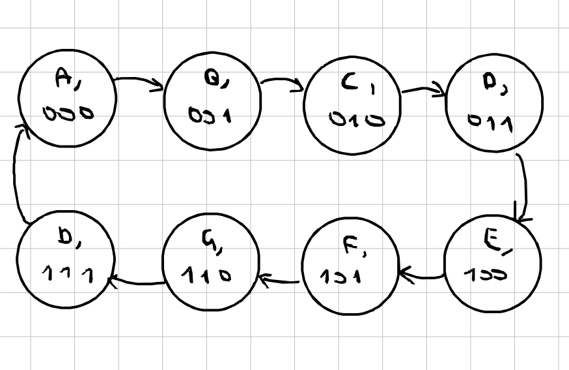
In generale, un BC2^n può essere realizzato come una catena di:



Nota: vale inoltre che:



ES. sintesi di un BC8



→ TdT (e codifica stati)

	$y_2y_1y_0$		Z
A	000	001	000
B	001	010	001
C	010	011	010
D	011	100	011
E	100	101	100
F	101	110	101
G	110	111	110
H	111	000	111

con questa codifica le variabili di stato corrispondono alla rispettiva uscita ⇒ non c'è bisogno della mappa delle uscite

Mappa di Karnaugh

y_1y_0 \ y_2	0	1
00	1	1
01	0	0
11	0	0
10	1	1

Y_0

y_1y_0 \ y_2	0	1
00	0	0
01	1	1
11	0	0
10	1	1

Y_1

y_1y_0 \ y_2	0	1
00	0	1
01	0	1
11	1	0
10	0	1

Y_2 ←

nota: le mappe di Karnaugh richiedono sempre l'adiacenza tra variabili

es:

00	. . . .
01	. — . .
11	— — . .
10	— .

| espressioni

y_1y_0 \ y_2	0	1
00	1	1
01	0	0
11	0	0
10	1	1

Y_0

y_1y_0 \ y_2	0	1
00	0	0
01	1	1
11	0	0
10	1	1

Y_1

y_1y_0 \ y_2	0	1
00	0	1
01	0	1
11	1	0
10	0	1

Y_2

MUX2 controllato da $y_1 y_0$

de Morgan ⇒ $y_2 !(y_1 y_0) + !y_2 (y_1 y_0)$

$Y_0 = !y_0$ $Y_1 = !y_1 y_0 + !y_0 y_1$ $Y_2 = !y_1 y_2 + y_1 y_0 !y_2 + !y_0 y_2$

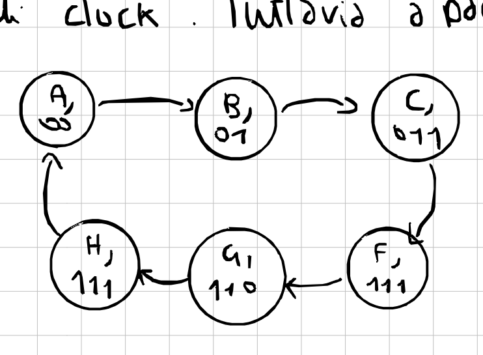
↳ MUX2 controllato da y_0

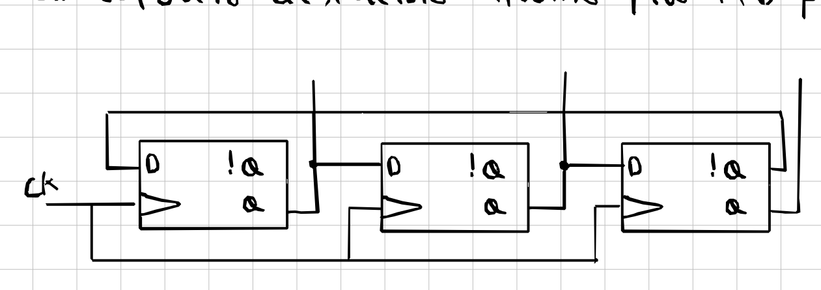
È anche possibile realizzare contatori asincroni (clock non sincronizzato).

Definizione: clock-gating = "alterare il segnale di clock"

Esistono anche:

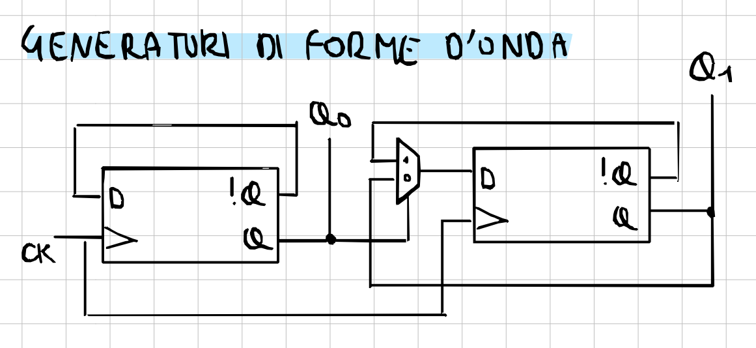
- Contatori UP/DOWN, che permettono il conteggio crescente o decrescente (versi delle frecce nei D.D.S. invertiti)
- Contatori di Johnson (JC), un contatore a riempimento/svuotamento che "aggiunge 1" in ogni FFD, e poi riporta tutti gli 1 (l'ultimo numero del contatore) a 0 (il primo numero del contatore). Si realizza con un anello chiuso di FFD. Non avendo gate, il ritardo è minimo ⇒ massima frequenza di clock. Tuttavia a parità di capacità desiderata servono più FFD per la realizzazione





↳ con n FFD si può contare fino a 2n valori (non 2^n come visto precedentemente)

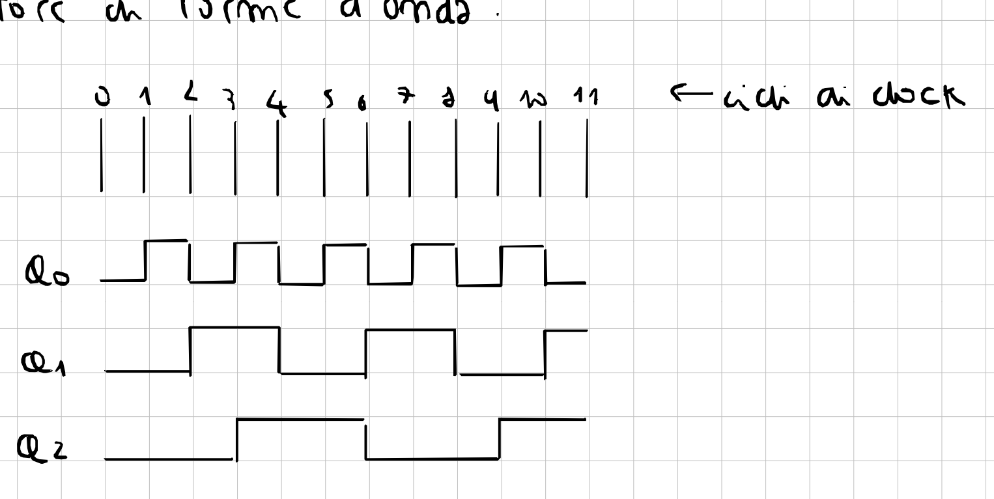
GENERATORI DI FORME D'ONDA



← Contatore BC4

(commuta = cambia valore)

Abbiamo visto che Q0 cambia valore ogni ciclo di clock, Q1 ogni 2. Allo stesso modo, in un BC8 ($BC2^3$) avremmo un uscita Q2 che commuta ogni 8 cicli di clock, e così via. Dunque un contatore è anche un generatore di forme d'onda:



...